

HelixTerminator — Client-Side Technical Specification

Document Version: 1.0.0

Date: 2026-06-28

Package: `io.helixterminator.client`

Technology Stack: Flutter 3.24 / Dart 3.x

Platforms: Web (WASM), macOS, Windows, Linux, iOS, Android

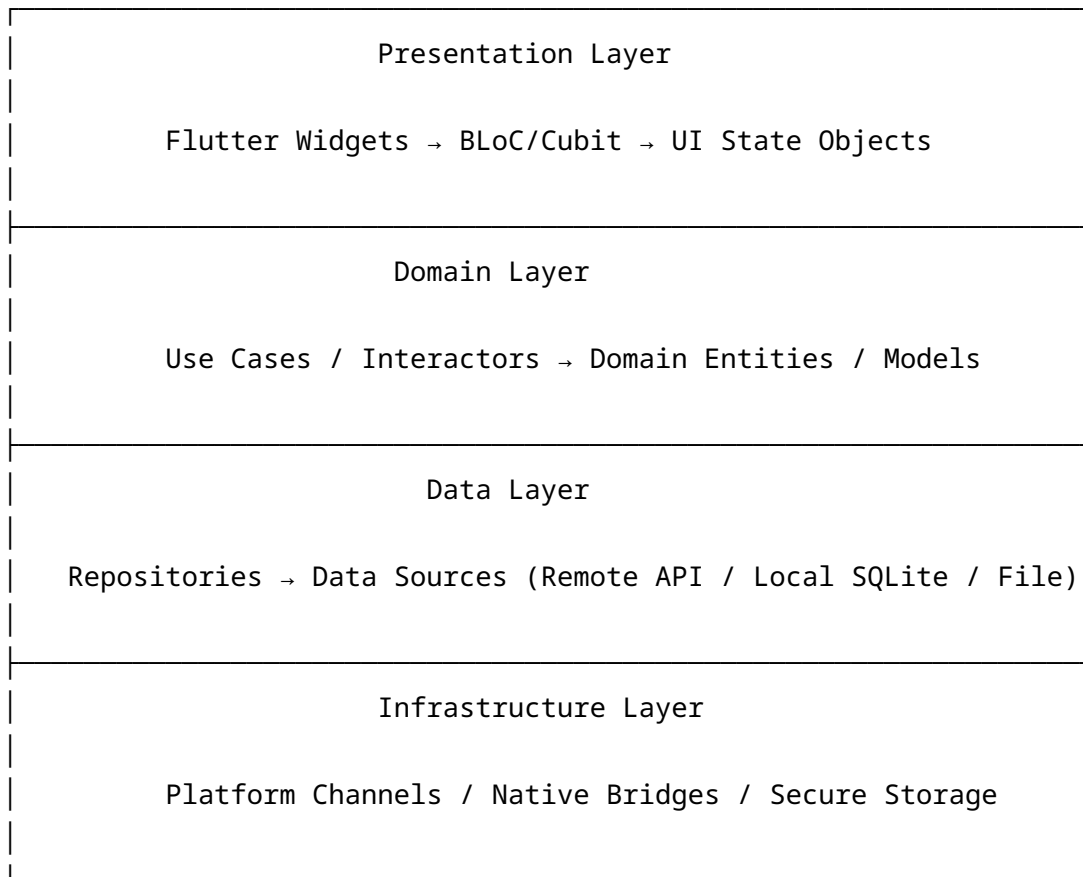
Table of Contents

1. [Flutter Architecture](#)
 2. [Terminal Emulator Design](#)
 3. [SSH Connection Architecture](#)
 4. [Offline Mode](#)
 5. [Security on Client](#)
 6. [UI/UX Complete Specification](#)
 7. [Performance Targets](#)
 8. [Platform-Specific Features](#)
 9. [Testing Strategy](#)
 10. [Accessibility](#)
 11. [Auto-Update Mechanism](#)
-

1. Flutter Architecture

1.1 Architectural Overview

HelixTerminator's client-side architecture follows a strict layered BLoC (Business Logic Component) pattern with unidirectional data flow. Every platform shares the same Dart business logic; only the presentation layer diverges for platform-specific adaptations.



Core Principle: UI widgets never call data sources directly. Every widget observes a BLoC or Cubit. Every BLoC calls a repository interface (not an implementation). The concrete repository implementation is injected via `get_it`.

1.2 Project Structure

```
lib/
├── core/
│   ├── di/                                # Dependency injection
│   │   ├── injection_container.dart
│   │   └── injection_container.config.dart # Generated
│   ├── navigation/
│   │   ├── app_router.dart                # go_router configuration
│   │   └── route_names.dart
│   ├── network/
│   │   ├── dio_client.dart
│   │   ├── interceptors/
│   │   │   ├── auth_interceptor.dart
│   │   │   ├── logging_interceptor.dart
│   │   │   └── retry_interceptor.dart
│   │   └── api_endpoints.dart
```

```
|   |─ storage/
|   |   |─ database/
|   |   |   |─ app_database.dart    # drift database definition
|   |   |   |─ app_database.g.dart # Generated
|   |   |   └─ tables/
|   |   └─ secure_storage_service.dart
|   └─ error/
|       |─ failures.dart
|       └─ exceptions.dart
|   └─ usecases/
|       └─ use_case.dart          # Abstract base class
|   └─ utils/
|       |─ constants.dart
|       └─ extensions/
|─ features/
|   |─ auth/
|   |─ vault/
|   |─ hosts/
|   |─ terminal/
|   |─ ssh_session/
|   |─ sftp/
|   |─ port_forwarding/
|   |─ workspace/
|   |─ snippets/
|   |─ keychain/
|   |─ collaboration/
|   |─ ai_autocomplete/
|   |─ session_logs/
|   |─ settings/
|   |─ organizations/
|   |─ audit/
|   |─ known_hosts/
|   └─ notifications/
|─ platform/
|   |─ biometrics/
|   |   |─ biometrics_platform_interface.dart
|   |   |─ biometrics_method_channel.dart
|   |   └─ biometrics_service.dart
|   |─ hardware_key/
|   |   |─ fido2_platform_interface.dart
|   |   └─ fido2_method_channel.dart
|   └─ ssh_agent/
|       └─ ssh_agent_platform_interface.dart
```

```
|      └─ ssh_agent_method_channel.dart
└─ app.dart
```

Each feature module follows an identical internal structure:

```
features/<feature>/
├─ data/
│   ├── datasources/
│   │   ├── <feature>_remote_data_source.dart
│   │   └─ <feature>_local_data_source.dart
│   ├── models/
│   │   └─ <feature>_model.dart          # JSON-serializable DTO
│   └─ repositories/
│       └─ <feature>_repository_impl.dart
├─ domain/
│   ├── entities/
│   │   └─ <feature>_entity.dart        # Pure domain object
│   ├── repositories/
│   │   └─ <feature>_repository.dart    # Abstract interface
│   └─ usecases/
│       ├── get_<feature>.dart
│       ├── create_<feature>.dart
│       └─ delete_<feature>.dart
└─ presentation/
    ├── bloc/
    │   ├── <feature>_bloc.dart
    │   ├── <feature>_event.dart
    │   └─ <feature>_state.dart
    ├── pages/
    │   └─ <feature>_page.dart
    └─ widgets/
        └─ <feature>_widget.dart
```

1.3 Dependency Injection with get_it + injectable

```
// core/di/injection_container.dart
import 'package:get_it/get_it.dart';
import 'package:injectable/injectable.dart';
import 'injection_container.config.dart';

final GetIt getIt = GetIt.instance;

@InjectableInit(
  initializerName: 'init',
```

```

    preferRelativeImports: true,
    asExtension: true,
  )
Future<void> configureDependencies(String environment) async =>
    getIt.init(environment: environment);

// core/di/environments.dart
abstract class Env {
  static const String dev = 'dev';
  static const String staging = 'staging';
  static const String prod = 'prod';
  static const String test = 'test';
}

// Example: ssh_session feature registration
@module
abstract class SshSessionModule {
  @lazySingleton
  SshConnectionPool get connectionPool => SshConnectionPool(
    maxConnections: 50,
    keepAliveInterval: const Duration(seconds: 30),
  );

  @lazySingleton
  SshSessionRepository get repository => SshSessionRepositoryImpl(
    remoteDataSource: getIt(),
    localDataSource: getIt(),
    connectionPool: getIt(),
  );
}

```

1.4 Navigation with go_router

```

// core/navigation/app_router.dart
import 'package:flutter/material.dart';
import 'package:go_router/go_router.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

@singleton
class AppRouter {
  final AuthBloc _authBloc;
  AppRouter(this._authBloc);

  late final GoRouter router = GoRouter(

```

```

debugLogDiagnostics: true,
initialLocation: RouteNames.splash,
refreshListenable: GoRouterRefreshStream(_authBloc.stream),
redirect: _redirect,
routes: [
  GoRoute(
    path: RouteNames.splash,
    builder: (_, __) => const SplashPage(),
  ),
  GoRoute(
    path: RouteNames.login,
    builder: (_, __) => const LoginPage(),
    routes: [
      GoRoute(
        path: 'mfa',
        builder: (_, state) => MfaPage(
          challenge: state.extra as MfaChallenge,
        ),
      ),
      GoRoute(
        path: 'sso',
        builder: (_, state) => SsoPage(
          provider: state.uri.queryParameters['provider'] ??
'',
        ),
      ),
    ],
  ),
  ShellRoute(
    builder: (context, state, child) => AppShell(child:
child),
    routes: [
      GoRoute(
        path: RouteNames.hosts,
        builder: (_, __) => const HostListPage(),
        routes: [
          GoRoute(
            path: ':hostId',
            builder: (_, state) => HostDetailPage(
              hostId: state.pathParameters['hostId']!,
            ),
          ),
        ],
      ),
      GoRoute(

```

```

        path: RouteNames.terminal,
        builder: (_, state) => TerminalPage(
            sessionId: state.uri.queryParameters['session'],
        ),
    ),
    GoRoute(
        path: RouteNames.sftp,
        builder: (_, state) => SftpBrowserPage(
            sessionId: state.uri.queryParameters['session']!,
        ),
    ),
    GoRoute(
        path: RouteNames.workspace,
        builder: (_, __) => const WorkspacePage(),
    ),
    GoRoute(
        path: RouteNames.portForwarding,
        builder: (_, __) => const PortForwardingPage(),
    ),
    GoRoute(
        path: RouteNames.snippets,
        builder: (_, __) => const SnippetLibraryPage(),
    ),
    GoRoute(
        path: RouteNames.keychain,
        builder: (_, __) => const KeychainManagerPage(),
    ),
    GoRoute(
        path: RouteNames.settings,
        builder: (_, __) => const SettingsPage(),
    ),
    GoRoute(
        path: RouteNames.sessionLogs,
        builder: (_, __) => const SessionLogsPage(),
    ),
    GoRoute(
        path: RouteNames.audit,
        builder: (_, __) => const AuditLogPage(),
    ),
],
),
],
);

```

```

String? _redirect(BuildContext context, GoRouterState state) {
  final authState = _authBloc.state;
  final isLoggingIn = state.matchedLocation == RouteNames.login
    ||
    state.matchedLocation.startsWith('/login/');

  if (authState is AuthUnauthenticated && !isLoggingIn) {
    return RouteNames.login;
  }
  if (authState is AuthAuthenticated && isLoggingIn) {
    return RouteNames.hosts;
  }
  return null;
}
}

```

1.5 Local Database with drift (SQLite)

```

// core/storage/database/app_database.dart
import 'package:drift/drift.dart';
import 'package:drift/native.dart';

part 'app_database.g.dart';

// --- Tables ---

class Hosts extends Table {
  IntColumn get id => integer().autoIncrement(());
  TextColumn get remoteId => text().withLength(min: 36, max: 36)
    ();
  TextColumn get label => text().withLength(min: 1, max: 255)();
  TextColumn get hostname => text()();
  IntColumn get port => integer().withDefault(const Constant(22))
    ();
  TextColumn get username => text()();
  TextColumn get protocol => textEnum<HostProtocol>()();
  TextColumn get groupId => text().nullable()();
  TextColumn get tags => text().withDefault(const Constant('[]'))
    ();
  TextColumn get credentialId => text().nullable()();
  DateTimeColumn get createdAt => dateTime()();
  DateTimeColumn get updatedAt => dateTime()();
  TextColumn get syncEtag => text().nullable()();
  BoolColumn get isDirty => boolean().withDefault(const
    Constant(false))();
}

```



```

    BoolColumn get isDeleted => boolean().withDefault(const
        Constant(false))();
}

class HostGroups extends Table {
    IntColumn get id => integer().autoIncrement();
    TextColumn get remoteId => text().withLength(min: 36, max: 36)
        ();
    TextColumn get name => text();
    TextColumn get color => text().withDefault(const
        Constant('#4A90D9'))();
    IntColumn get parentId => integer().nullable();
    DateTimeColumn get createdAt => dateTime();
    DateTimeColumn get updatedAt => dateTime();
}

class SshKeys extends Table {
    IntColumn get id => integer().autoIncrement();
    TextColumn get remoteId => text().withLength(min: 36, max: 36)
        ();
    TextColumn get label => text();
    TextColumn get keyType => textEnum<SshKeyType>();
    TextColumn get publicKey => text();
    // Private key is stored in flutter_secure_storage, never in
    // SQLite
    BoolColumn get isProtectedByPassphrase => boolean();
    BoolColumn get isHardwareKey => boolean();
    DateTimeColumn get createdAt => dateTime();
    DateTimeColumn get updatedAt => dateTime();
}

class Snippets extends Table {
    IntColumn get id => integer().autoIncrement();
    TextColumn get remoteId => text().withLength(min: 36, max: 36)
        ();
    TextColumn get title => text();
    TextColumn get content => text();
    TextColumn get language => text().nullable();
    TextColumn get tags => text().withDefault(const Constant('[]'))
        ();
    TextColumn get folderId => text().nullable();
    DateTimeColumn get createdAt => dateTime();
    DateTimeColumn get updatedAt => dateTime();
    BoolColumn get isDirty => boolean().withDefault(const
        Constant(false))();
}

```

```

class SessionLogs extends Table {
  IntColumn get id => integer().autoIncrement();
  TextColumn get sessionId => text().withLength(min: 36, max: 36)
    ();
  TextColumn get hostId => text();
  TextColumn get hostLabel => text();
  TextColumn get username => text();
  DateTimeColumn get connectedAt => dateTime();
  DateTimeColumn get disconnectedAt => dateTime().nullable();
  IntColumn get durationSeconds => integer().nullable();
  TextColumn get disconnectReason => text().nullable();
  TextColumn get logFilePath => text().nullable();
}

```

```

class KnownHosts extends Table {
  IntColumn get id => integer().autoIncrement();
  TextColumn get hostname => text();
  IntColumn get port => integer().withDefault(const Constant(22))
    ();
  TextColumn get keyType => text();
  TextColumn get fingerprint => text();
  DateTimeColumn get addedAt => dateTime();
  BoolColumn get trusted => boolean().withDefault(const
    Constant(true));
}

```

```

class PortForwardingRules extends Table {
  IntColumn get id => integer().autoIncrement();
  TextColumn get remoteId => text().withLength(min: 36, max: 36)
    ();
  TextColumn get label => text();
  TextColumn get hostId => text();
  TextColumn get ruleType => textEnum<PortForwardType>();
  TextColumn get bindAddress => text().withDefault(const
    Constant('127.0.0.1'));
  IntColumn get localPort => integer();
  TextColumn get remoteHost => text();
  IntColumn get remotePort => integer();
  BoolColumn get isActive => boolean().withDefault(const
    Constant(false));
  BoolColumn get autoStart => boolean().withDefault(const
    Constant(false));
}

```

```

class AppSettings extends Table {
  TextColumn get key => text();
  TextColumn get value => text();
}

```

```

    @override
    Set<Column> get primaryKey => {key};
}

// --- Database ---

@DriftDatabase(
  tables: [
    Hosts,
    HostGroups,
    SshKeys,
    Snippets,
    SessionLogs,
    KnownHosts,
    PortForwardingRules,
    AppSettings,
  ],
)
class AppDatabase extends _$AppDatabase {
  AppDatabase(QueryExecutor e) : super(e);

  @override
  int get schemaVersion => 1;

  @override
  MigrationStrategy get migration {
    return MigrationStrategy(
      onCreate: (Migrator m) async {
        await m.createAll();
      },
      onUpgrade: (Migrator m, int from, int to) async {
        // Migrations handled per version
      },
    );
  }
}

enum HostProtocol { ssh, mosh, telnet }
enum SshKeyType { rsa2048, rsa4096, ed25519, ecdsaP256,
  ecdsaP521 }
enum PortForwardType { local, remote, dynamic }

```

1.6 HTTP Client with Dio

```
// core/network/dio_client.dart
import 'package:dio/dio.dart';
import 'package:injectable/injectable.dart';

@singleton
class DioClient {
  late final Dio _dio;

  DioClient(
    AppConfig config,
    AuthTokenRepository tokenRepo,
    ConnectivityService connectivity,
  ) {
    _dio = Dio(
      BaseOptions(
        baseUrl: config.apiBaseUrl,
        connectTimeout: const Duration(seconds: 10),
        receiveTimeout: const Duration(seconds: 30),
        sendTimeout: const Duration(seconds: 30),
        headers: {
          'Content-Type': 'application/json',
          'X-Client-Platform': _platformString(),
          'X-Client-Version': config.version,
        },
      ),
    );

    _dio.interceptors.addAll([
      AuthInterceptor(tokenRepo, _dio),
      LoggingInterceptor(),
      RetryInterceptor(
        dio: _dio,
        retries: 3,
        retryDelays: const [
          Duration(seconds: 1),
          Duration(seconds: 2),
          Duration(seconds: 5),
        ],
      ),
      ConnectivityInterceptor(connectivity),
    ]);
  }
}
```

```

    Dio get instance => _dio;
  }

// core/network/interceptors/auth_interceptor.dart
class AuthInterceptor extends Interceptor {
  final AuthTokenRepository _tokenRepo;
  final Dio _dio;
  bool _isRefreshing = false;
  final List<RequestOptions> _retryQueue = [];

  AuthInterceptor(this._tokenRepo, this._dio);

  @override
  void onRequest(
    RequestOptions options,
    RequestInterceptorHandler handler,
  ) async {
    final token = await _tokenRepo.getAccessToken();
    if (token != null) {
      options.headers['Authorization'] = 'Bearer $token';
    }
    handler.next(options);
  }

  @override
  void onError(DioException err, ErrorInterceptorHandler handler)
    async {
    if (err.response?.statusCode == 401 && !_isRefreshing) {
      _isRefreshing = true;
      try {
        final refreshed = await _tokenRepo.refreshAccessToken();
        if (refreshed) {
          // Retry all queued requests
          for (final options in _retryQueue) {
            final token = await _tokenRepo.getAccessToken();
            options.headers['Authorization'] = 'Bearer $token';
            await _dio.fetch(options);
          }
          _retryQueue.clear();
          // Retry current request
          final token = await _tokenRepo.getAccessToken();
          err.requestOptions.headers['Authorization'] = 'Bearer $token';
          final response = await _dio.fetch(err.requestOptions);

```

```

        handler.resolve(response);
        return;
    }
} catch (_) {
    // Refresh failed – force logout
    _tokenRepo.clearTokens();
} finally {
    _isRefreshing = false;
}
}
handler.next(err);
}
}

```

1.7 Feature Module: auth

The auth module manages the complete authentication lifecycle including login, MFA, biometrics, and SSO.

```

// features/auth/domain/entities/auth_user.dart
class AuthUser {
    final String id;
    final String email;
    final String displayName;
    final String? avatarUrl;
    final AuthRole role;
    final List<MfaMethod> enabledMfaMethods;
    final BiometricStatus biometricStatus;
    final DateTime lastLoginAt;
    final String? organizationId;

    const AuthUser({
        required this.id,
        required this.email,
        required this.displayName,
        this.avatarUrl,
        required this.role,
        required this.enabledMfaMethods,
        required this.biometricStatus,
        required this.lastLoginAt,
        this.organizationId,
    });
}

```

```

// Canonical RBAC vocabulary (CD-8): super_admin, org_admin,
    team_admin,
// member, auditor, api_user – normalized to Dart camelCase.
enum AuthRole { superAdmin, orgAdmin, teamAdmin, member, auditor,
    apiUser }
enum MfaMethod { totp, fido2, push, backupCodes }
enum BiometricStatus { notEnrolled, enrolled, disabled }

// features/auth/presentation/bloc/auth_bloc.dart
import 'package:flutter_bloc/flutter_bloc.dart';

// Events
abstract class AuthEvent {}

class AuthLoginRequested extends AuthEvent {
    final String email;
    final String password;
    AuthLoginRequested({required this.email, required
        this.password});
}

class AuthMfaSubmitted extends AuthEvent {
    final String code;
    final MfaMethod method;
    AuthMfaSubmitted({required this.code, required this.method});
}

class AuthSsoLoginRequested extends AuthEvent {
    final String provider; // 'google', 'github', 'okta', 'azure_ad'
    AuthSsoLoginRequested({required this.provider});
}

class AuthBiometricLoginRequested extends AuthEvent {}

class AuthLogoutRequested extends AuthEvent {}

class AuthTokenRefreshRequired extends AuthEvent {}

class AuthSessionRestored extends AuthEvent {
    final AuthUser user;
    AuthSessionRestored({required this.user});
}

// States
abstract class AuthState {}

```

```

class AuthInitial extends AuthState {}

class AuthLoading extends AuthState {}

class AuthEmailPasswordPending extends AuthState {
    // User submitted credentials, waiting for MFA or success
}

class AuthMfaRequired extends AuthState {
    final List<MfaMethod> availableMethods;
    final MfaMethod defaultMethod;
    final MfaChallenge challenge;

    AuthMfaRequired({
        required this.availableMethods,
        required this.defaultMethod,
        required this.challenge,
    });
}

class AuthBiometricPromptActive extends AuthState {}

class AuthAuthenticated extends AuthState {
    final AuthUser user;
    AuthAuthenticated({required this.user});
}

class AuthUnauthenticated extends AuthState {}

class AuthError extends AuthState {
    final String message;
    final AuthErrorCode code;

    AuthError({required this.message, required this.code});
}

enum AuthErrorCode {
    invalidCredentials,
    accountLocked,
    mfaFailed,
    networkError,
    biometricFailed,
    sessionExpired,
}

```



```

// BLoC
@injectable
class AuthBloc extends Bloc<AuthEvent, AuthState> {
  final LoginUseCase _login;
  final SubmitMfaUseCase _submitMfa;
  final SsoLoginUseCase _ssoLogin;
  final BiometricAuthUseCase _biometricAuth;
  final LogoutUseCase _logout;
  final RestoreSessionUseCase _restoreSession;

  AuthBloc({
    required LoginUseCase login,
    required SubmitMfaUseCase submitMfa,
    required SsoLoginUseCase ssoLogin,
    required BiometricAuthUseCase biometricAuth,
    required LogoutUseCase logout,
    required RestoreSessionUseCase restoreSession,
  }) : _login = login,
      _submitMfa = submitMfa,
      _ssoLogin = ssoLogin,
      _biometricAuth = biometricAuth,
      _logout = logout,
      _restoreSession = restoreSession,
      super(AuthInitial()) {
    on<AuthLoginRequested>(_onLoginRequested);
    on<AuthMfaSubmitted>(_onMfaSubmitted);
    on<AuthSsoLoginRequested>(_onSsoLoginRequested);
    on<AuthBiometricLoginRequested>(_onBiometricLoginRequested);
    on<AuthLogoutRequested>(_onLogoutRequested);
    on<AuthSessionRestored>(_onSessionRestored);
  }

  Future<void> _onLoginRequested(
    AuthLoginRequested event,
    Emitter<AuthState> emit,
  ) async {
    emit(AuthLoading());
    final result = await _login(LoginParams(
      email: event.email,
      password: event.password,
    ));
    result.fold(
      (failure) => emit(AuthError(

```

```

        message: failure.message,
        code: _mapFailureToCode(failure),
    )),
    (loginResult) {
        if (loginResult.requiresMfa) {
            emit(AuthMfaRequired(
                availableMethods: loginResult.mfaMethods,
                defaultMethod: loginResult.mfaMethods.first,
                challenge: loginResult.challenge!,
            ));
        } else {
            emit(AuthAuthenticated(user: loginResult.user!));
        }
    },
);
}

```

```

Future<void> _onMfaSubmitted(
    AuthMfaSubmitted event,
    Emitter<AuthState> emit,
) async {
    emit(AuthLoading());
    final result = await _submitMfa(MfaParams(
        code: event.code,
        method: event.method,
    ));
    result.fold(
        (failure) => emit(AuthError(
            message: failure.message,
            code: AuthErrorCode.mfaFailed,
        )),
        (user) => emit(AuthAuthenticated(user: user)),
    );
}

```

```

Future<void> _onBiometricLoginRequested(
    AuthBiometricLoginRequested event,
    Emitter<AuthState> emit,
) async {
    emit(AuthBiometricPromptActive());
    final result = await _biometricAuth(NoParams());
    result.fold(
        (failure) => emit(AuthError(
            message: failure.message,

```

```

        code: AuthErrorCode.biometricFailed,
      )),
      (user) => emit(AuthAuthenticated(user: user)),
    );
  }

Future<void> _onLogoutRequested(
  AuthLogoutRequested event,
  Emitter<AuthState> emit,
) async {
  await _logout(NoParams());
  emit(AuthUnauthenticated());
}

AuthErrorCode _mapFailureToCode(Failure failure) {
  if (failure is NetworkFailure) return
    AuthErrorCode.networkError;
  if (failure is InvalidCredentialsFailure) return
    AuthErrorCode.invalidCredentials;
  if (failure is AccountLockedFailure) return
    AuthErrorCode.accountLocked;
  return AuthErrorCode.networkError;
}
}

```

1.8 Feature Module: vault

The vault module manages encrypted credential storage, including passwords, SSH keys, and secure notes.

```

// features/vault/domain/entities/vault_item.dart
abstract class VaultItem {
  final String id;
  final String label;
  final String? notes;
  final DateTime createdAt;
  final DateTime updatedAt;
  final String vaultId;

  const VaultItem({
    required this.id,
    required this.label,
    this.notes,
    required this.createdAt,
    required this.updatedAt,
  });
}

```

```
        required this.vaultId,
    });
}
```

```
class SshCredential extends VaultItem {
    final String username;
    final String? passwordRef;           // Key into secure storage
    final String? sshKeyId;
    final String? passphrase;           // Key into secure storage
    final bool useAgentForwarding;

    const SshCredential({
        required super.id,
        required super.label,
        super.notes,
        required super.createdAt,
        required super.updatedAt,
        required super.vaultId,
        required this.username,
        this.passwordRef,
        this.sshKeyId,
        this.passphrase,
        this.useAgentForwarding = false,
    });
}
```

```
class ApiToken extends VaultItem {
    final String tokenRef;               // Key into secure storage
    final String? serviceUrl;
    final DateTime? expiresAt;

    const ApiToken({
        required super.id,
        required super.label,
        super.notes,
        required super.createdAt,
        required super.updatedAt,
        required super.vaultId,
        required this.tokenRef,
        this.serviceUrl,
        this.expiresAt,
    });
}
```

```

// Cubit
@injectable
class VaultCubit extends Cubit<VaultState> {
  final GetVaultItemsUseCase _getItems;
  final CreateVaultItemUseCase _createItem;
  final UpdateVaultItemUseCase _updateItem;
  final DeleteVaultItemUseCase _deleteItem;
  final UnlockVaultUseCase _unlockVault;

  VaultCubit({
    required GetVaultItemsUseCase getItems,
    required CreateVaultItemUseCase createItem,
    required UpdateVaultItemUseCase updateItem,
    required DeleteVaultItemUseCase deleteItem,
    required UnlockVaultUseCase unlockVault,
  }) : _getItems = getItems,
      _createItem = createItem,
      _updateItem = updateItem,
      _deleteItem = deleteItem,
      _unlockVault = unlockVault,
      super(VaultLocked());

  Future<void> unlock(String masterPassword) async {
    emit(VaultUnlocking());
    final result = await _unlockVault(UnlockVaultParams(
      masterPassword: masterPassword,
    ));
    result.fold(
      (failure) => emit(VaultError(message: failure.message)),
      (_) => loadItems(),
    );
  }

  Future<void> loadItems({String? query}) async {
    emit(VaultLoading());
    final result = await _getItems(GetVaultItemsParams(query:
      query));
    result.fold(
      (failure) => emit(VaultError(message: failure.message)),
      (items) => emit(VaultLoaded(items: items)),
    );
  }
}

```

```

abstract class VaultState {}
class VaultLocked extends VaultState {}
class VaultUnlocking extends VaultState {}
class VaultLoading extends VaultState {}
class VaultLoaded extends VaultState {
  final List<VaultItem> items;
  VaultLoaded({required this.items});
}
class VaultError extends VaultState {
  final String message;
  VaultError({required this.message});
}

```

1.9 Feature Module: hosts

```

// features/hosts/presentation/bloc/host_list_bloc.dart

// Events
abstract class HostListEvent {}
class HostListLoadRequested extends HostListEvent {}
class HostListSearchChanged extends HostListEvent {
  final String query;
  HostListSearchChanged({required this.query});
}
class HostListFilterChanged extends HostListEvent {
  final HostFilter filter;
  HostListFilterChanged({required this.filter});
}
class HostListViewToggled extends HostListEvent {} // grid ↔ list
class HostListGroupSelected extends HostListEvent {
  final String? groupId; // null = all groups
  HostListGroupSelected({this.groupId});
}
class HostListSortChanged extends HostListEvent {
  final HostSortField field;
  final SortDirection direction;
  HostListSortChanged({required this.field, required
    this.direction});
}
class HostConnectRequested extends HostListEvent {
  final String hostId;
  final ConnectionProtocol protocol;
  HostConnectRequested({required this.hostId, required
    this.protocol});
}

```

```

// States
abstract class HostListState {}
class HostListInitial extends HostListState {}
class HostListLoading extends HostListState {}
class HostListLoaded extends HostListState {
    final List<HostGroup> groups;
    final List<HostEntity> hosts;
    final HostFilter activeFilter;
    final HostListViewMode viewMode;
    final HostSortField sortField;
    final SortDirection sortDirection;
    final String searchQuery;
    final String? selectedGroupId;

    HostListLoaded({
        required this.groups,
        required this.hosts,
        required this.activeFilter,
        required this.viewMode,
        required this.sortField,
        required this.sortDirection,
        required this.searchQuery,
        this.selectedGroupId,
    });

    HostListLoaded copyWith({
        List<HostGroup>? groups,
        List<HostEntity>? hosts,
        HostFilter? activeFilter,
        HostListViewMode? viewMode,
        HostSortField? sortField,
        SortDirection? sortDirection,
        String? searchQuery,
        String? selectedGroupId,
    }) => HostListLoaded(
        groups: groups ?? this.groups,
        hosts: hosts ?? this.hosts,
        activeFilter: activeFilter ?? this.activeFilter,
        viewMode: viewMode ?? this.viewMode,
        sortField: sortField ?? this.sortField,
        sortDirection: sortDirection ?? this.sortDirection,
        searchQuery: searchQuery ?? this.searchQuery,
        selectedGroupId: selectedGroupId ?? this.selectedGroupId,
    );
}

```

```

    );
  }
  class HostListError extends HostListState {
    final String message;
    HostListError({required this.message});
  }

  enum HostListViewMode { grid, list }
  enum HostSortField { label, hostname, lastConnected, createdAt }
  enum SortDirection { ascending, descending }

  class HostFilter {
    final String? groupId;
    final List<String> tags;
    final HostProtocol? protocol;
    final bool? onlyFavorites;

    const HostFilter({
      this.groupId,
      this.tags = const [],
      this.protocol,
      this.onlyFavorites,
    });
  }

```

1.10 Feature Module: terminal

// features/terminal/presentation/bloc/terminal_bloc.dart

```

abstract class TerminalEvent {}
class TerminalSessionStarted extends TerminalEvent {
  final SshSession session;
  TerminalSessionStarted({required this.session});
}
class TerminalDataReceived extends TerminalEvent {
  final Uint8List data;
  TerminalDataReceived({required this.data});
}
class TerminalResized extends TerminalEvent {
  final int columns;
  final int rows;
  TerminalResized({required this.columns, required this.rows});
}
class TerminalThemeChanged extends TerminalEvent {

```



```

    final TerminalTheme theme;
    TerminalThemeChanged({required this.theme});
}
class TerminalFontChanged extends TerminalEvent {
    final String fontFamily;
    final double fontSize;
    TerminalFontChanged({required this.fontFamily, required
        this.fontSize});
}
class TerminalSearchRequested extends TerminalEvent {
    final String query;
    final bool caseSensitive;
    final bool useRegex;
    TerminalSearchRequested({
        required this.query,
        this.caseSensitive = false,
        this.useRegex = false,
    });
}
class TerminalScrollbarCleared extends TerminalEvent {}
class TerminalSessionClosed extends TerminalEvent {}

abstract class TerminalState {}
class TerminalInitial extends TerminalState {}
class TerminalConnecting extends TerminalState {}
class TerminalConnected extends TerminalState {
    final SshSession session;
    final TerminalTheme theme;
    final String fontFamily;
    final double fontSize;
    final int columns;
    final int rows;
    final bool searchActive;
    final List<SearchMatch> searchMatches;
    final int? currentMatchIndex;

    TerminalConnected({
        required this.session,
        required this.theme,
        required this.fontFamily,
        required this.fontSize,
        required this.columns,
        required this.rows,
        this.searchActive = false,
    });
}

```

```

        this.searchMatches = const [],
        this.currentMatchIndex,
    ));
}
class TerminalDisconnected extends TerminalState {
    final String? reason;
    final bool canReconnect;
    TerminalDisconnected({this.reason, this.canReconnect = true});
}
class TerminalError extends TerminalState {
    final String message;
    // Categorized cause – see §3.7 Connection Error Taxonomy.
    //   `message` remains
    // for backward-compatible plain-text rendering; new call sites
    //   MUST also
    // populate `connectionError` so `_TerminalErrorView` can render
    //   the
    // per-category action pair from the §3.7 table instead of a
    //   bare string.
    final ConnectionError? connectionError;
    TerminalError({required this.message, this.connectionError});
}

```

1.11 Feature Module: ssh_session

```

// features/ssh_session/domain/entities/ssh_session.dart
class SshSession {
    final String id;
    final String hostId;
    final String hostLabel;
    final String hostname;
    final int port;
    final String username;
    final SshAuthMethod authMethod;
    final SshSessionStatus status;
    final DateTime connectedAt;
    final DateTime? disconnectedAt;
    final SshConnection? connection; // null if not yet connected
    final List<PortForwardChannel> portForwards;
    final bool agentForwarding;
    final SshJumpChain? jumpChain;
    final int reconnectAttempts;
    final bool isSharing; // Collaboration mode
    final String? shareCode;
}

```

```

const SshSession({
    required this.id,
    required this.hostId,
    required this.hostLabel,
    required this.hostname,
    required this.port,
    required this.username,
    required this.authMethod,
    required this.status,
    required this.connectedAt,
    this.disconnectedAt,
    this.connection,
    this.portForwards = const [],
    this.agentForwarding = false,
    this.jumpChain,
    this.reconnectAttempts = 0,
    this.isSharing = false,
    this.shareCode,
});
}

enum SshSessionStatus {
    connecting,
    authenticating,
    connected,
    reconnecting,
    disconnected,
    error,
}

class SshJumpChain {
    final List<JumpHost> hops;
    const SshJumpChain({required this.hops});
}

class JumpHost {
    final String hostname;
    final int port;
    final String username;
    final SshAuthMethod authMethod;
    const JumpHost({
        required this.hostname,
        required this.port,
        required this.username,

```

```

        required this.authMethod,
    });
}

abstract class SshAuthMethod {}

class SshPasswordAuth extends SshAuthMethod {
    final String passwordRef; // Key in secure storage
    SshPasswordAuth({required this.passwordRef});
}

class SshKeyAuth extends SshAuthMethod {
    final String keyId;
    final String? passphraseRef;
    SshKeyAuth({required this.keyId, this.passphraseRef});
}

class SshCertificateAuth extends SshAuthMethod {
    final String certificatePath;
    final String privateKeyId;
    SshCertificateAuth({required this.certificatePath, required
        this.privateKeyId});
}

class SshKeyboardInteractiveAuth extends SshAuthMethod {
    SshKeyboardInteractiveAuth();
}

```

1.12 Feature Module: sftp

```
// features/sftp/presentation/cubit/sftp_cubit.dart
```

```

class SftpState {
    final String sessionId;
    final SftpPane leftPane;
    final SftpPane rightPane;
    final List<SftpTransferJob> transferQueue;
    final SftpSortConfig sortConfig;

    const SftpState({
        required this.sessionId,
        required this.leftPane,
        required this.rightPane,
        this.transferQueue = const [],
        this.sortConfig = const SftpSortConfig(),
    });
}

```

```

class SftpPane {
    final String currentPath;
    final List<SftpEntry> entries;
    final bool isLoading;
    final String? error;
    final Set<String> selectedPaths;
    final SftpPaneStatus status;

    const SftpPane({
        required this.currentPath,
        this.entries = const [],
        this.isLoading = false,
        this.error,
        this.selectedPaths = const {},
        this.status = SftpPaneStatus.idle,
    });
}

class SftpEntry {
    final String name;
    final String path;
    final SftpEntryType type;
    final int size;
    final DateTime modifiedAt;
    final String permissions; // "-rwxr-xr-x"
    final String owner;
    final String group;
    final bool isSymlink;
    final String? linkTarget;

    const SftpEntry({
        required this.name,
        required this.path,
        required this.type,
        required this.size,
        required this.modifiedAt,
        required this.permissions,
        required this.owner,
        required this.group,
        this.isSymlink = false,
        this.linkTarget,
    });
}

```

```

class SftpTransferJob {
  final String id;
  final TransferDirection direction;
  final String sourcePath;
  final String destinationPath;
  final int totalBytes;
  final int transferredBytes;
  final TransferStatus status;
  final double speedBytesPerSec;
  final DateTime startedAt;
  final DateTime? completedAt;
  final String? errorMessage;

  double get progress => totalBytes == 0 ? 0 : transferredBytes /
    totalBytes;
  Duration get eta {
    if (speedBytesPerSec == 0) return Duration.zero;
    return Duration(seconds: ((totalBytes - transferredBytes) /
      speedBytesPerSec).round());
  }
}

enum SftpEntryType { file, directory, symlink, socket, pipe,
  blockDevice, charDevice }
enum TransferDirection { upload, download }
enum TransferStatus { queued, transferring, paused, completed,
  failed, cancelled }
enum SftpPaneStatus { idle, loading, transferring }

```

1.13 Feature Module: port_forwarding

```

// features/port_forwarding/domain/entities/port_forward_rule.dart
class PortForwardRule {
  final String id;
  final String label;
  final String hostId;
  final PortForwardType type;
  final String bindAddress;
  final int localPort;
  final String remoteHost;
  final int remotePort;
  final bool isActive;
  final bool autoStart;
  final PortForwardChannel? activeChannel;
}

```

```

const PortForwardRule({
  required this.id,
  required this.label,
  required this.hostId,
  required this.type,
  required this.bindAddress,
  required this.localPort,
  required this.remoteHost,
  required this.remotePort,
  this.isActive = false,
  this.autoStart = false,
  this.activeChannel,
});

}

class PortForwardChannel {
  final String channelId;
  final DateTime openedAt;
  final int bytesSent;
  final int bytesReceived;
  final List<PortForwardConnection> activeConnections;

  const PortForwardChannel({
    required this.channelId,
    required this.openedAt,
    this.bytesSent = 0,
    this.bytesReceived = 0,
    this.activeConnections = const [],
  });
}

```

1.14 Feature Module: workspace

The workspace module manages multi-session layouts with split views and tab groups.

```

// features/workspace/domain/entities/workspace.dart
class Workspace {
  final String id;
  final String name;
  final WorkspaceLayout layout;
  final List<WorkspacePane> panes;
  final String? templateId;
  final DateTime createdAt;
}

```

```

const Workspace({
  required this.id,
  required this.name,
  required this.layout,
  required this.panes,
  this.templateId,
  required this.createdAt,
});

enum WorkspaceLayout {
  single,          // 1 pane full screen
  splitH,          // 2 panes side by side
  splitV,          // 2 panes stacked
  grid2x2,         // 4 panes in 2x2 grid
  threeLeft,       // 1 large left + 2 stacked right
  threeRight,      // 2 stacked left + 1 large right
  custom,          // User-defined split ratios
}

class WorkspacePane {
  final String paneId;
  final String? sessionId; // null = empty pane
  final PaneType type;
  final double flexFactor; // Relative size
  final bool isFocused;

  const WorkspacePane({
    required this.paneId,
    this.sessionId,
    required this.type,
    this.flexFactor = 1.0,
    this.isFocused = false,
  });
}

enum PaneType { terminal, sftp, portForwarding, logs, empty }

```

1.15 Feature Module: keychain

```

// features/keychain/domain/entities/ssh_key.dart
class SshKey {
  final String id;

```



```

final String label;
final SshKeyType keyType;
final int keyBits;
final String publicKey;           // OpenSSH format
final String fingerprint;         // SHA256:...
final bool isProtected;           // Has passphrase
final bool isHardwareKey;         // Stored on FIDO2 key
final bool isImported;
final DateTime createdAt;
final DateTime? lastUsedAt;

const SshKey({
  required this.id,
  required this.label,
  required this.keyType,
  required this.keyBits,
  required this.publicKey,
  required this.fingerprint,
  this.isProtected = false,
  this.isHardwareKey = false,
  this.isImported = false,
  required this.createdAt,
  this.lastUsedAt,
});
}

// Key generation parameters
class KeyGenerationParams {
  final SshKeyType type;
  final int? bits;                 // For RSA; null for Ed25519/ECDSA
  final String? comment;
  final String? passphrase;
  final bool saveToSecureStorage;

  const KeyGenerationParams({
    required this.type,
    this.bits,
    this.comment,
    this.passphrase,
    this.saveToSecureStorage = true,
  });
}

```

1.16 Feature Module: collaboration

```
// features/collaboration/domain/entities/
  collaboration_session.dart

class CollaborationSession {
  final String id;
  final String terminalSessionId;
  final String ownerUserId;
  final List<CollaborationParticipant> participants;
  final CollaborationMode mode;
  final String shareCode;           // 6-char alphanumeric
  final String shareUrl;
  final DateTime startedAt;
  final bool isReadOnly;

  const CollaborationSession({
    required this.id,
    required this.terminalSessionId,
    required this.ownerUserId,
    required this.participants,
    required this.mode,
    required this.shareCode,
    required this.shareUrl,
    required this.startedAt,
    this.isReadOnly = false,
  });
}

class CollaborationParticipant {
  final String userId;
  final String displayName;
  final String? avatarUrl;
  final CollaborationRole role;
  final bool isActive;
  final CursorPosition? cursorPosition;
  final Color cursorColor;

  const CollaborationParticipant({
    required this.userId,
    required this.displayName,
    this.avatarUrl,
    required this.role,
    this.isActive = true,
    this.cursorPosition,
  });
}
```

```

        required this.cursorColor,
      });
}

enum CollaborationMode { view, control, pair }
enum CollaborationRole { owner, editor, viewer }

```

BLoC — Events and States

The collaboration entities above are domain-layer only; the presentation layer is driven by CollaborationBloc, structured like every other feature module (AuthBloc, §1.7):

```

// features/collaboration/presentation/bloc/
  collaboration_event.dart
abstract class CollaborationEvent {}

class CollaborationSessionStarted extends CollaborationEvent {
  final String terminalSessionId;
  final CollaborationMode mode;
  CollaborationSessionStarted({required this.terminalSessionId,
    required this.mode});
}

class CollaborationJoinRequested extends CollaborationEvent {
  final String shareCode;
  CollaborationJoinRequested({required this.shareCode});
}

class CollaborationParticipantJoined extends CollaborationEvent {
  final CollaborationParticipant participant;
  CollaborationParticipantJoined({required this.participant});
}

class CollaborationParticipantLeft extends CollaborationEvent {
  final String userId;
  CollaborationParticipantLeft({required this.userId});
}

class CollaborationCursorMoved extends CollaborationEvent {
  final String userId;
  final CursorPosition position;
  CollaborationCursorMoved({required this.userId, required
    this.position});
}

class CollaborationControlRequested extends CollaborationEvent {
  final String requestingUserId;
  CollaborationControlRequested({required this.requestingUserId});
}

class CollaborationControlGranted extends CollaborationEvent {

```

```

    final String userId;
    CollaborationControlGranted({required this.userId});
}
class CollaborationControlRevoked extends CollaborationEvent {
    final String userId;
    final String reason; // 'owner-override' | 'idle-timeout' |
                        'left-session'
    CollaborationControlRevoked({required this.userId, required
                                this.reason});
}
class CollaborationParticipantKicked extends CollaborationEvent {
    final String userId;
    CollaborationParticipantKicked({required this.userId});
}
class CollaborationChannelDisconnected extends CollaborationEvent
{
    final String? reason;
    CollaborationChannelDisconnected({this.reason});
}
class CollaborationSessionEnded extends CollaborationEvent {}

// features/collaboration/presentation/bloc/
// collaboration_state.dart
abstract class CollaborationState {}
class CollaborationInactive extends CollaborationState {}
class CollaborationConnecting extends CollaborationState {}
class CollaborationActive extends CollaborationState {
    final CollaborationSession session;
    final String? controllingUserId; // null when mode == view (no
                                    floor holder)
    final Map<String, CursorPosition> remoteCursors;
    CollaborationActive({
        required this.session,
        required this.controllingUserId,
        this.remoteCursors = const {},
    });
}
class CollaborationDisconnected extends CollaborationState {
    final String? reason;
    final bool canReconnect;
    CollaborationDisconnected({this.reason, this.canReconnect =
                                true});
}

```

Transport — WebSocket Channel

The collaboration channel is a dedicated WebSocket connection, separate from the terminal's own SSH data path, established against `HelixConfig.wsBaseUrl` (Appendix C) at `wss://<wsBaseUrl>/v1/collaboration/{terminalSessionId}`:

```
// features/collaboration/data/services/
    collaboration_transport.dart

@injectable
class CollaborationTransport {
  final AuthTokenProvider _tokenProvider; // supplies the EdDSA-
    signed access JWT (see §5, doc 05)
  WebSocketChannel? _channel;
  Timer? _heartbeatTimer;

  Future<void> connect(String terminalSessionId) async {
    final jwt = await _tokenProvider.currentAccessToken(); //
      EdDSA (Ed25519), CD-7
    _channel = WebSocketChannel.connect(
      Uri.parse('wss://${Env.wsBaseUrl}/v1/collaboration/
        $terminalSessionId'),
      protocols: ['helix.collab.v1'],
    );
    // First frame on every collaboration socket is the auth
      handshake –
    // the gateway validates the JWT signature + RBAC scope before
      admitting
    // any subsequent frame onto the session's participant roster.
    _send(CollabFrame.auth(jwt: jwt));
    _heartbeatTimer = Timer.periodic(const Duration(seconds: 5),
      (_) {
        _send(CollabFrame.heartbeat());
      });
    _channel!.stream.listen(_onFrame, onDone: _onDisconnected,
      onError: _onError);
  }

  void _send(CollabFrame frame) =>
    _channel?.sink.add(frame.toBinary());

  Future<void> disconnect() async {
    _heartbeatTimer?.cancel();
    await _channel?.sink.close(status.normalClosure);
  }
}
```

Frame format is a compact binary envelope (1-byte frame type + MessagePack-encoded payload) to keep cursor/presence chatter cheap on constrained mobile uplinks:

Frame type	Direction	Payload	Cadence
auth	client → server	JWT bearer	once, first frame
heartbeat	client → server	—	every 5s
presence	client → server	{cursorRow, cursorCol, ts}	debounced 50ms, max 20/s
keystroke	client → server (controller only)	raw SSH input bytes	per keypress
output	server → client (fan-out)	raw SSH output bytes	per PTY write
roster	server → client	full participant list	on join/leave
control_request / control_grant / control_revoke	bidirectional	{userId, reason?}	on demand
error	server → client	{code, message}	on protocol violation

The server is the authoritative fan-out point: `keystroke` frames from the current controller are forwarded to the underlying SSH session (§3) and the resulting output is broadcast to every connected participant, so all viewers see byte-identical terminal output regardless of who is typing.

Presence

- Each participant publishes a presence frame (cursor row/column + monotonic timestamp) at most every 50ms, coalescing rapid cursor moves client-side before sending (Timer debounce, not per-frame).
- A participant with no heartbeat for 15s (3 missed 5s beats) is marked `isActive = false` and greyed out in `ParticipantRow` (§6.14); after 30s with no heartbeat the server evicts the participant and emits `roster` with the entry removed.
- `CollaborationCursorMoved` updates
`CollaborationActive.remoteCursors`, rendered by `CursorOverlay` (§6.14) — one colored caret + username label per active participant,

keyed by `cursorColor` assigned at join time (round-robin from a 8-color palette, collision-avoided against currently-seated participants).

Control-Contention Arbitration

Exactly one participant holds the “floor” (may send `keystroke` frames) at any moment in `control` mode; `pair` mode relaxes this to at most two concurrent floor-holders (owner + one invited co-driver) with server-side `keystroke` interleaving preserved in receipt order (last-writer per PTY byte position — acceptable because a human co-driving session is not editing the same buffer position concurrently the way a text CRDT would need to reconcile). `view` mode never grants a floor; all non-owner participants are read-only.

Arbitration state machine (owner is always eligible to reclaim the floor unconditionally):

1. **Idle** — only the owner holds the floor (default on session start).
2. An editor-role participant sends `CollaborationControlRequested`.
3. The current floor-holder (usually the owner) sees a non-blocking in-terminal toast (“<name> requests control”) with `Grant / Deny` actions; **no auto-grant** — an unanswered request expires after 20s and the requester is notified `control_revoke(reason: 'request-timed-out')`.
4. On `Grant`, the server emits `control_grant` to the new holder and `control_revoke` to the prior holder (reason `'transferred'`); exactly one `keystroke`-eligible participant exists at all times in `control` mode.
5. **Owner override** — the owner may at any time emit `CollaborationControlRevoked` against the current non-owner holder (reason `'owner-override'`), reclaiming the floor immediately without the requestee’s consent — this is the deadlock-breaker for an unresponsive or disruptive co-driver.
6. **Idle-timeout reclaim** — a non-owner floor-holder who sends no `keystroke` for 120s has the floor silently reclaimed by the owner (reason `'idle-timeout'`) so the session is never stranded on a disconnected/idle holder.
7. Floor-holder disconnect (`CollaborationChannelDisconnected`) always reclaims the floor to the owner (reason `'left-session'`), never leaves the session floor-less.

This mirrors the read/write-lock discipline used for the vault’s zero-knowledge sync (§4.3) at a conceptual level — “exactly one writer, arbitrated, never silently dual-write” — but is a distinct mechanism:

collaboration floor arbitration is a live human-turn-taking protocol over a WebSocket, whereas §4.3's `crdtVectorClock` strategy resolves offline/concurrent SQLite edits. They **MUST NOT** be conflated; a collaboration session emits no vector clocks and vault sync never grants a “floor.”

Channel Encryption

- Transport is always `wss://` (TLS 1.3), matching every other client ↔ gateway connection.
- Because a shared terminal session routinely carries the same operational content a solo SSH session would (command output that may include secrets pasted by the remote host), the collaboration channel additionally supports an **optional** application-layer E2E encryption mode (`CollaborationSession.isReadOnly == false` sessions created with `e2eEncrypted: true`): each participant's client performs an X25519 ECDH key exchange over the already-authenticated auth frame to derive a per-session symmetric key, and `keystroke/output` frame payloads are then encrypted with ChaCha20-Poly1305 before being sent — the collaboration relay forwards opaque ciphertext and cannot read session content even though it is the fan-out point. This is **not** the CD-10 zero-knowledge vault guarantee (that clause is scoped to vault secrets only); it is a defense-in-depth option for collaboration sessions that may traverse operationally sensitive terminal output, and it is OFF by default (plain-TLS is the default, matching every other realtime feature) because it disables server-side session recording/audit for that session — enabling it **MUST** surface an explicit “this session will not be recorded for audit” UI warning to the owner before start.

Cross-doc note: `09_performance_analysis.md` does not yet carry a collaboration-specific latency-budget or SLO entry (tracked as an open cross-doc gap in `docs/research/mvp/REMEDIATION_REGISTER.md` §4 “Real-time collaboration”). Until that lands, the authoritative client-observable target is §7.8 below (presence ≤ 100ms, keystroke fan-out ≤ 150ms p95) — doc 09's eventual collaboration SLO row **MUST** reconcile against these client-side numbers rather than introduce a divergent target.

1.17 Feature Module: `ai_autocomplete`

```
// features/ai_autocomplete/domain/entities/ai_suggestion.dart
class AiSuggestion {
  final String id;
```



```

final String command;
final String explanation;
final double confidence;
final SuggestionSource source;
final List<String> alternatives;

const AiSuggestion({
  required this.id,
  required this.command,
  required this.explanation,
  required this.confidence,
  required this.source,
  this.alternatives = const [],
});
}

enum SuggestionSource { history, aiModel, snippets,
  documentation }

// Cubit
@injectable
class AiAutocompleteCubit extends Cubit<AiAutocompleteState> {
  final GetAiSuggestionsUseCase _getSuggestions;
  final CommandHistoryRepository _historyRepo;
  Timer? _debounceTimer;

  AiAutocompleteCubit({
    required GetAiSuggestionsUseCase getSuggestions,
    required CommandHistoryRepository historyRepo,
  }) : _getSuggestions = getSuggestions,
      _historyRepo = historyRepo,
      super(AiAutocompleteIdle());

  void onTerminalInput(String currentInput, String shellContext) {
    _debounceTimer?.cancel();
    if (currentInput.trim().length < 3) {
      emit(AiAutocompleteIdle());
      return;
    }
    _debounceTimer = Timer(const Duration(milliseconds: 300), () {
      _fetchSuggestions(currentInput, shellContext);
    });
  }
}

```

```

Future<void> _fetchSuggestions(String input, String context)
  async {
    emit(AiAutocompleteLoading());
    final result = await _getSuggestions(AiSuggestionParams(
      partialCommand: input,
      shellContext: context,
      historyContext: await _historyRepo.getRecentCommands(limit:
        20),
    ));
    result.fold(
      (_) => emit(AiAutocompleteIdle()),
      (suggestions) => emit(AiAutocompleteLoaded(suggestions:
        suggestions)),
    );
  }
}

abstract class AiAutocompleteState {}
class AiAutocompleteIdle extends AiAutocompleteState {}
class AiAutocompleteLoading extends AiAutocompleteState {}
class AiAutocompleteLoaded extends AiAutocompleteState {
  final List<AiSuggestion> suggestions;
  AiAutocompleteLoaded({required this.suggestions});
}

```

1.18 Feature Module: notifications

```

// features/notifications/domain/entities/app_notification.dart
class AppNotification {
  final String id;
  final NotificationType type;
  final String title;
  final String? body;
  final Map<String, dynamic> payload;
  final bool isRead;
  final DateTime receivedAt;
  final NotificationPriority priority;

  const AppNotification({
    required this.id,
    required this.type,
    required this.title,
    this.body,
    required this.payload,
    this.isRead = false,
  });
}

```

```

        required this.receivedAt,
        required this.priority,
    });
}

enum NotificationType {
    sessionConnected,
    sessionDisconnected,
    sessionFailed,
    transferCompleted,
    transferFailed,
    collaborationInvite,
    vaultExpiry,
    newDeviceLogin,
    securityAlert,
    syncConflict,
}

enum NotificationPriority { low, normal, high, critical }

```

2. Terminal Emulator Design

2.1 xterm.dart Integration Architecture

HelixTerminator uses xterm.dart by TerminalStudio as the core terminal emulator. The integration is structured as a dedicated service layer that bridges the SSH transport with the terminal widget.

```

// features/terminal/data/services/terminal_service.dart
import 'package:xterm/xterm.dart';

@injectable
class TerminalService {
    final Terminal _terminal;
    final TerminalController _controller;
    final SSHSessionService _sshService;
    StreamSubscription? _outputSubscription;
    StreamSubscription? _inputSubscription;

    TerminalService(this._sshService)
        : _terminal = Terminal(
            maxLines: 100000,    // 100k line scrollbar

```

```

        onPrivateModeSet: _handlePrivateModeSet,
        onPrivateModeReset: _handlePrivateModeReset,
    ),
    _controller = TerminalController();

Terminal get terminal => _terminal;
TerminalController get controller => _controller;

void attachToSession(SshShell shell) {
    // SSH → Terminal: receive remote output
    _outputSubscription = shell.stdout.listen((data) {
        _terminal.write(utf8.decode(data, allowMalformed: true));
    });

    // Terminal → SSH: send user keystrokes
    _inputSubscription = _terminal.onOutput.listen((data) {
        shell.stdin.add(utf8.encode(data));
    });

    // Resize events
    _terminal.onResize = (width, height, pixelWidth, pixelHeight)
    {
        shell.resizeTerminal(width, height, pixelWidth,
            pixelHeight);
    };
}

void detach() {
    _outputSubscription?.cancel();
    _inputSubscription?.cancel();
    _outputSubscription = null;
    _inputSubscription = null;
}

void dispose() {
    detach();
    _terminal.dispose();
}

static void _handlePrivateModeSet(int mode) {
    // Handle DECSET (e.g., mouse reporting, alternate screen)
}

static void _handlePrivateModeReset(int mode) {
    // Handle DECRST
}

```

```
}  
}
```

2.2 Terminal Widget Tree

```
// features/terminal/presentation/widgets/terminal_view.dart  
class TerminalView extends StatefulWidget {  
  final String sessionId;  
  const TerminalView({required this.sessionId, super.key});  
  
  @override  
  State<TerminalView> createState() => _TerminalViewState();  
}  
  
class _TerminalViewState extends State<TerminalView> {  
  late final FocusNode _focusNode;  
  
  @override  
  void initState() {  
    super.initState();  
    _focusNode = FocusNode();  
    // Auto-focus terminal on mount  
    WidgetsBinding.instance.addPostFrameCallback((_) {  
      _focusNode.requestFocus();  
    });  
  }  
  
  @override  
  Widget build(BuildContext context) {  
    return BlocConsumer<TerminalBloc, TerminalState>(  
      listener: (context, state) {  
        if (state is TerminalDisconnected) {  
          _showDisconnectBanner(context, state);  
        }  
      },  
      builder: (context, state) {  
        if (state is TerminalConnected) {  
          return _buildConnectedView(context, state);  
        }  
        if (state is TerminalConnecting) {  
          return const _TerminalConnectingOverlay();  
        }  
        if (state is TerminalError) {  
          return _TerminalErrorView(message: state.message);  
        }  
      }  
    );  
  }  
}
```

```

    }
    return const SizedBox.shrink();
  },
);
}

```

```

Widget _buildConnectedView(BuildContext context,
  TerminalConnected state) {
  final terminalService = context.read<TerminalService>();
  return Stack(
    children: [
      TerminalViewWidget(
        terminal: terminalService.terminal,
        controller: terminalService.controller,
        focusNode: _focusNode,
        theme: _buildTerminalTheme(state.theme),
        textStyle: TerminalStyle(
          fontSize: state.fontSize,
          fontFamily: state.fontFamily,
        ),
        keyboardType: TextInputType.multiline,
        autofocus: true,
        onTap: () => _focusNode.requestFocus(),
      ),
      if (state.searchActive)
        Positioned(
          top: 8,
          right: 8,
          child: TerminalSearchBar(
            sessionId: widget.sessionId,
            matches: state.searchMatches,
            currentMatch: state.currentMatchIndex,
          ),
        ),
      // AI suggestion overlay
      const Positioned(
        bottom: 0,
        left: 0,
        right: 0,
        child: AiSuggestionOverlay(),
      ),
      // Connection info bar
      Positioned(
        top: 0,

```

```

        left: 0,
        right: 0,
        child: TerminalInfoBar(
          session: state.session,
          fontSize: state.fontSize,
        ),
      ),
    ],
  );
}

```

```

TerminalTheme _buildTerminalTheme(TerminalTheme appTheme) {
  return TerminalTheme(
    cursor: appTheme.cursor,
    selection: appTheme.selection,
    foreground: appTheme.foreground,
    background: appTheme.background,
    black: appTheme.black,
    white: appTheme.white,
    red: appTheme.red,
    green: appTheme.green,
    yellow: appTheme.yellow,
    blue: appTheme.blue,
    magenta: appTheme.magenta,
    cyan: appTheme.cyan,
    brightBlack: appTheme.brightBlack,
    brightRed: appTheme.brightRed,
    brightGreen: appTheme.brightGreen,
    brightYellow: appTheme.brightYellow,
    brightBlue: appTheme.brightBlue,
    brightMagenta: appTheme.brightMagenta,
    brightCyan: appTheme.brightCyan,
    brightWhite: appTheme.brightWhite,
  );
}

```

2.3 VT100/VT220/xterm Escape Sequence Support

The terminal must correctly process the complete set of ANSI/VT escape sequences. The following escape sequence groups are supported and tested:

Control Characters (C0) - BEL (0x07) — Audio bell + visual flash - BS (0x08) — Backspace (destructive) - HT (0x09) — Horizontal tab (8-space default, configurable) - LF/VT/FF (0x0A/0x0B/0x0C) — Line feed - CR (0x0D) — Carriage return - ESC (0x1B) — Escape sequence introducer - DEL (0x7F) — Delete character

Cursor Movement Sequences - CSI A/B/C/D — Cursor Up/Down/Forward/Backward - CSI H / CSI f — Cursor Position (absolute) - CSI G — Cursor Horizontal Absolute - CSI d — Cursor Vertical Absolute - ESC 7 / ESC 8 — Save/Restore Cursor (DEC private) - CSI s / CSI u — Save/Restore Cursor (ANSI)

Erase Operations - CSI J — Erase in Display (0=to end, 1=to start, 2=all, 3=scrollback) - CSI K — Erase in Line (0=to end, 1=to start, 2=all) - CSI L/M — Insert/Delete Lines - CSI P — Delete Characters - CSI @ — Insert Characters

SGR — Select Graphic Rendition (CSI ... m) - Colors 0-7, 10-17 (standard 16 colors) - Colors 90-97, 100-107 (bright colors) - 38;5;n — 256 color foreground - 48;5;n — 256 color background - 38;2;r;g;b — True color (24-bit) foreground - 48;2;r;g;b — True color (24-bit) background - Bold, Dim, Italic, Underline, Blink, Reverse, Invisible, Strikethrough - Underline styles (double, curly, dotted, dashed) via 58; extension - Underline color via 58;2;r;g;b

DEC Private Modes - ?1 — DECCKM (Application Cursor Keys) - ?7 — DECAWM (Auto Wrap Mode) - ?25 — DECTCEM (Cursor Visibility) - ?47/?1047/?1049 — Alternate Screen Buffer - ?1000/?1002/?1003 — Mouse Tracking modes - ?1004 — Focus Reporting - ?1006 — SGR Mouse Extensions - ?2004 — Bracketed Paste Mode

OSC Sequences - OSC 0 / OSC 2 — Set window title - OSC 7 — Notify current working directory (shell integration) - OSC 52 — Clipboard operations - OSC 133 — Shell integration marks (prompt, command, output) - OSC 1337 — iTerm2 extensions (inline images) - OSC 8 — Hyperlinks

```
// features/terminal/data/services/osc_handler.dart
```

```
class OscHandler {  
  final TerminalUrlLauncher _urlLauncher;  
  final TerminalImageRenderer _imageRenderer;  
  final ClipboardService _clipboard;  
  
  OscHandler({  
    required TerminalUrlLauncher urlLauncher,  
    required TerminalImageRenderer imageRenderer,  
    required ClipboardService clipboard,  
  }) {  
    _urlLauncher = urlLauncher;  
    _imageRenderer = imageRenderer;  
    _clipboard = clipboard;  
  }  
}
```



```

        required ClipboardService clipboard,
    }) : _urlLauncher = urlLauncher,
        _imageRenderer = imageRenderer,
        _clipboard = clipboard;

void handle(int oscCode, String params) {
    switch (oscCode) {
        case 0:
        case 2:
            // Window title
            _handleWindowTitle(params);
            break;
        case 7:
            // CWD notification (shell integration)
            _handleCwdNotification(params);
            break;
        case 8:
            // Hyperlink: OSC 8 ; params ; uri ST
            _handleHyperlink(params);
            break;
        case 52:
            // Clipboard
            _handleClipboard(params);
            break;
        case 133:
            // Shell integration marks
            _handleShellIntegration(params);
            break;
        case 1337:
            // iTerm2 inline images
            _handleIterm2Image(params);
            break;
    }
}

void _handleIterm2Image(String params) {
    // Parse: File=[params]:[base64data]
    final colonIdx = params.indexOf(':');
    if (colonIdx < 0) return;
    final metaPart = params.substring(0, colonIdx);
    final dataPart = params.substring(colonIdx + 1);

    final metaMap = <String, String>{};
    for (final kv in metaPart.split(';')) {

```

```

    final eq = kv.indexOf('=');
    if (eq > 0) metaMap[kv.substring(0, eq)] = kv.substring(eq
        + 1);
}

try {
    final imageBytes = base64.decode(dataPart);
    final width = int.tryParse(metaMap['width'] ?? '') ?? 0;
    final height = int.tryParse(metaMap['height'] ?? '') ?? 0;
    _imageRenderer.render(imageBytes, width: width, height:
        height);
} catch (_) {
    // Ignore malformed image data
}
}

```

2.4 Color Scheme System

```

// features/terminal/domain/entities/terminal_theme.dart
class TerminalTheme {
    final String id;
    final String name;
    final Color background;
    final Color foreground;
    final Color cursor;
    final Color cursorText;
    final Color selection;
    final Color selectionText;

    // Standard 16 ANSI colors
    final Color black;
    final Color red;
    final Color green;
    final Color yellow;
    final Color blue;
    final Color magenta;
    final Color cyan;
    final Color white;
    final Color brightBlack;
    final Color brightRed;
    final Color brightGreen;
    final Color brightYellow;
    final Color brightBlue;
    final Color brightMagenta;

```

```

final Color brightCyan;
final Color brightWhite;

const TerminalTheme({
  required this.id,
  required this.name,
  required this.background,
  required this.foreground,
  required this.cursor,
  required this.cursorText,
  required this.selection,
  required this.selectionText,
  required this.black,
  required this.red,
  required this.green,
  required this.yellow,
  required this.blue,
  required this.magenta,
  required this.cyan,
  required this.white,
  required this.brightBlack,
  required this.brightRed,
  required this.brightGreen,
  required this.brightYellow,
  required this.brightBlue,
  required this.brightMagenta,
  required this.brightCyan,
  required this.brightWhite,
});

static const TerminalTheme dracula = TerminalTheme(
  id: 'dracula',
  name: 'Dracula',
  background: Color(0xFF282A36),
  foreground: Color(0xFFFF8F8F2),
  cursor: Color(0xFFFF8F8F2),
  cursorText: Color(0xFF282A36),
  selection: Color(0xFF44475A),
  selectionText: Color(0xFFFF8F8F2),
  black: Color(0xFF21222C),
  red: Color(0xFFFF5555),
  green: Color(0xFF50FA7B),
  yellow: Color(0xFFFF1FA8C),
  blue: Color(0xFFBD93F9),

```

```

    magenta: Color(0xFFFF79C6),
    cyan: Color(0xFF8BE9FD),
    white: Color(0xFFF8F8F2),
    brightBlack: Color(0xFF6272A4),
    brightRed: Color(0xFFFF6E6E),
    brightGreen: Color(0xFF69FF94),
    brightYellow: Color(0xFFFFFFFFA5),
    brightBlue: Color(0xFFD6ACFF),
    brightMagenta: Color(0xFFFF92DF),
    brightCyan: Color(0xFFA4FFFF),
    brightWhite: Color(0xFFFFFFFF),
);

static const TerminalTheme oneDark = TerminalTheme(
    id: 'one_dark',
    name: 'One Dark',
    background: Color(0xFF282C34),
    foreground: Color(0xFFABB2BF),
    cursor: Color(0xFF528BFF),
    cursorText: Color(0xFF282C34),
    selection: Color(0xFF3E4451),
    selectionText: Color(0xFFABB2BF),
    black: Color(0xFF3F4451),
    red: Color(0xFFE06C75),
    green: Color(0xFF98C379),
    yellow: Color(0xFFE5C07B),
    blue: Color(0xFF61AFEF),
    magenta: Color(0xFFC678DD),
    cyan: Color(0xFF56B6C2),
    white: Color(0xFFABB2BF),
    brightBlack: Color(0xFF4F5666),
    brightRed: Color(0xFFBE5046),
    brightGreen: Color(0xFF98C379),
    brightYellow: Color(0xFFD19A66),
    brightBlue: Color(0xFF61AFEF),
    brightMagenta: Color(0xFFC678DD),
    brightCyan: Color(0xFF56B6C2),
    brightWhite: Color(0xFFFFFFFF),
);

static const TerminalTheme solarizedDark = TerminalTheme(
    id: 'solarized_dark',
    name: 'Solarized Dark',
    background: Color(0xFF002B36),

```

```

    foreground: Color(0xFF839496),
    cursor: Color(0xFF93A1A1),
    cursorText: Color(0xFF002B36),
    selection: Color(0xFF073642),
    selectionText: Color(0xFF93A1A1),
    black: Color(0xFF073642),
    red: Color(0xFFDC322F),
    green: Color(0xFF859900),
    yellow: Color(0xFFB58900),
    blue: Color(0xFF268BD2),
    magenta: Color(0xFFD33682),
    cyan: Color(0xFF2AA198),
    white: Color(0xFFEEEE8D5),
    brightBlack: Color(0xFF002B36),
    brightRed: Color(0xFFCB4B16),
    brightGreen: Color(0xFF586E75),
    brightYellow: Color(0xFF657B83),
    brightBlue: Color(0xFF839496),
    brightMagenta: Color(0xFF6C71C4),
    brightCyan: Color(0xFF93A1A1),
    brightWhite: Color(0xFFFFDF6E3),
  );

  static const List<TerminalTheme> builtIn = [
    dracula,
    oneDark,
    solarizedDark,
    // catppuccinMocha, nordDark, gruvboxDark, tokyoNight, etc.
  ];
}

```

2.5 Font Rendering

```

// features/terminal/data/services/terminal_font_service.dart
class TerminalFontService {
  static const List<String> bundledFonts = [
    'JetBrainsMono', // Default
    'FiraCode',
    'CascadiaCode',
    'Hack',
    'IBMPlexMono',
    'SourceCodePro',
    'UbuntuMono',
    'Inconsolata',
  ];
}

```

```

];

static const List<String> ligaturesEnabledFonts = [
  'JetBrainsMono',
  'FiraCode',
  'CascadiaCode',
];

static bool supportsLigatures(String fontFamily) =>
  ligaturesEnabledFonts.contains(fontFamily);

// Load a custom font from file system (desktop only)
static Future<void> loadCustomFont(String path) async {
  final fontData = await File(path).readAsBytes();
  final loader = FontLoader(p.basenameWithoutExtension(path));
  loader.addFont(Future.value(ByteData.view(fontData.buffer)));
  await loader.load();
}

// Font size constraints
static const double minFontSize = 8.0;
static const double maxFontSize = 48.0;
static const double defaultFontSize = 14.0;
static const double defaultLineHeight = 1.2;
}

```

2.6 Keyboard Input Handling

All modifier keys and special keys must be correctly mapped across all platforms.

```

// features/terminal/data/services/keyboard_handler.dart
class TerminalKeyboardHandler {
  final Terminal _terminal;
  final AppSettings _settings;

  TerminalKeyboardHandler(this._terminal, this._settings);

  KeyEventResult handleKeyEvent(KeyEvent event) {
    if (event is! KeyDownEvent && event is! KeyRepeatEvent) {
      return KeyEventResult.ignored;
    }

    final key = event.logicalKey;

```

```

final ctrl = HardwareKeyboard.instance.isControlPressed;
final alt = HardwareKeyboard.instance.isAltPressed;
final shift = HardwareKeyboard.instance.isShiftPressed;
final meta = HardwareKeyboard.instance.isMetaPressed;

// macOS: Cmd+C copies in terminal, doesn't send SIGINT
if (meta && key == LogicalKeyboardKey.keyC) {
  _handleCopy();
  return KeyEventResult.handled;
}
if (meta && key == LogicalKeyboardKey.keyV) {
  _handlePaste();
  return KeyEventResult.handled;
}

// Ctrl+C → SIGINT (0x03)
if (ctrl && !shift && !alt && key == LogicalKeyboardKey.keyC)
{
  _terminal.keyInput(TerminalKey.keyC, ctrl: true);
  return KeyEventResult.handled;
}

// Application Keypad / Cursor Key sequences
if (key == LogicalKeyboardKey.arrowUp) {
  _terminal.keyInput(TerminalKey.arrowUp, ctrl: ctrl, shift:
    shift, alt: alt);
  return KeyEventResult.handled;
}
// ... (all arrow keys, F1-F20, Page Up/Down, Home, End,
//   Insert, Delete)

// Pass-through for regular characters
return KeyEventResult.ignored;
}

void _handleCopy() {
  final selected = _terminal.selectedText;
  if (selected != null && selected.isNotEmpty) {
    Clipboard.setData(ClipboardData(text: selected));
  }
}

void _handlePaste() async {
  final data = await Clipboard.getData(Clipboard.kTextPlain);
  if (data?.text != null) {

```

```

        // Bracketed paste if enabled
        if (_terminal.bracketedPasteMode) {
            _terminal.paste(data!.text!);
        } else {
            _terminal.paste(data!.text!);
        }
    }
}
}

```

2.7 URL Detection and Clickable Links

```

// features/terminal/data/services/url_detector.dart
class TerminalUrlDetector {
    static final RegExp _urlPattern = RegExp(
        r'(:https?|ftp|ssh|git)://[^\s\x1b\x07\x0d\x0a"'`\\[\]{}()<>]
        +',
        caseSensitive: false,
    );

    static final RegExp _ipPattern = RegExp(
        r'\b(?:(:25[0-5]|2[0-4]\d|[01]?\d\d?)\.){3}(:25[0-5]|
        2[0-4]\d|[01]?\d\d?):\d{1,5}\b',
    );

    /// Finds all URL spans in a line of terminal text
    static List<UrlSpan> findUrls(String line) {
        final spans = <UrlSpan>[];

        for (final match in _urlPattern.allMatches(line)) {
            spans.add(UrlSpan(
                start: match.start,
                end: match.end,
                url: match.group(0)!,
                type: UrlSpanType.http,
            ));
        }

        // Detect bare IP:port combinations
        for (final match in _ipPattern.allMatches(line)) {
            spans.add(UrlSpan(
                start: match.start,
                end: match.end,
                url: 'http://${match.group(0)!}',
                type: UrlSpanType.ip,
            ));
        }
    }
}

```



```

        ));
    }

    return spans;
}

}

class UrlSpan {
    final int start;
    final int end;
    final String url;
    final UrlSpanType type;

    const UrlSpan({
        required this.start,
        required this.end,
        required this.url,
        required this.type,
    });
}

enum UrlSpanType { http, ip, osc8 }

```

2.8 Scrollback Buffer Management

The scrollback buffer is managed to prevent unbounded memory growth while maintaining user-configured history depth.

```

// features/terminal/data/services/scrollback_manager.dart
class ScrollbackManager {
    static const int defaultMaxLines = 10000;
    static const int absoluteMaxLines = 500000;

    final int maxLines;
    int _currentLines = 0;

    ScrollbackManager({this.maxLines = defaultMaxLines});

    // xterm.dart Terminal is configured with maxLines at
    // construction
    // This service tracks memory pressure and triggers compaction
    void onLineAdded() {
        _currentLines++;
        if (_currentLines > maxLines) {
            _triggerCompaction();
        }
    }
}

```

```

    }
  }

  void _triggerCompaction() {
    // Signal to Terminal to flush oldest lines
    // xterm.dart handles this internally via maxLines
  }

  // Estimate memory usage
  int estimateMemoryBytes(int avgLineLength) {
    return _currentLines * avgLineLength * 2; // UTF-16
  }

  // For memory pressure warnings on mobile
  bool get isApproachingLimit => _currentLines > (maxLines * 0.9);
}

```

2.9 Terminal Search

```

// features/terminal/data/services/terminal_search_service.dart
class TerminalSearchService {
  final Terminal _terminal;

  TerminalSearchService(this._terminal);

  List<SearchMatch> search(
    String query, {
    bool caseSensitive = false,
    bool useRegex = false,
  }) {
    if (query.isEmpty) return [];

    final matches = <SearchMatch>[];
    final pattern = useRegex
      ? RegExp(query, caseSensitive: caseSensitive)
      : RegExp(
        RegExp.escape(query),
        caseSensitive: caseSensitive,
      );

    // Search through terminal buffer
    for (int row = 0; row < _terminal.buffer.lines.length; row++)
    {
      final line = _terminal.buffer.lines[row];
    }
  }
}

```

```

    final lineText = line.toString();

    for (final match in pattern.allMatches(lineText)) {
      matches.add(SearchMatch(
        row: row,
        startCol: match.start,
        endCol: match.end,
        text: match.group(0)!,
      ));
    }
  }

  return matches;
}

void highlightMatch(SearchMatch match) {
  _terminal.scrollTo(match.row);
}
}

class SearchMatch {
  final int row;
  final int startCol;
  final int endCol;
  final String text;

  const SearchMatch({
    required this.row,
    required this.startCol,
    required this.endCol,
    required this.text,
  });
}

```

2.10 Performance Optimization

Terminal rendering performance is critical. The target is 60fps with < 16ms frame budget per frame.

```

// features/terminal/presentation/widgets/
//   terminal_view_widget.dart
// Custom RepaintBoundary and dirty region tracking

class OptimizedTerminalWrapper extends StatefulWidget {
  final Terminal terminal;

```

```

final TerminalController controller;
final TerminalTheme theme;
final TerminalStyle textStyle;

const OptimizedTerminalWrapper({
  required this.terminal,
  required this.controller,
  required this.theme,
  required this.textStyle,
  super.key,
});

@override
State<OptimizedTerminalWrapper> createState() =>
  _OptimizedTerminalWrapperState();
}

class _OptimizedTerminalWrapperState extends
  State<OptimizedTerminalWrapper>
  with SingleTickerProviderStateMixin {
  late final AnimationController _frameController;
  final _repaintKey = GlobalKey();

  @override
  void initState() {
    super.initState();
    // Drive repaints at 60fps when terminal has dirty content
    _frameController = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 16),
    );

    widget.terminal.addListener(_onTerminalChange);
  }

  void _onTerminalChange() {
    // Coalesce multiple terminal updates into a single frame
    if (!_frameController.isAnimating) {
      _frameController.forward(from: 0).then((_) {
        if (mounted) setState(() {});
      });
    }
  }
}

```

```

@override
Widget build(BuildContext context) {
  return RepaintBoundary(
    key: _repaintKey,
    child: TerminalViewWidget(
      terminal: widget.terminal,
      controller: widget.controller,
      theme: widget.theme,
      textStyle: widget.textStyle,
    ),
  );
}

@override
void dispose() {
  widget.terminal.removeListener(_onTerminalChange);
  _frameController.dispose();
  super.dispose();
}
}

```

3. SSH Connection Architecture

3.1 dartssh2 Integration

HelixTerminator uses dartssh2 (pure Dart SSH client) as the transport layer. This provides SSH2 protocol support without any native dependencies, enabling full cross-platform support including Web/WASM.

```

// features/ssh_session/data/services/ssh_connection_service.dart
import 'package:dartssh2/dartssh2.dart';

@injectable
class SshConnectionService {
  final SshKeyRepository _keyRepo;
  final SecureStorageService _secureStorage;
  final KnownHostsService _knownHosts;

  SshConnectionService({
    required SshKeyRepository keyRepo,
    required SecureStorageService secureStorage,

```

```

        required KnownHostsService knownHosts,
    }) : _keyRepo = keyRepo,
        _secureStorage = secureStorage,
        _knownHosts = knownHosts;

Future<SshSessionResult> connect(SshConnectionParams params)
    async {
        // Step 1: Resolve jump chain if applicable
        SSHTSocket socket;
        if (params.jumpChain != null &&
            params.jumpChain!.hops.isNotEmpty) {
            socket = await _buildJumpSocket(params.jumpChain!.hops,
                params);
        } else {
            socket = await _buildDirectSocket(params.hostname,
                params.port);
        }

        // Step 2: Create SSH client
        final client = SSHClient(
            socket,
            username: params.username,
            onVerifyHostKey: (host, port, fingerprint) =>
                _verifyHostKey(host, port, fingerprint),
            identities: await _buildIdentities(params.authMethod),
            onPasswordRequest: params.authMethod is SshPasswordAuth
                ? () async {
                    final ref = (params.authMethod as
                        SshPasswordAuth).passwordRef;
                    return await _secureStorage.read(ref);
                }
                : null,
            onUserInfoRequest: params.authMethod is
                SshKeyboardInteractiveAuth
                ? _handleKeyboardInteractive
                : null,
            keepAliveInterval: params.keepAliveInterval ??
                const Duration(seconds: 30),
        );

        await client.authenticated;

        return SshSessionResult(
            client: client,
            socket: socket,
            connectedAt: DateTime.now(),

```

```

    );
}

Future<bool> _verifyHostKey(
    String host,
    int port,
    SSHHostKey hostKey,
) async {
    final fingerprint = hostKey.fingerprint;
    final knownResult = await _knownHosts.verify(host, port,
        fingerprint);

    switch (knownResult) {
        case KnownHostResult.trusted:
            return true;
        case KnownHostResult.unknown:
            // Surface to UI for user confirmation
            return await _promptUserForUnknownHost(host, port,
                fingerprint);
        case KnownHostResult.mismatch:
            // MITM warning – block connection
            await _showMitmWarning(host, port);
            return false;
    }
}

Future<SSHSocket> _buildJumpSocket(
    List<JumpHost> hops,
    SshConnectionParams finalTarget,
) async {
    // Build an SSH channel through each hop
    SSHSocket currentSocket = await _buildDirectSocket(
        hops[0].hostname,
        hops[0].port,
    );

    for (int i = 0; i < hops.length; i++) {
        final hop = hops[i];
        final nextHost = i + 1 < hops.length
            ? hops[i + 1].hostname
            : finalTarget.hostname;
        final nextPort = i + 1 < hops.length
            ? hops[i + 1].port
            : finalTarget.port;
    }
}

```

```

        final hopClient = SSHClient(
            currentSocket,
            username: hop.username,
            identities: await _buildIdentities(hop.authMethod),
        );
        await hopClient.authenticated;

        // Forward through to next hop
        currentSocket = await hopClient.forwardLocal(nextHost,
            nextPort);
    }

    return currentSocket;
}

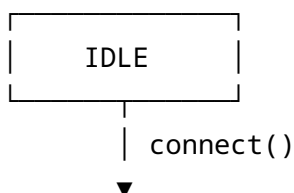
Future<SSHSocket> _buildDirectSocket(String host, int port)
    async {
        return SSHSocket.connect(host, port);
    }

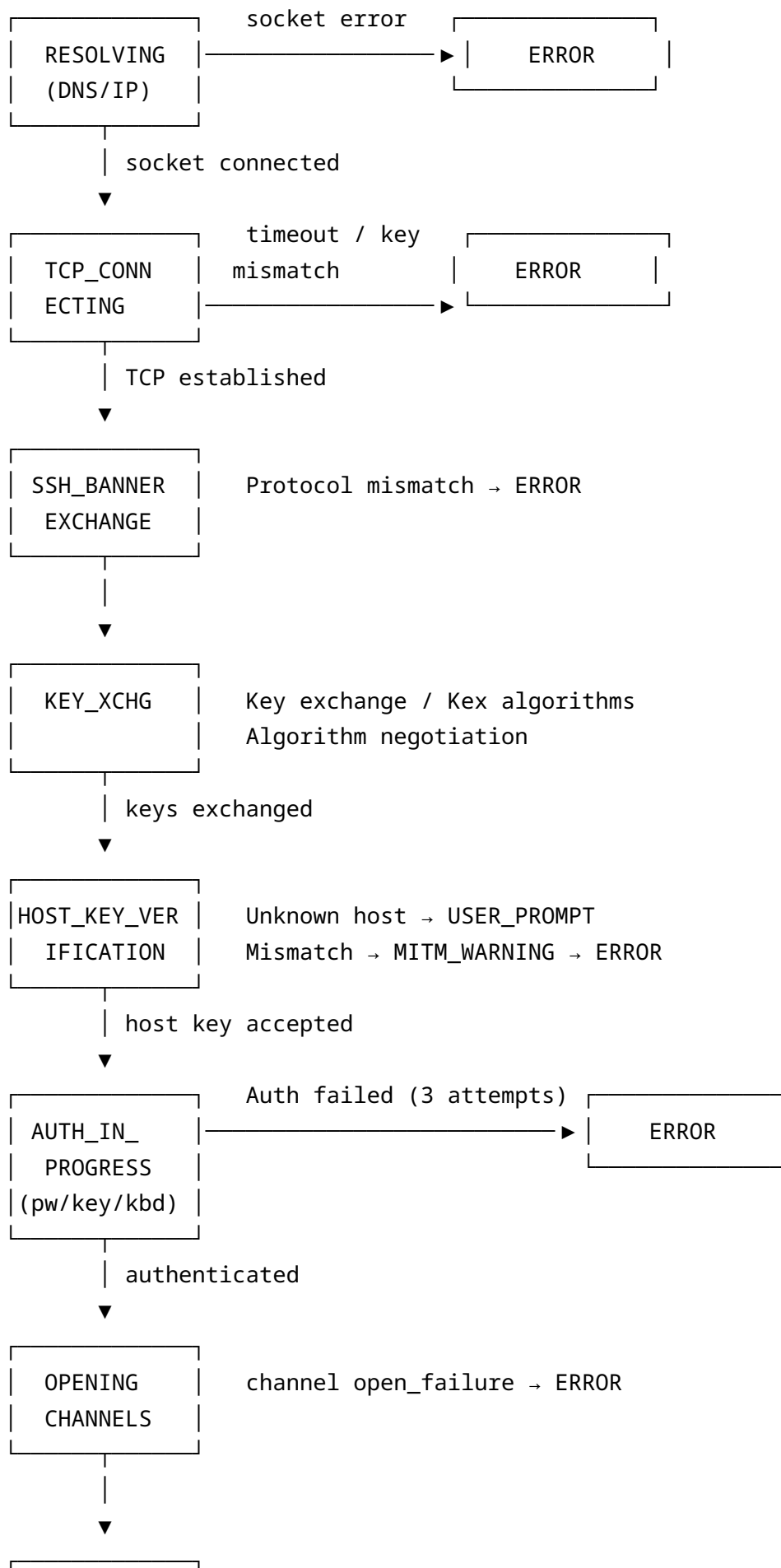
Future<List<SSHKeyPair>> _buildIdentities(SshAuthMethod method)
    async {
        if (method is SshKeyAuth) {
            final keyPem = await _secureStorage.read(method.keyId);
            if (keyPem == null) return [];
            String? passphrase;
            if (method.passphraseRef != null) {
                passphrase = await
                    _secureStorage.read(method.passphraseRef!);
            }
            return [SSHKeyPair.fromPem(keyPem, passphrase)];
        }
        return [];
    }
}

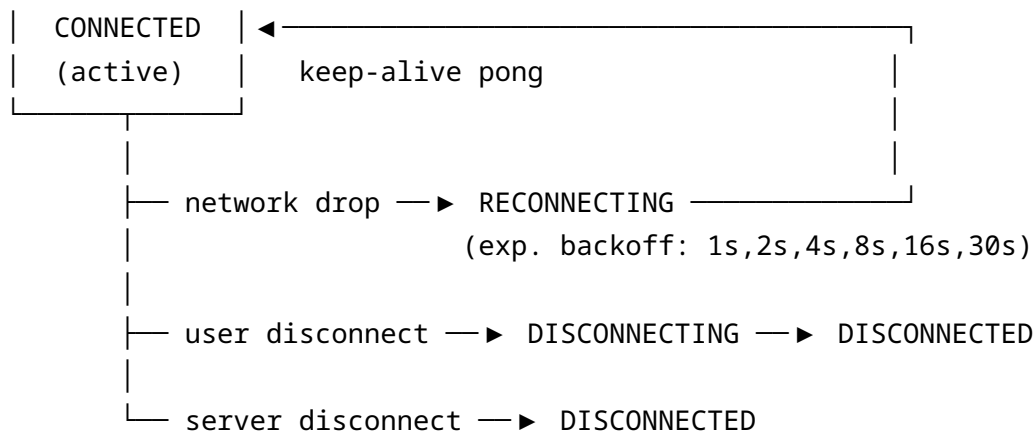
```

3.2 Connection State Machine

The SSH connection lifecycle is governed by a finite state machine with well-defined transitions:







```
// features/ssh_session/domain/entities/
connection_state_machine.dart
```

```
enum SshConnectionState {
  idle,
  resolving,
  tcpConnecting,
  sshBannerExchange,
  keyExchange,
  hostKeyVerification,
  authInProgress,
  openingChannels,
  connected,
  reconnecting,
  disconnecting,
  disconnected,
  error,
}

class SshStateMachine {
  SshConnectionState _state = SshConnectionState.idle;
  String? _errorMessage;
  int _reconnectAttempts = 0;
  static const int _maxReconnectAttempts = 6;
  static const List<Duration> _reconnectDelays = [
    Duration(seconds: 1),
    Duration(seconds: 2),
    Duration(seconds: 4),
    Duration(seconds: 8),
    Duration(seconds: 16),
    Duration(seconds: 30),
  ];

  SshConnectionState get state => _state;
```

```

bool get canReconnect => _reconnectAttempts <
    _maxReconnectAttempts;

Duration get nextReconnectDelay {
    final idx = _reconnectAttempts.clamp(0,
        _reconnectDelays.length - 1);
    return _reconnectDelays[idx];
}

void transition(SshConnectionState newState) {
    assert(_isValidTransition(_state, newState),
        'Invalid transition: $_state → $newState');
    _state = newState;
    if (newState == SshConnectionState.reconnecting) {
        _reconnectAttempts++;
    } else if (newState == SshConnectionState.connected) {
        _reconnectAttempts = 0;
    }
}

bool _isValidTransition(SshConnectionState from,
    SshConnectionState to) {
    const validTransitions = {
        SshConnectionState.idle: {
            SshConnectionState.resolving,
        },
        SshConnectionState.resolving: {
            SshConnectionState.tcpConnecting,
            SshConnectionState.error,
        },
        SshConnectionState.tcpConnecting: {
            SshConnectionState.sshBannerExchange,
            SshConnectionState.error,
        },
        SshConnectionState.sshBannerExchange: {
            SshConnectionState.keyExchange,
            SshConnectionState.error,
        },
        SshConnectionState.keyExchange: {
            SshConnectionState.hostKeyVerification,
            SshConnectionState.error,
        },
        SshConnectionState.hostKeyVerification: {
            SshConnectionState.authInProgress,
            SshConnectionState.error,
        },
    };
    return validTransitions[from]!.contains(to);
}

```

```

    },
    SshConnectionState.authInProgress: {
      SshConnectionState.openingChannels,
      SshConnectionState.error,
    },
    SshConnectionState.openingChannels: {
      SshConnectionState.connected,
      SshConnectionState.error,
    },
    SshConnectionState.connected: {
      SshConnectionState.reconnecting,
      SshConnectionState.disconnecting,
      SshConnectionState.disconnected,
    },
    SshConnectionState.reconnecting: {
      SshConnectionState.resolving,
      SshConnectionState.disconnected,
      SshConnectionState.error,
    },
    SshConnectionState.disconnecting: {
      SshConnectionState.disconnected,
    },
  },
};
return validTransitions[from]?.contains(to) ?? false;
}
}

```

3.3 Connection Multiplexing

HelixTerminator implements a ControlMaster-equivalent connection pool that shares a single SSH transport for multiple sessions to the same host.

```

// features/ssh_session/data/services/ssh_connection_pool.dart
@singleton
class SshConnectionPool {
  final Map<_PoolKey, _PooledConnection> _pool = {};
  final int maxConnections;
  final Duration keepAliveInterval;

  SshConnectionPool({
    this.maxConnections = 50,
    this.keepAliveInterval = const Duration(seconds: 30),
  });
}

```

```

Future<SSHClient> getOrCreate(SshConnectionParams params) async
{
    final key = _PoolKey.fromParams(params);

    if (_pool.containsKey(key)) {
        final pooled = _pool[key]!;
        if (pooled.isAlive) {
            pooled.refCount++;
            return pooled.client;
        } else {
            _pool.remove(key);
        }
    }

    // Create new connection
    final client = await _createConnection(params);
    _pool[key] = _PooledConnection(
        client: client,
        createdAt: DateTime.now(),
        refCount: 1,
    );

    // Enforce pool size limit
    if (_pool.length > maxConnections) {
        _evictOldest();
    }

    return client;
}

void release(SshConnectionParams params) {
    final key = _PoolKey.fromParams(params);
    final pooled = _pool[key];
    if (pooled != null) {
        pooled.refCount--;
        if (pooled.refCount <= 0) {
            // Keep connection alive for potential reuse
            // Will be evicted after idle timeout
            pooled.lastReleasedAt = DateTime.now();
        }
    }
}
}

```

```

void _evictOldest() {
    if (_pool.isEmpty) return;
    final oldest = _pool.entries
        .where((e) => e.value.refCount <= 0)
        .reduce((a, b) =>

            a.value.lastReleasedAt!.isBefore(b.value.lastReleasedAt!)
                ? a
                : b);
    _pool.remove(oldest.key);
    oldest.value.client.close();
}
}

```

```

class _PoolKey {
    final String hostname;
    final int port;
    final String username;

    const _PoolKey({
        required this.hostname,
        required this.port,
        required this.username,
    });

    factory _PoolKey.fromParams(SshConnectionParams p) => _PoolKey(
        hostname: p.hostname,
        port: p.port,
        username: p.username,
    );
}

```

```

@override
bool operator ==(Object other) =>
    other is _PoolKey &&
    other.hostname == hostname &&
    other.port == port &&
    other.username == username;

```

```

@override
int get hashCode => Object.hash(hostname, port, username);
}

```

```

class _PooledConnection {
    final SSHClient client;
}

```

```

    final DateTime createdAt;
    int refCount;
    DateTime? lastReleasedAt;

    _PooledConnection({
      required this.client,
      required this.createdAt,
      required this.refCount,
    });

    bool get isAlive => !client.isClosed;
  }

```

3.4 Auto-Reconnect with Exponential Backoff

```

// features/ssh_session/data/services/reconnect_manager.dart
class ReconnectManager {
  final SshConnectionService _connectionService;
  final SshStateMachine _stateMachine;
  final StreamController<ReconnectEvent> _events =
    StreamController.broadcast();

  Timer? _reconnectTimer;
  bool _isCancelled = false;

  Stream<ReconnectEvent> get events => _events.stream;

  ReconnectManager({
    required SshConnectionService connectionService,
    required SshStateMachine stateMachine,
  }) : _connectionService = connectionService,
      _stateMachine = stateMachine;

  void scheduleReconnect(SshConnectionParams params) {
    if (!_stateMachine.canReconnect || _isCancelled) return;

    final delay = _stateMachine.nextReconnectDelay;
    _stateMachine.transition(SshConnectionState.reconnecting);

    _events.add(ReconnectScheduled(
      attempt: _stateMachine._reconnectAttempts,
      delay: delay,
    ));
  }

```

```

_reconnectTimer = Timer(delay, () async {
    if (_isCancelled) return;
    try {
        _events.add(ReconnectAttempting());
        _stateMachine.transition(SshConnectionState.resolving);
        final result = await _connectionService.connect(params);
        _stateMachine.transition(SshConnectionState.connected);
        _events.add(ReconnectSucceeded(result: result));
    } catch (e) {
        _events.add(ReconnectFailed(error: e.toString()));
        if (_stateMachine.canReconnect) {
            scheduleReconnect(params);
        } else {
            _stateMachine.transition(SshConnectionState.error);
            _events.add(ReconnectExhausted());
        }
    }
});
}

void cancel() {
    _isCancelled = true;
    _reconnectTimer?.cancel();
}

}

abstract class ReconnectEvent {}
class ReconnectScheduled extends ReconnectEvent {
    final int attempt;
    final Duration delay;
    ReconnectScheduled({required this.attempt, required
        this.delay});
}
class ReconnectAttempting extends ReconnectEvent {}
class ReconnectSucceeded extends ReconnectEvent {
    final SshSessionResult result;
    ReconnectSucceeded({required this.result});
}
class ReconnectFailed extends ReconnectEvent {
    final String error;
    ReconnectFailed({required this.error});
}
class ReconnectExhausted extends ReconnectEvent {}

```


3.5 Port Forwarding Implementation

```
// features/port_forwarding/data/services/
port_forward_service.dart

class PortForwardService {
  final Map<String, _ActiveForward> _activeForwards = {};

  Future<void> startLocalForward(
    SSHClient client,
    PortForwardRule rule,
  ) async {
    // Local forward: bind local port, tunnel to remote host:port
    final localSocket = await ServerSocket.bind(
      rule.bindAddress,
      rule.localPort,
    );

    final forward = _ActiveForward(
      rule: rule,
      localSocket: localSocket,
    );
    _activeForwards[rule.id] = forward;

    localSocket.listen((socket) async {
      try {
        final channel = await client.forwardLocal(
          rule.remoteHost,
          rule.remotePort,
        );
        _pipeStreams(socket, channel);
      } catch (e) {
        socket.close();
      }
    });
  }

  Future<void> startRemoteForward(
    SSHClient client,
    PortForwardRule rule,
  ) async {
    // Remote forward: bind remote port, tunnel to local host:port
    final serverChannel = await client.forwardRemote(
      remoteHost: rule.bindAddress,
      remotePort: rule.remotePort,
```

```

);

serverChannel.channelsStream.listen((channel) async {
  try {
    final localSocket = await Socket.connect(
      rule.remoteHost,
      rule.localPort,
    );
    _pipeStreams(localSocket, channel);
  } catch (e) {
    channel.close();
  }
});
}

```

```

Future<void> startDynamicForward(
  SSHClient client,
  PortForwardRule rule,
) async {
  // Dynamic SOCKS5 proxy
  final proxyServer = await ServerSocket.bind(
    rule.bindAddress,
    rule.localPort,
  );

  proxyServer.listen((socket) async {
    final socks5Handler = Socks5Handler(
      onConnect: (host, port) async {
        return await client.forwardLocal(host, port);
      },
    );
    await socks5Handler.handle(socket);
  });
}

```

```

void _pipeStreams(Socket socket, SSHChannel channel) {
  socket.listen(
    (data) => channel.sink.add(data),
    onDone: () => channel.close(),
    onError: (_) => channel.close(),
  );
  channel.stream.listen(
    (data) => socket.add(data),
    onDone: () => socket.close(),
  );
}

```

```

        onError: (_) => socket.close(),
    );
}

Future<void> stopForward(String ruleId) async {
    final forward = _activeForwards.remove(ruleId);
    await forward?.localSocket?.close();
}
}

```

3.6 SFTP Subsystem

```

// features/sftp/data/services/sftp_service.dart
class SftpService {
    Future<SftpClient> openSftpSubsystem(SSHClient sshClient) async
    {
        return await sshClient.sftp();
    }

    Future<List<SftpEntry>> listDirectory(
        SftpClient sftp,
        String path,
    ) async {
        final dir = await sftp.open(path, mode:
            SftpFileOpenMode.read);
        // Use readdir for directory listing
        final rawEntries = await sftp.listdir(path);
        return rawEntries
            .where((e) => e.filename != '.' && e.filename != '..')
            .map((e) => SftpEntry(
                name: e.filename,
                path: '$path/${e.filename}',
                type: _mapEntryType(e.attr.type),
                size: e.attr.size ?? 0,
                modifiedAt: DateTime.fromMillisecondsSinceEpoch(
                    (e.attr.modifyTime ?? 0) * 1000),
                permissions: _formatPermissions(e.attr.permissions ??
                    0),
                owner: e.attr.userId?.toString() ?? '0',
                group: e.attr.groupId?.toString() ?? '0',
                isSymlink: e.attr.type == SftpFileType.link,
            ))
            .toList();
    }
}

```

```

Future<void> uploadFile(
    SftpClient sftp,
    String localPath,
    String remotePath, {
    void Function(int sent, int total)? onProgress,
}) async {
    final localFile = File(localPath);
    final totalBytes = await localFile.length();
    int sentBytes = 0;

    final remoteFile = await sftp.open(
        remotePath,
        mode: SftpFileOpenMode.write |
            SftpFileOpenMode.create |
            SftpFileOpenMode.truncate,
    );

    final stream = localFile.openRead();
    await for (final chunk in stream) {
        await remoteFile.writeBytes(Uint8List.fromList(chunk));
        sentBytes += chunk.length;
        onProgress?.call(sentBytes, totalBytes);
    }

    await remoteFile.close();
}

Future<void> downloadFile(
    SftpClient sftp,
    String remotePath,
    String localPath, {
    void Function(int received, int total)? onProgress,
}) async {
    final remoteFile = await sftp.open(remotePath, mode:
        SftpFileOpenMode.read);
    final stat = await remoteFile.stat();
    final totalBytes = stat.size ?? 0;
    int receivedBytes = 0;

    final localFile = File(localPath);
    final sink = localFile.openWrite();

    await for (final chunk in remoteFile.read()) {
        sink.add(chunk);
    }
}

```

```

        receivedBytes += chunk.length;
        onProgress?.call(receivedBytes, totalBytes);
    }

    await sink.close();
    await remoteFile.close();
}

SftpEntryType _mapEntryType(SftpFileType? type) {
    switch (type) {
        case SftpFileType.directory:
            return SftpEntryType.directory;
        case SftpFileType.link:
            return SftpEntryType.symlink;
        case SftpFileType.socket:
            return SftpEntryType.socket;
        case SftpFileType.pipe:
            return SftpEntryType.pipe;
        case SftpFileType.blockDevice:
            return SftpEntryType.blockDevice;
        case SftpFileType.charDevice:
            return SftpEntryType.charDevice;
        default:
            return SftpEntryType.file;
    }
}

String _formatPermissions(int mode) {
    final sb = StringBuffer();
    // File type
    sb.write((mode & 0x4000) != 0 ? 'd' : '-');
    // Owner
    sb.write((mode & 0x100) != 0 ? 'r' : '-');
    sb.write((mode & 0x80) != 0 ? 'w' : '-');
    sb.write((mode & 0x40) != 0 ? 'x' : '-');
    // Group
    sb.write((mode & 0x20) != 0 ? 'r' : '-');
    sb.write((mode & 0x10) != 0 ? 'w' : '-');
    sb.write((mode & 0x8) != 0 ? 'x' : '-');
    // Others
    sb.write((mode & 0x4) != 0 ? 'r' : '-');
    sb.write((mode & 0x2) != 0 ? 'w' : '-');
    sb.write((mode & 0x1) != 0 ? 'x' : '-');
    return sb.toString();
}

```

```
}  
}
```

3.7 Connection Error Taxonomy

`TerminalError(message)` (§1.10) was, until this revision, the single generic terminal-level failure state — every connect/auth/network/protocol failure rendered the same way, with no per-cause user guidance. This section replaces that with a closed, categorized error taxonomy shared by `TerminalBloc`, `SshConnectionService` (§3.1), and `ConnectivityService` (§4.4), so a failure mode determines both the message shown and the recovery action offered — never a single opaque string.

```
// core/error/connection_error.dart  
enum ConnectionErrorCategory {  
  
    authFailed,          // credentials/key/MFA rejected by the  
    remote host  
    hostUnreachable,    // TCP connect timeout / DNS failure / no  
    route  
  
    hostKeyMismatch,    // known_hosts fingerprint changed –  
    possible MITM  
    hostKeyUnknown,     // first-time connection, fingerprint  
    unverified  
    certificateExpired, // SSH certificate (short-lived cert auth)  
    past validity  
    timeout,            // TCP connected, protocol handshake did not  
    complete  
    networkChanged,     // client-side connectivity dropped mid-  
    session  
  
    serverRejected,     // server-side policy denial (RBAC  
    scope, org suspended, rate limit)  
    protocolError,      // malformed SSH/SFTP frame, unsupported  
    cipher/kex  
    vaultLocked,        // credential fetch blocked – vault requires  
    unlock (§5.1)  
  
    unknown,            // uncategorized – always paired with  
    the raw underlying message  
}  
  
class ConnectionError {  
    final ConnectionErrorCategory category;  
    final String rawMessage;    // underlying exception text, for  
    logs/support only  
    final bool isRetryable;  
    final bool requiresUserAction;
```

```

const ConnectionError({
  required this.category,
  required this.rawMessage,
  required this.isRetryable,
  required this.requiresUserAction,
});

factory ConnectionError.classify(Object error) {
  if (error is SSHAuthFailError) {
    return const ConnectionError(
      category: ConnectionErrorCategory.authFailed,
      rawMessage: 'SSH authentication rejected',
      isRetryable: true,
      requiresUserAction: true,
    );
  }
  if (error is SocketException) {
    return ConnectionError(
      category: ConnectionErrorCategory.hostUnreachable,
      rawMessage: error.message,
      isRetryable: true,
      requiresUserAction: false,
    );
  }
  if (error is TimeoutException) {
    return const ConnectionError(
      category: ConnectionErrorCategory.timeout,
      rawMessage: 'Connection attempt timed out',
      isRetryable: true,
      requiresUserAction: false,
    );
  }
  if (error is HostKeyMismatchException) {
    return const ConnectionError(
      category: ConnectionErrorCategory.hostKeyMismatch,
      rawMessage: 'Host key fingerprint changed since last
        connection',
      isRetryable: false,
      requiresUserAction: true,
    );
  }
  // ... remaining categories map from their respective
  dartssh2 /

```

```

// ApiClient / VaultRepository exception types via the same
pattern.
return ConnectionError(
    category: ConnectionErrorCategory.unknown,
    rawMessage: error.toString(),
    isRetryable: true,
    requiresUserAction: false,
);
}
}

```

Per-category user-facing treatment (rendered by `_TerminalErrorView`, §2.2, replacing the single generic message):

Category	User-facing message	Primary action	Secondary action
authFailed	“Sign-in was rejected by <host>. Check your key, password, or MFA code.”	Edit credentials	Retry
hostUnreachable	“Can’t reach <host>. Check the address, port, and your network connection.”	Retry	Edit host
hostKeyMismatch	“This host’s identity changed since your last connection. This can indicate a security problem.”	View fingerprint diff	Cancel (never auto-trust)
hostKeyUnknown	“First connection to <host>. Verify the fingerprint below before trusting it.”	Trust & connect	Cancel

Category	User-facing message	Primary action	Secondary action
certificateExpired	“Your SSH certificate for <host> has expired.”	Request new certificate	Cancel
timeout	“<host> did not respond in time.”	Retry	Edit host
networkChanged	“Connection lost — network changed.”	Auto-reconnect (silent, §3.4)	Manual retry
serverRejected	“<host> is not accessible with your current permissions.”	Contact admin	Dismiss
protocolError	“Unexpected response from <host>. This may indicate an incompatible SSH configuration.”	Retry	Report issue
vaultLocked	“Unlock your vault to retrieve credentials for <host>.”	Unlock vault	Cancel
unknown	“Something went wrong connecting to <host>.”	Retry	Report issue (attaches rawMessage)

hostKeyMismatch is never auto-dismissable and never offers a one-tap “trust anyway” — it always routes through the fingerprint-diff view (reusing `_showMitmWarning`, §3.1) requiring explicit confirmation, consistent with the MITM-defense posture already specified for `_verifyHostKey`.

4. Offline Mode

4.1 Offline Capability Overview

HelixTerminator provides rich offline functionality by maintaining a synchronized local SQLite database (via drift). The client can operate fully without network access for browsing hosts, editing snippets, viewing session logs, and preparing port forwarding rules.

What works offline:

Feature	Offline Support	Notes
View host list	Full	Read from local SQLite
Edit host settings	Full	Saved locally, synced on reconnect
Create/edit snippets	Full	Saved locally with dirty flag
Browse session logs	Full	Read from local SQLite
View SSH keys (public)	Full	Public keys in SQLite
Connect to SSH host	No	Requires network
Vault unlock (biometric)	Partial	If session token cached in secure storage
AI autocomplete	No	Requires server
Collaboration	No	Requires WebSocket

4.2 Sync Strategy

```
// core/sync/sync_manager.dart
@singleton
class SyncManager {
  final AppDatabase _db;
  final ApiClient _api;
  final ConnectivityService _connectivity;
  final ConflictResolver _conflictResolver;

  SyncManager({
    required AppDatabase db,
```

```

        required ApiClient api,
        required ConnectivityService connectivity,
        required ConflictResolver conflictResolver,
    }) : _db = db,
        _api = api,
        _connectivity = connectivity,
        _conflictResolver = conflictResolver;

Future<SyncResult> performFullSync() async {
    if (!await _connectivity.isConnected()) {
        return SyncResult.skipped(reason: 'No network connection');
    }

    final results = await Future.wait([
        _syncHosts(),
        _syncSnippets(),
        _syncPortForwardingRules(),
    ]);

    return SyncResult.merged(results);
}

Future<SyncEntityResult> _syncHosts() async {
    // 1. Get remote changes since last sync
    final lastSyncAt = await _getLastSyncTimestamp('hosts');
    final remoteChanges = await _api.getHostChanges(since:
        lastSyncAt);

    // 2. Get local dirty records
    final localDirty = await _db.hosts
        .where()
        .isDirtyEqualTo(true)
        .get();

    // 3. Resolve conflicts. `hosts` is not zero-knowledge vault
    // data (the
    // server already sees plaintext host records), so
    // lastWriteWins is
    // acceptable here. Vault entity sync MUST use
    // ConflictStrategy.crdtVectorClock instead – see §4.3.
    final resolved = await _conflictResolver.resolve(
        localRecords: localDirty,
        remoteChanges: remoteChanges,
        strategy: ConflictStrategy.lastWriteWins,
    );
}

```

```

// 4. Apply resolved changes
await _db.transaction(() async {
    for (final record in resolved.toApplyLocally) {
        await _db.hosts.replace(record);
    }
});

// 5. Push local changes to server
for (final local in resolved.toPushToRemote) {
    await _api.upsertHost(local);
}

// 6. Clear dirty flags
await _db.hosts.update()
    ..where((t) => t.isDirty.equals(true))
    ..write(const HostsCompanion(isDirty: Value(false)));

await _updateLastSyncTimestamp('hosts', DateTime.now());
return SyncEntityResult(
    entity: 'hosts',
    pulled: remoteChanges.length,
    pushed: resolved.toPushToRemote.length,
    conflicts: resolved.conflictsResolved,
);
}
}

```

4.3 Conflict Resolution

CANONICAL: `lastWriteWins` MUST NOT be used for vault data. Because vault items are zero-knowledge / client-side end-to-end encrypted (server never sees plaintext), a timestamp-based “last write wins” merge can silently discard a divergent encrypted edit with no way for either client to detect or recover the loss. Vault entity sync MUST use `ConflictStrategy.crdtVectorClock`: each client maintains a per-item vector clock (`Map<deviceId, int>`), advances its own counter on every local mutation, and a remote change is merged rather than overwritten whenever the two clocks are concurrent (neither dominates the other) — in that case both encrypted versions are retained and surfaced for user-driven manual merge instead of being silently dropped. `lastWriteWins` remains acceptable for non-secret entities (hosts, snippets, port-forwarding rules) where the server can already

see the plaintext and a clobbered edit is low-risk and recoverable via audit log. (DEEP-WORK: full item-level vault vector-clock wire format, gossip/merge protocol, and key-rotation interplay — next increment.)

```
// core/sync/conflict_resolver.dart
enum ConflictStrategy {
  lastWriteWins,
  serverWins,
  clientWins,
  manualMerge,
  crdtVectorClock, // REQUIRED for vault data – see note above
}

class ConflictResolver {
  Future<ResolvedChanges<T>> resolve<T extends SyncableEntity>({
    required List<T> localRecords,
    required List<T> remoteChanges,
    required ConflictStrategy strategy,
  }) async {
    final toApplyLocally = <T>[];
    final toPushToRemote = <T>[];
    final pendingManualMerges = <ConflictDetected<T>>[];
    int conflictsResolved = 0;

    final remoteMap = {for (final r in remoteChanges) r.remoteId:
      r};
    final localMap = {for (final l in localRecords) l.remoteId:
      l};

    // Records only in remote – apply locally
    for (final remote in remoteChanges) {
      if (!localMap.containsKey(remote.remoteId)) {
        toApplyLocally.add(remote);
      }
    }

    // Records only in local dirty – push to remote
    for (final local in localRecords) {
      if (!remoteMap.containsKey(local.remoteId)) {
        toPushToRemote.add(local);
      }
    }

    // Records in both – conflict
```

```

final conflictIds = localMap.keys
    .toSet()
    .intersection(remoteMap.keys.toSet());

for (final id in conflictIds) {
    final local = localMap[id]!;
    final remote = remoteMap[id]!;
    conflictsResolved++;

    switch (strategy) {
        case ConflictStrategy.lastWriteWins:
            if (remote.updatedAt.isAfter(local.updatedAt)) {
                toApplyLocally.add(remote);
            } else {
                toPushToRemote.add(local);
            }
            break;
        case ConflictStrategy.serverWins:
            toApplyLocally.add(remote);
            break;
        case ConflictStrategy.clientWins:
            toPushToRemote.add(local);
            break;
        case ConflictStrategy.manualMerge:
            // Neither side is auto-applied. Record a
            ConflictDetected entry
            // carrying both the local and remote record;
            SyncManager reads
            // ResolvedChanges.manualMerges and routes each one to
            // ConflictResolutionBloc, which drives the user to the
            Conflict
            // Resolution screen (§6.16) rather than resolving it
            here.
            pendingManualMerges.add(ConflictDetected<T>(local:
            local, remote: remote));
            break;
        case ConflictStrategy.crdtVectorClock:
            // Vault data only. Compare per-device vector clocks
            (not wall-clock
            // timestamps): if one clock causally dominates the
            other, the
            // dominant encrypted version wins; if the clocks are
            concurrent,
            // retain both encrypted versions and emit a manual-
            merge conflict
            // event rather than discarding either one.
            switch
            (local.vectorClock.compareTo(remote.vectorClock)) {

```

```

        case VectorClockOrder.localDominates:
            toPushToRemote.add(local);
            break;
        case VectorClockOrder.remoteDominates:
            toApplyLocally.add(remote);
            break;
        case VectorClockOrder.concurrent:
            // Do not pick a winner – surface both encrypted
            versions via
            // the same manual-merge queue `manualMerge` uses
            above, so
            // ConflictResolutionBloc (§6.16) has one intake
            path for both
            // non-secret and vault conflicts.
            pendingManualMerges.add(ConflictDetected<T>(local:
            local, remote: remote));
            break;
    }
    break;
}
}

return ResolvedChanges(
    toApplyLocally: toApplyLocally,
    toPushToRemote: toPushToRemote,
    conflictsResolved: conflictsResolved,
    manualMerges: pendingManualMerges,
);
}
}

```

4.4 Network Change Detection

```

// core/network/connectivity_service.dart
@singleton
class ConnectivityService {
    final Connectivity _connectivity = Connectivity();
    late final StreamController<ConnectivityResult> _controller;

    ConnectivityService() {
        _controller = StreamController.broadcast();
        _connectivity.onConnectivityChanged.listen((result) {
            _controller.add(result);
            if (result != ConnectivityResult.none) {
                _onNetworkRestored();
            }
        });
    }
}

```

```

    }
  });
}

Stream<ConnectivityResult> get changes => _controller.stream;

Future<bool> isConnected() async {
  final result = await _connectivity.checkConnectivity();
  return result != ConnectivityResult.none;
}

void _onNetworkRestored() {
  // Trigger sync and SSH reconnection attempts
  getIt<SyncManager>().performFullSync();
  getIt<SshConnectionPool>().reconnectAll();
}
}

```

4.5 Offline SQLite Schema Details

The local SQLite database mirrors the server-side data model with additional sync metadata columns:

- `remote_id` (UUID): server-assigned identifier
 - `sync_etag` (string): ETag from last server response for this record
 - `is_dirty` (bool): modified locally since last sync
 - `is_deleted` (bool): marked for deletion, not yet pushed to server
 - `updated_at` (datetime): local modification timestamp for conflict detection
-

5. Security on Client

5.1 Vault Decryption Architecture

The vault key hierarchy:

```

User Password
  ↓ Argon2id(iterations=3, memory=65536, parallelism=4)
Vault Encryption Key (VEK, 256-bit)
  ↓ AES-256-GCM
Vault Root Key (VRK, 256-bit) ← encrypted, stored on server
  ↓ AES-256-GCM

```


Individual Item Keys (per-item, 256-bit)

↓ AES-256-GCM

Plaintext Item Data

```
// features/vault/data/services/vault_crypto_service.dart
import 'package:cryptography/cryptography.dart';

class VaultCryptoService {
  static const int argon2Iterations = 3;
  static const int argon2Memory = 65536;    // 64 MB
  static const int argon2Parallelism = 4;
  static const int saltLength = 16;
  static const int keyLength = 32;         // 256-bit

  final Argon2id _argon2 = Argon2id(
    parallelism: argon2Parallelism,
    memorySize: argon2Memory,
    iterations: argon2Iterations,
    hashLength: keyLength,
  );

  final AesGcm _aesGcm = AesGcm.with256bits();

  Future<Uint8List> deriveVaultEncryptionKey(
    String password,
    Uint8List salt,
  ) async {
    final secretKey = await _argon2.deriveKey(
      secretKey: SecretKey(utf8.encode(password)),
      nonce: salt,
    );
    final keyBytes = await secretKey.extractBytes();
    // Zero the intermediate secret key
    secretKey.destroy();
    return Uint8List.fromList(keyBytes);
  }

  Future<Uint8List> decryptVaultRootKey(
    Uint8List encryptedVrk,
    Uint8List vek,
    Uint8List nonce,
  ) async {
    final secretKey = SecretKey(vek);
    final mac = Mac(encryptedVrk.sublist(encryptedVrk.length -
      16));
```

```

    final ciphertext = encryptedVrk.sublist(0,
        encryptedVrk.length - 16);

    final secretBox = SecretBox(ciphertext, nonce: nonce, mac:
        mac);
    final plaintext = await _aesGcm.decrypt(secretBox, secretKey:
        secretKey);

    // Zero the VEK bytes after use
    vek.fillRange(0, vek.length, 0);
    secretKey.destroy();

    return Uint8List.fromList(plaintext);
}

Future<Uint8List> decryptItem(
    Uint8List encryptedData,
    Uint8List itemKey,
    Uint8List nonce,
) async {
    final secretKey = SecretKey(itemKey);
    final mac = Mac(encryptedData.sublist(encryptedData.length -
        16));
    final ciphertext = encryptedData.sublist(0,
        encryptedData.length - 16);

    final secretBox = SecretBox(ciphertext, nonce: nonce, mac:
        mac);
    final plaintext = await _aesGcm.decrypt(secretBox, secretKey:
        secretKey);

    // Zero item key after use
    itemKey.fillRange(0, itemKey.length, 0);
    secretKey.destroy();

    return Uint8List.fromList(plaintext);
}

/// Generate a cryptographically random nonce for AES-GCM (96-
    bit / 12 bytes)
Uint8List generateNonce() {
    final random = Random.secure();
    return Uint8List.fromList(
        List.generate(12, (_) => random.nextInt(256)));
}

/// Securely zero a byte buffer

```

```

    static void zeroBytes(Uint8List bytes) {
      bytes.fillRange(0, bytes.length, 0);
    }
  }
}

```

5.2 Platform Secure Storage

```

// platform/secure_storage/secure_storage_service.dart
import 'package:flutter_secure_storage/
  flutter_secure_storage.dart';

@singleton
class SecureStorageService {
  late final FlutterSecureStorage _storage;

  SecureStorageService() {
    _storage = FlutterSecureStorage(
      iOptions: const IOSOptions(
        // iOS Keychain:
        kSecAttrAccessibleAfterFirstUnlockThisDeviceOnly
        accessibility:
        KeychainAccessibility.first_unlock_this_device,
        synchronizable: false, // Don't sync to iCloud
      ),
      aOptions: const AndroidOptions(
        encryptedSharedPreferences: true,
        // Android KeyStore backed encryption
      ),
      mOptions: const MacOSOptions(
        accessibility:
        KeychainAccessibility.first_unlock_this_device,
        synchronizable: false,
      ),
      wOptions: const WindowsOptions(
        useBackwardCompatibility: false,
        // Uses DPAPI (Data Protection API)
      ),
      lOptions: const LinuxOptions(
        // Uses libsecret (GNOME Keyring / KWallet)
      ),
    );
  }

  Future<void> write(String key, String value) async {
    await _storage.write(key: key, value: value);
  }
}

```

```

}

Future<String?> read(String key) async {
  return await _storage.read(key: key);
}

Future<void> delete(String key) async {
  await _storage.delete(key: key);
}

Future<void> deleteAll() async {
  await _storage.deleteAll();
}

// Store binary data (SSH keys, etc.) as base64
Future<void> writeBytes(String key, Uint8List bytes) async {
  await write(key, base64.encode(bytes));
}

Future<Uint8List?> readBytes(String key) async {
  final value = await read(key);
  if (value == null) return null;
  return base64.decode(value);
}
}

```

5.3 Biometric Authentication

```

// platform/biometrics/biometrics_service.dart
import 'package:local_auth/local_auth.dart';

@singleton
class BiometricsService {
  final LocalAuthentication _auth = LocalAuthentication();

  Future<BiometricCapability> checkCapability() async {
    final isAvailable = await _auth.isDeviceSupported();
    if (!isAvailable) return BiometricCapability.unavailable;

    final biometrics = await _auth.getAvailableBiometrics();
    if (biometrics.contains(BiometricType.face)) {
      return BiometricCapability.faceId;
    }
    if (biometrics.contains(BiometricType.fingerprint)) {

```

```

        return BiometricCapability.touchId;
    }
    if (biometrics.contains(BiometricType.strong)) {
        return BiometricCapability.strong;
    }
    return BiometricCapability.deviceCredential;
}

```

```

Future<bool> authenticate({
    required String reason,
    bool sensitiveTransaction = true,
}) async {
    try {
        return await _auth.authenticate(
            localizedReason: reason,
            options: AuthenticationOptions(
                biometricOnly: false,
                stickyAuth: true,
                sensitiveTransaction: sensitiveTransaction,
                useErrorDialogs: true,
            ),
        );
    } on PlatformException catch (e) {
        if (e.code == auth_error.lockedOut ||
            e.code == auth_error.permanentlyLockedOut) {
            throw BiometricLockedException();
        }
        rethrow;
    }
}

```

```

/// Enroll biometric – saves a protected token in secure storage
Future<void> enrollBiometric(String vaultSessionToken) async {
    final authenticated = await authenticate(
        reason: 'Verify your identity to enable biometric login',
    );
    if (!authenticated) throw BiometricAuthFailedException();

    // Store the session token under biometric protection
    await getIt<SecureStorageService>().write(
        'biometric_session_token',
        vaultSessionToken,
    );
}

```

```

    /// Retrieve biometric-protected token
    Future<String?> getBiometricToken() async {
      final authenticated = await authenticate(
        reason: 'Authenticate to open HelixTerminator',
      );
      if (!authenticated) return null;
      return await
        getIt<SecureStorageService>().read('biometric_session_token');
    }
  }
}

```

```

enum BiometricCapability {
  unavailable,
  deviceCredential,
  strong,
  touchId,
  faceId,
}

```

5.4 Screen Capture Protection

```

// platform/security/screen_protection_service.dart
class ScreenProtectionService {
  static const _channel =
    MethodChannel('io.helixterminator.client/
      screen_protection');

  static Future<void> enable() async {
    await _channel.invokeMethod('enableScreenProtection');
  }

  static Future<void> disable() async {
    await _channel.invokeMethod('disableScreenProtection');
  }
}

// Android implementation (MainActivity.kt):
// window.setFlags(WindowManager.LayoutParams.FLAG_SECURE,
//                  WindowManager.LayoutParams.FLAG_SECURE)
//
// macOS implementation (AppDelegate.swift):
// view.layer?.isOpaque = false
// NSApp.windows.forEach { $0.sharingType = .none }

```

5.5 Auto-Lock

```
// features/security/auto_lock_service.dart
@singleton
class AutoLockService {
  final AppSettings _settings;
  final VaultCubit _vaultCubit;
  Timer? _lockTimer;
  DateTime _lastActivity = DateTime.now();

  AutoLockService({
    required AppSettings settings,
    required VaultCubit vaultCubit,
  }) : _settings = settings,
      _vaultCubit = vaultCubit;

  Duration get lockTimeout {
    final minutes = _settings.autoLockMinutes;
    if (minutes == 0) return Duration.zero; // Disabled
    return Duration(minutes: minutes);
  }

  void recordActivity() {
    _lastActivity = DateTime.now();
    _resetTimer();
  }

  void _resetTimer() {
    _lockTimer?.cancel();
    if (lockTimeout == Duration.zero) return;
    _lockTimer = Timer(lockTimeout, _lock);
  }

  void _lock() {
    _vaultCubit.lock();
    getIt<AuthBloc>().add(AuthSessionExpired());
  }

  void dispose() {
    _lockTimer?.cancel();
  }
}
```

5.6 Clipboard Auto-Clear

```
// features/security/clipboard_guard.dart
class ClipboardGuard {
  static const Duration _clearDelay = Duration(seconds: 30);
  Timer? _clearTimer;
  String? _sensitiveValue;

  Future<void> copySecure(String text) async {
    _sensitiveValue = text;
    await Clipboard.setData(ClipboardData(text: text));

    // Schedule auto-clear
    _clearTimer?.cancel();
    _clearTimer = Timer(_clearDelay, () async {
      // Only clear if our value is still on clipboard
      final current = await
        Clipboard.getData(Clipboard.kTextPlain);
      if (current?.text == _sensitiveValue) {
        await Clipboard.setData(const ClipboardData(text: ''));
      }
      _sensitiveValue = null;
    });
  }

  void cancel() {
    _clearTimer?.cancel();
    _sensitiveValue = null;
  }
}
```

5.7 FIDO2 Hardware Security Key

```
// platform/hardware_key/fido2_service.dart
class Fido2Service {
  static const _channel =
    MethodChannel('io.helixterminator.client/fido2');

  Future<Fido2Assertion> authenticate({
    required Uint8List challenge,
    required List<String> allowedCredentialIds,
    required String rpId,
  }) async {
    final result = await _channel.invokeMapMethod<String,
      dynamic>(
```



```

        'authenticate',
        {
            'challenge': base64.encode(challenge),
            'allowedCredentialIds': allowedCredentialIds,
            'rpId': rpId,
            'userVerification': 'preferred',
        },
    );

    if (result == null) throw Fido2AuthCancelledException();

    return Fido2Assertion(
        credentialId: base64.decode(result['credentialId'] as
            String),
        authenticatorData:
            base64.decode(result['authenticatorData'] as String),
        clientDataJSON: base64.decode(result['clientDataJSON'] as
            String),
        signature: base64.decode(result['signature'] as String),
    );
}

Future<Fido2Attestation> register({
    required Uint8List challenge,
    required String rpId,
    required String rpName,
    required String userId,
    required String userName,
}) async {
    final result = await _channel.invokeMapMethod<String,
        dynamic>(
        'register',
        {
            'challenge': base64.encode(challenge),
            'rpId': rpId,
            'rpName': rpName,
            'userId': userId,
            'userName': userName,
            'requireResidentKey': false,
            'userVerification': 'preferred',
        },
    );

    if (result == null) throw
        Fido2RegistrationCancelledException();
}

```

```
    return Fido2Attestation(  
      credentialId: base64.decode(result['credentialId'] as  
        String),  
      attestationObject:  
        base64.decode(result['attestationObject'] as String),  
      clientDataJSON: base64.decode(result['clientDataJSON'] as  
        String),  
    );  
  }  
}
```

6. UI/UX Complete Specification

6.1 Design System

Typography: - Primary: Inter (body, labels, buttons) - Monospace: JetBrains Mono (terminal, code, SSH commands) - Heading weights: 600 (SemiBold), 700 (Bold) - Body weight: 400 (Regular)

Color System:

```
// core/theme/app_colors.dart  
abstract class AppColors {  
  // Light mode  
  static const Color backgroundLight = Color(0xFFF5F5F5);  
  static const Color surfaceLight = Color(0xFFFFFFFF);  
  static const Color surfaceElevatedLight = Color(0xFFF0F0F0);  
  static const Color borderLight = Color(0xFFE0E0E0);  
  static const Color textPrimaryLight = Color(0xFF1A1A1A);  
  static const Color textSecondaryLight = Color(0xFF666666);  
  static const Color primaryLight = Color(0xFF0080FF);  
  static const Color primaryHoverLight = Color(0xFF0060CC);  
  static const Color errorLight = Color(0xFFE53935);  
  static const Color successLight = Color(0xFF43A047);  
  static const Color warningLight = Color(0xFFFB8C00);  
  
  // Dark mode (default – terminal apps are predominantly dark)  
  static const Color backgroundDark = Color(0xFF0D0D0D);  
  static const Color surfaceDark = Color(0xFF1A1A1A);  
  static const Color surfaceElevatedDark = Color(0xFF252525);  
  static const Color borderDark = Color(0xFF2E2E2E);  
  static const Color textPrimaryDark = Color(0xFFE8E8E8);  
  static const Color textSecondaryDark = Color(0xFF9A9A9A);
```

```

static const Color primaryDark = Color(0xFF4DA6FF);
static const Color primaryHoverDark = Color(0xFF3399FF);
static const Color errorDark = Color(0xFFEF5350);
static const Color successDark = Color(0xFF66BB6A);
static const Color warningDark = Color(0xFFFFA726);
}

```

Spacing Grid (8pt base): - xs: 4px - sm: 8px - md: 16px - lg: 24px - xl: 32px - 2xl: 48px - 3xl: 64px

Border Radius: - sm: 4px (inputs, chips) - md: 8px (cards, dialogs) - lg: 12px (sheets) - xl: 16px (large cards) - full: 999px (pills, avatars)

6.2 Screen: Onboarding / First Launch

Purpose: Welcome new users, establish vault, configure initial preferences.

Layout: Full-screen single-column, step-based wizard (5 steps).

Steps: 1. **Welcome** — Product name, tagline, feature highlights (3 bullet points), “Get Started” CTA 2. **Account** — Options: “Create Account”, “Sign In”, “Continue with SSO” 3. **Vault Setup** — Master password creation with strength indicator, passphrase alternative toggle, biometric setup prompt 4. **Default Settings** — Font family selector, default theme, preferred port (22), proxy configuration 5. **Complete** — Confirmation, optional import (.ssh/config, previous backup)

State Variations: - initial — blank step 1 - loading — spinner overlay during account creation - error — inline error under relevant field - complete — animated checkmark, transition to host list

Component List: - OnboardingProgressBar — 5 dots, active dot scaled 1.5x - OnboardingStepCard — elevated card with icon, title, body - PasswordStrengthIndicator — 4-segment bar (weak/fair/good/strong) - OnboardingNavigation — Back + Next/Complete buttons

6.3 Screen: Login

Purpose: Authenticate returning users.

Layout: Centered card (max-width 400px), dark background.

```

// features/auth/presentation/pages/login_page.dart
class LoginPage extends StatelessWidget {

```

```
const LoginPage({super.key});
```

```
@override
```

```
Widget build(BuildContext context) {  
  return Scaffold(  
    backgroundColor: AppColors.backgroundDark,  
    body: BlocConsumer<AuthBloc, AuthState>(  
      listener: (context, state) {  
        if (state is AuthMfaRequired) {  
          context.go('/login/mfa', extra: state.challenge);  
        }  
        if (state is AuthAuthenticated) {  
          context.go(RouteNames.hosts);  
        }  
        if (state is AuthError) {  
          ScaffoldMessenger.of(context).showSnackBar(  
            SnackBar(  
              content: Text(state.message),  
              backgroundColor: AppColors.errorDark,  
            ),  
          );  
        }  
      },  
      builder: (context, state) {  
        return Center(  
          child: ConstrainedBox(  
            constraints: const BoxConstraints(maxWidth: 400),  
            child: Padding(  
              padding: const EdgeInsets.all(24),  
              child: Column(  
                mainAxisAlignment: MainAxisAlignment.center,  
                children: [  
                  // Logo  
                  const HelixTerminatorLogo(size: 64),  
                  const SizedBox(height: 32),  
                  // Form card  
                  LoginCard(isLoading: state is AuthLoading),  
                  const SizedBox(height: 16),  
                  // SSO options  
                  const SsoButtons(),  
                  const SizedBox(height: 16),  
                  // Biometric if enrolled  
                  if (state is! AuthLoading)  
                    const BiometricLoginButton(),  
                ],  
              ),  
            ),  
          ),  
        );  
      },  
    ),  
  );  
}
```

```

        ],
    ),
),
),
);
},
),
);
}
}

```

State Variations: - idle — email/password form, SSO buttons, optional biometric button - loading — form fields disabled, spinner on submit button - mfa_required — transitions to MFA screen with slide animation - error — inline error with specific message (invalid credentials, account locked) - biometric_prompt — native OS biometric dialog overlaid

Component List: - LoginCard — Surface card with email TextField, password TextField (visibility toggle), “Remember me” checkbox, “Forgot password” text link, “Sign In” ElevatedButton - SsoButtons — Row of provider icons (Google, GitHub, Okta, Azure AD, Generic SAML) - BiometricLoginButton — Icon + label (“Use Face ID” / “Use Touch ID” / “Use Windows Hello”)

6.4 Screen: MFA Verification

Layout: Continues from login card (animated transition), same centered card.

State Variations: - totp — 6-digit OTP input with auto-submit on 6th digit - fido2 — “Tap your security key” instruction with animated icon - push — “Approve the push notification” with animated phone icon, timeout countdown - backup_code — 8-character code input - loading — spinner overlay on submit - error — shake animation + error message

Component List: - OtpInputField — 6 separate single-character inputs - MfaMethodSwitcher — Tab bar for available methods - Fido2WaitingIndicator — Animated NFC/USB icon - PushWaitingIndicator — Animated phone icon, countdown timer

6.5 Screen: Host List

Purpose: The main hub for managing and connecting to SSH hosts.

Layout (Desktop): Left sidebar (navigation + group tree) + main content area. **Layout (Mobile):** Bottom navigation + full-width list.

```
// features/hosts/presentation/pages/host_list_page.dart
class HostListPage extends StatelessWidget {
  const HostListPage({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocBuilder<HostListBloc, HostListState>(
      builder: (context, state) {
        return Scaffold(
          appBar: HostListAppBar(state: state),
          drawer: context.isNarrow ? const HostGroupDrawer() :
            null,
          body: Row(
            children: [
              // Sidebar (desktop only)
              if (!context.isNarrow)
                SizedBox(
                  width: 240,
                  child: HostGroupSidebar(state: state),
                ),
              // Main content
              Expanded(
                child: Column(
                  children: [
                    HostListSearchBar(state: state),
                    HostListFilterChips(state: state),
                    Expanded(
                      child: state is HostListLoaded
                        ? (state.viewMode ==
                          HostListViewMode.grid
                            ? HostGridView(hosts: state.hosts)
                            : HostListView(hosts: state.hosts))
                        : state is HostListLoading
                          ? const HostListShimmer()
                          : const HostListEmptyState(),
                    ),
                  ],
                ),
              ),
            ],
          ),
          floatingActionButton: FloatingActionButton.extended(
```

```

        onPressed: () => context.go('/hosts/new'),
        icon: const Icon(Icons.add),
        label: const Text('Add Host'),
      ),
    );
  },
);
}
}

```

Host Card (Grid Mode): - Connection status indicator (dot: green/gray/red) - Host label (bold, 16px) - username@hostname:port (secondary color, 12px) - Group tag chip - Last connected timestamp - Quick-connect button (play icon) - Context menu (Edit, Duplicate, Move to Group, Delete)

State Variations: - loading — shimmer skeleton cards - loaded_empty — illustration + “Add your first host” CTA - loaded_with_results — grid or list of host cards - search_empty — “No hosts match your search” with clear button - filter_active — filter chips shown with clear-all button - error — error card with retry button

Virtualization: The host list uses `flutter_layout_grid` for grid mode and `ListView.builder` for list mode. Both use lazy rendering — only visible items are built. Supports 10,000+ hosts without scroll jitter via `SliverList` with `SliverChildBuilderDelegate`.

6.6 Screen: Terminal (Main)

The terminal screen is the most feature-dense view in the application.

Layout: Full screen, minimal chrome.

```

// features/terminal/presentation/pages/terminal_page.dart
class TerminalPage extends StatelessWidget {
  final String? sessionId;
  const TerminalPage({this.sessionId, super.key});

  @override
  Widget build(BuildContext context) {
    return BlocProvider(
      create: (_) => getIt<TerminalBloc>()
        ..add(TerminalSessionStarted(
          session:
            context.read<SshSessionCubit>().getSession(sessionId),
        )),
    ),
  },
);

```

```

child: BlocBuilder<TerminalBloc, TerminalState>(
  builder: (context, state) {
    return Scaffold(
      backgroundColor: Colors.black,
      body: SafeArea(
        child: Column(
          children: [
            // Top bar (collapsible on mobile)
            const TerminalTopBar(),
            // Terminal body
            Expanded(
              child: Stack(
                children: [
                  const OptimizedTerminalWrapper(),
                  // Search overlay
                  const TerminalSearchOverlay(),
                  // AI suggestion overlay
                  const AiSuggestionOverlay(),
                  // Connection state overlay
                  const ConnectionStateOverlay(),
                ],
              ),
            ),
            // Mobile keyboard toolbar
            if (context.isMobile)
              const MobileKeyboardToolbar(),
          ],
        ),
      ),
    );
  },
);
}

```

Top Bar Components: - Session title (label@hostname) - Connection status badge - Tab strip (for multiple sessions in single window) - Action menu: Search, SFTP, Port Forwarding, Share, Recording, Close

Mobile Keyboard Toolbar: - Ctrl, Alt, Tab, Esc keys - Arrow keys (compact 4-way) - Page Up/Down - Function keys (F1-F12, collapsed behind “Fn” button) - Drag handle for height adjustment

State Variations: - connecting — pulsing connection animation - connected — live terminal, full interactive - reconnecting — amber banner “Reconnecting...” with attempt counter - disconnected — gray overlay, “Reconnect” button prominent - error — red banner, error message, “Try Again” - search_active — search bar at top, matches highlighted in terminal - recording — red recording indicator in top bar

6.7 Screen: Split View Terminal

Layout: Configurable split layouts supporting 2x1, 2x2, and 3-way arrangements.

Components: - WorkspaceSplitter — Draggable divider bar between panes (6px wide, snaps to 1/4, 1/3, 1/2, 2/3, 3/4) - WorkspacePaneHeader — Per-pane title bar with session info and close button - WorkspaceLayoutPicker — Bottom sheet with layout presets (6 options as icon grid) - WorkspaceTabBar — Above the terminal, shows sessions in this pane

```
// features/workspace/presentation/widgets/workspace_layout.dart
class WorkspaceLayout extends StatelessWidget {
  final Workspace workspace;
  const WorkspaceLayout({required this.workspace, super.key});

  @override
  Widget build(BuildContext context) {
    return switch (workspace.layout) {
      WorkspaceLayout.single => _SinglePane(workspace.panes[0]),
      WorkspaceLayout.splith => _SplitHorizontal(
        left: workspace.panes[0],
        right: workspace.panes[1],
      ),
      WorkspaceLayout.splitv => _SplitVertical(
        top: workspace.panes[0],
        bottom: workspace.panes[1],
      ),
      WorkspaceLayout.grid2x2 => _Grid2x2(panes: workspace.panes),
      WorkspaceLayout.threeLeft => _ThreeLeft(panes:
        workspace.panes),
      WorkspaceLayout.threeRight => _ThreeRight(panes:
        workspace.panes),
      WorkspaceLayout.custom => _CustomLayout(workspace:
        workspace),
    };
  }
}
```



```

        child: WorkspacePaneWidget(pane: widget.right),
      ),
    ],
  );
},
);
}
}

```

6.8 Screen: SFTP Browser

Layout: Dual-pane file manager. Left pane = remote, right pane = local.

Components: - SftpAddressBar — Current path with breadcrumb navigation - SftpEntryList — Virtual list of SftpEntryRow items - SftpEntryRow — Icon, name, size, permissions, modified date, selection checkbox - SftpTransferQueuePanel — Collapsible bottom drawer with transfer items - SftpContextMenu — Right-click/long-press: Open, Download, Rename, Permissions, Delete, New Folder - SftpPermissionsDialog — Octal + checkboxes for owner/group/other rwx - SftpProgressCard — Per-transfer card with progress bar, speed, ETA, pause/cancel

Drag and Drop: - Desktop: Drag files from local pane to remote (or vice versa) initiates transfer - Web: Drag files from OS file manager into SFTP remote pane triggers upload - iPad: Long press + drag for local-to-remote transfers

State Variations: - loading — spinner in pane header - loaded — file list - empty_directory — “This folder is empty” illustration - error — error card with retry - uploading / downloading — progress indicator in transfer queue

6.9 Screen: Port Forwarding Manager

Layout: List of rules + add/edit sheet.

Components: - PortForwardRuleCard — Rule label, type badge (Local/Remote/Dynamic), addresses, active indicator, toggle switch - PortForwardEditSheet — Bottom sheet with: type selector, label, bind address, local port, remote host, remote port, auto-start toggle - ActiveConnectionsPanel — For an active rule: connected clients count, bytes sent/received, individual connection rows

State Variations: - empty — “No forwarding rules yet” + Add CTA - rules_inactive — List of configured-but-not-started rules - rules_active — Rules with green status dot, byte counters updating - conflict — Red indicator on rule with port conflict message - error — Rule that failed to start with error tooltip

6.10 Screen: Snippet Library

Layout: Left sidebar (folders/tags) + right content (snippet cards).

Components: - SnippetFolderTree — Collapsible folder tree, drag-to-reorder - SnippetCard — Title, first 2 lines of content (monospace), tags, “Copy” and “Execute” buttons - SnippetEditor — Full-screen code editor (uses code_editor package), syntax highlighting, variable placeholders (`{variable}`) - SnippetSearchBar — Live search across title + content - SnippetExecuteDialog — Preview with variable substitution prompts before execution - SnippetTagFilter — Tag chips sidebar for filtering

6.11 Screen: Keychain Manager

Layout: List view with key cards.

Components: - SshKeyCard — Key label, type badge (Ed25519/RSA/ECDSA), fingerprint (truncated), “Copy Public Key”, “Export”, “Delete” - KeyGeneratorSheet — Key type dropdown, bits (for RSA: 2048/4096), comment, passphrase, hardware key option - KeyImportSheet — PEM paste area or file picker, passphrase input, label - KeyDetailSheet — Full public key, fingerprint (MD5 + SHA256), last used timestamp

6.12 Screen: Settings - Appearance

Components: - ThemeSelector — Grid of terminal theme previews (mini 80x24 terminal thumbnails) - FontFamilySelector — Dropdown with live preview using “Hello, World!” in each font - FontSizeSlider — 8–48px range with live terminal preview - LineHeightSlider — 1.0–2.0 range - CursorStyleSelector — Block / Underline / Bar × Blinking / Static - BackgroundOpacitySlider — For platforms supporting transparency (macOS, Windows 11) - InterfaceThemeToggle — App chrome: Light / Dark / System

6.13 Screen: Organization Management

Layout: Multi-tab: Members, Vault, Audit, Settings.

Members Tab: - MemberTable — Avatar, name, email, role, last active, Actions (Change Role, Remove) - InvitePanel — Email input, role dropdown, “Send Invitation” button - PendingInviteList — Pending invitations with expiry time, resend/revoke actions

Vault Tab: - Shared vault items visible to organization - Vault sharing permissions per member/group

6.14 Screen: Collaboration

Layout: Terminal view augmented with collaboration panel.

Components: - CollaborationPanel — Slide-out right drawer with participant list - ParticipantRow — Avatar, name, cursor color indicator, role badge, kick button (owner only) - ShareCodeBanner — Top bar: share code in monospace font, copy button, QR code button - CursorOverlay — Each participant’s cursor rendered as a colored caret with username label - PermissionSwitch — Toggle read-only/interactive for each participant - InviteViaLinkSheet — URL with expiry time, one-time use option

6.15 Screen: AI Command Palette

Trigger: Ctrl+Space or Cmd+Space in terminal.

Layout: Floating modal (centered, 600px wide), full keyboard navigation.

Components: - AiCommandInput — Single text input with cursor - SuggestionList — Up to 8 suggestions - SuggestionItem — Command in monospace (bold), explanation below (small regular), confidence badge, source icon (AI/history/snippet) - CommandExplanationPanel — Expands when navigating to show full explanation, flags, examples - ContextIndicator — Current shell, CWD, recent commands feed visible below input

Interaction: - Type → debounced fetch (300ms) → show suggestions - ↑↓ to navigate suggestions - Enter → execute selected command - Tab → insert into terminal buffer without executing - Esc → dismiss

6.16 Screen: Conflict Resolution

Purpose: Give the user an explicit path to resolve a sync conflict that `ConflictStrategy.manualMerge` (§4.3) or a `crdtVectorClock` concurrent result flagged instead of silently picking a side. This is the screen the bare // Emit conflict event for user resolution via UI comment in the conflict resolver (§4.3) previously had no counterpart for.

Trigger: `ConflictResolutionBloc` receives one or more `ConflictDetected<T>` entries from `ResolvedChanges.manualMerges` (§4.3) after a sync pass completes; a non-blocking banner (“N sync conflicts need your attention”) appears on the Host List (§6.5) / Snippet Library (§6.10) depending on entity type, and opens this screen on tap. Conflicts are never auto-resolved by a timeout — they persist in a local `pending_conflicts` table until the user acts.

Layout: Full-screen (mobile) / centered modal, max-width 720px (desktop), side-by-side diff.

Components: - `ConflictQueueHeader` — “Conflict 2 of 5”, Skip / Resolve All Remaining (bulk “keep local” or “keep remote” for low-stakes entities only — never offered for vault items) - `ConflictDiffView` — two columns, **Your version** (left) vs. **Their version** (right); for non-secret entities (hosts, snippets, port-forwarding rules) renders a field-level diff highlighting changed values; for vault items (`crdtVectorClock` concurrent case) renders only metadata (label, last-modified device, timestamp) — the encrypted payload is never decrypted for comparison inside this screen, consistent with CD-10 zero-knowledge posture; the user must open each version individually (via the vault unlock flow, §5.1) to inspect content before choosing. - `ConflictResolutionActions` — **Keep mine, Keep theirs, Keep both** (available only when the entity type supports duplication, e.g. snippets — renamed with a (2) suffix), and, for field-level-diffable entities, **Merge fields** (per-field radio choice, then a synthesized merged record is pushed as a new local edit) - `ConflictMetaRow` — device/session that made each edit (from `sync_etag` / vector-clock device id), timestamp, entity type badge

State Variations: - loading — fetching the queue - active — diff shown, actions enabled - resolving — spinner on the chosen action while the resolution is pushed - empty — “No conflicts remaining” confirmation, auto-dismiss after 1.5s - error — resolution push failed (network); the conflict remains queued, retry offered

Interaction: - Choosing an action calls

`ConflictResolutionBloc.add(ConflictResolved(id, resolution))`, which removes the entry from `pending_conflicts`, applies the choice via `SyncManager`, and advances to the next queued conflict (or empty state). - Keep both for vault items is intentionally unavailable — vault conflicts always resolve to either “keep mine” / “keep theirs” / defer (leave queued); silently duplicating a vault item every time two devices race would leak information about edit frequency and is out of scope for this increment.

7. Performance Targets

7.1 Startup Performance

Target: < 1.5s cold start on mid-range Android (Snapdragon 700 series, 4GB RAM).

Optimization Strategies:

```
// main.dart – Deferred loading of heavy features
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

// Use deferred imports for non-critical features
import 'features/collaboration/collaboration.dart' deferred as
  collaboration;
import 'features/ai_autocomplete/ai_autocomplete.dart' deferred
  as ai;
import 'features/audit/audit.dart' deferred as audit;

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  // Critical path – synchronous
  await configureDependencies(Env.prod);

  // Non-critical – deferred
  collaboration.loadLibrary();
  ai.loadLibrary();
  audit.loadLibrary();
```

```

    runApp(const HelixTerminatorApp());
}

```

Flutter startup optimizations: - AOT compilation (release builds only)
 - Tree-shaking all unused widgets and icons - defer non-critical feature libraries - Warm cache of most-recent host list on app resume - Persist splash screen until first meaningful frame (FlutterNativeSplash) - Use const constructors universally for widget subtrees that don't change

Startup Measurement:

```

// core/telemetry/startup_tracer.dart
class StartupTracer {
  static final Stopwatch _sw = Stopwatch();
  static final Map<String, int> _milestones = {};

  static void start() => _sw.start();

  static void milestone(String name) {
    _milestones[name] = _sw.elapsedMilliseconds;
  }

  static void report() {
    // Report to analytics: main(), di_init, first_frame,
    // host_list_render
    debugPrint('Startup milestones: $_milestones');
  }
}

```

7.2 Terminal Rendering Performance

Target: 60fps (< 16.67ms per frame), smooth scrolling at maximum scrollbar (100k lines).

Frame Budget Allocation (16ms): - SSH data parsing → xterm.dart write: 2ms - Dirty region marking: 1ms - Layout computation (only changed cells): 4ms - Rasterization (GPU): 6ms - Composite + present: 3ms

Optimization Techniques: - RepaintBoundary wrapping the terminal widget — repaints only the terminal, not parent widgets - Throttled repaints: coalesce SSH data bursts into a single frame via SchedulerBinding.scheduleFrame() - Fixed-cell grid rendering — monospace font means all cells have identical width/height, enabling

direct pixel addressing - GPU layer caching for static lines via Flutter
saveLayer - Skipping layout for lines that didn't change (dirty bit per row)

```
// features/terminal/presentation/widgets/performance_monitor.dart
class TerminalFrameMonitor extends StatefulWidget {
  final Widget child;
  const TerminalFrameMonitor({required this.child, super.key});

  @override
  State<TerminalFrameMonitor> createState() =>
    _TerminalFrameMonitorState();
}

class _TerminalFrameMonitorState extends
  State<TerminalFrameMonitor> {
  final List<double> _frameTimes = [];
  late final FrameCallback _callback;

  @override
  void initState() {
    super.initState();
    _callback = (timestamp) {
      if (!mounted) return;
      final now = DateTime.now().microsecondsSinceEpoch;
      _frameTimes.add(now / 1000.0); // Convert to ms

      if (_frameTimes.length > 60) {
        _frameTimes.removeAt(0);
      }

      SchedulerBinding.instance.addPostFrameCallback(_callback);
    };
    SchedulerBinding.instance.addPostFrameCallback(_callback);
  }

  double get averageFrameTime {
    if (_frameTimes.length < 2) return 0;
    double total = 0;
    for (int i = 1; i < _frameTimes.length; i++) {
      total += _frameTimes[i] - _frameTimes[i - 1];
    }
    return total / (_frameTimes.length - 1);
  }
}
```

@override

```
Widget build(BuildContext context) => widget.child;  
}
```

7.3 Memory Targets

Platform	Target	Maximum
Android (mid-range)	< 120MB	150MB
iOS	< 130MB	150MB
macOS	< 200MB	300MB
Windows	< 200MB	300MB
Linux	< 180MB	300MB
Web (WASM)	< 250MB	400MB

Memory Management: - Terminal scrollback: 10,000 lines default (configurable 1,000–500,000) - Session log buffering: 1MB circular buffer per session, spill to file after - SFTP transfer buffer: 8MB chunk size, single buffer per active transfer - Image cache (terminal inline images): LRU cache, max 50MB - Host list: SQLite cursor-based pagination, never load all 10,000 hosts into memory at once

```
// core/memory/memory_pressure_handler.dart  
class MemoryPressureHandler {  
  static void initialize() {  
    // Listen for OS memory pressure signals  
    SystemChannels.lifecycle.setMessageHandler((message) async {  
      if (message == AppLifecycleState.paused.toString()) {  
        _onAppPaused();  
      }  
      return null;  
    });  
  }  
  
  static void _onAppPaused() {  
    // Reduce scrollback buffer  
    getIt<TerminalScrollbackManager>().trimToMinimum();  
    // Clear image cache  
    imageCache.clear();  
    imageCache.clearLiveImages();  
    // Release non-essential SSH connections  
    getIt<SshConnectionPool>().releaseIdleConnections();  
  }  
}
```

```
}  
}
```

7.4 Network Performance

SSH connection establishment: < 500ms on LAN, < 2s on WAN.

Reconnection after network change: < 2s (network change detection
→ TCP reconnect → SSH handshake).

Optimization: - SSH keep-alive (30s interval) to prevent NAT timeout
disconnections - TCP connection pool reuse (ControlMaster equivalent)
- DNS prefetching for all hosts in list on app foreground - SFTP read-
ahead buffering: 4 concurrent read requests per file transfer

7.5 Host List Rendering

Target: Initial render of 10,000 hosts within the frame budget.

```
// features/hosts/presentation/widgets/host_list_view.dart  
class VirtualizedHostList extends StatelessWidget {  
  final List<HostEntity> hosts;  
  const VirtualizedHostList({required this.hosts, super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return CustomScrollView(  
      slivers: [  
        SliverList(  
          delegate: SliverChildBuilderDelegate(  
            (context, index) {  
              // Only builds visible + a small over-scan buffer  
              return HostListItem(  
                key: ValueKey(hosts[index].id),  
                host: hosts[index],  
              );  
            },  
            childCount: hosts.length,  
            findChildIndexCallback: (key) {  
              final id = (key as ValueKey<String>).value;  
              final idx = hosts.indexWhere((h) => h.id == id);  
              return idx == -1 ? null : idx;  
            },  
          ),  
        ],  
      ),  
    );  
  }  
}
```

```

    },
    1,
);
}
}

```

7.6 Per-Platform Startup & Performance Targets

§7.1’s <1.5s cold-start target is stated only for mid-range Android; every one of the six release platforms (§8) has a materially different startup profile (native AOT binary vs. CanvasKit/WASM parse-and-JIT-warm on Web) and therefore needs its own target rather than inheriting the Android number by omission.

Platform	Cold start (target)	Cold start (maximum)	Warm start / resume	Notes
Android (mid-range, Snapdragon 700-series, 4GB RAM)	< 1.5s	2.5s	< 400ms	§7.1 baseline; AOT release build
iOS (A14 or newer)	< 1.2s	2.0s	< 350ms	AOT release build; faster single-core boost than mid-range Android
macOS (Apple Silicon)	< 0.8s	1.5s	< 250ms	Native ARM64 desktop build, no WASM overhead
macOS (Intel, min-spec)	< 1.2s	2.0s	< 350ms	Rosetta-free native x86_64 build
Windows (min-spec: 4-core, 8GB RAM)	< 1.3s	2.2s	< 350ms	Native x86_64 desktop build
Linux (min-spec: 4-core, 8GB RAM)	< 1.3s	2.2s	< 350ms	GTK embedder; parity with Windows target
Web (WASM, TTI)	< 3.5s TTI	5.0s TTI	< 1.0s (cached SW)	Dominated by CanvasKit WASM parse/instantiate; matches the ~3.5s TTI

Platform	Cold start (target)	Cold start (maximum)	Warm start / resume	Notes
first visit, uncached)				figure cross-referenced from 09_performance_analysis.md §10.6 — this is the one platform where the client's own target is bound by that doc's browser-engine measurement rather than an independent client-side figure
Web (WASM, repeat visit, service-worker cached)	< 1.2s TTI	2.0s TTI	< 1.0s	Service worker (§8.6) serves main.dart.js/CanvasKit from cache, skipping network fetch

Measurement: every non-Web platform reuses `StartupTracer` (§7.1) with the same milestone set (`main()`, `di_init`, `first_frame`, `host_list_render`); Web additionally records `canvaskit_wasm_instantiate` as a distinct milestone so WASM parse cost is never conflated with Dart-side initialization when triaging a regression.

7.7 Cross-Platform Feature Parity Matrix

Not every capability is available on every platform — sandboxing, browser API surface, and hardware presence all constrain which features apply where. This matrix is the authoritative per-platform capability reference (Full / Partial / Unavailable), the same three-state vocabulary already used by the Offline Capability table (§4.1):

Feature	Android	iOS/iPadOS	macOS	Windows	Linux
SSH / SFTP core	Full	Full	Full	Full	Full
SSH agent forwarding	Full	Partial (no system ssh-agent inside app)	Full	Full	Full

Feature	Android	iOS/iPadOS	macOS	Windows	Linux
		sandbox; in-app agent only)			
FIDO2 hardware security key (\$5.7)	Full (USB-C/NFC)	Full (NFC/Lightning security keys)	Full (USB)	Full (USB)	Full
Biometric unlock (\$5.3)	Full (BiometricPrompt)	Full (Face ID/Touch ID)	Full (Touch ID, supported Macs)	Full (Windows Hello, supported hardware)	Unimplemented (no standard biometric API)
Background SSH/SFTP execution (\$8.7)	Full (foreground service)	Partial (bounded BGTaskScheduler window + VoIP push wake, not indefinite)	Full (no OS-imposed background suspension for a running desktop app)	Full	Full
Real-time collaboration (\$1.16)	Full	Full	Full	Full	Full
Native accessibility (VoiceOver/TalkBack/NVDA/Orca, \$10.2)	Full	Full	Full	Full (NVDA/JAWS via Windows UIA)	Partial (Orca covers native, but Windows makes screen reader customization more tedious, especially for complex UIs)
Material You dynamic	Full		Unavailable	Unavailable	Unimplemented

Feature	Android	iOS/iPadOS	macOS	Windows	Linux
theming (§8.5)		Unavailable (not an OS concept on iOS)			
Handoff / iPad Stage Manager multi-window (§8.4)	Unavailable	Full	N/A (native multi-window is default)	N/A	N/A
Auto-update (§11)	App-store managed (Play Store) or sideload channel	App-store managed (App Store) — in-app update channel logic is store-gated	Full (in-app updater + notarized)	Full (in-app updater + code-signed)	Full (in-app updater + code-signed)
Screen capture protection (§5.4)	Full (FLAG_SECURE)	Full (screenshot/recording detection)	Partial (no OS-level screenshot block; recording-detection only)	Partial (no OS-level screenshot block; recording-detection only)	Partial (no OS-level screenshot block; recording-detection only)

7.8 Real-Time Collaboration Latency Budget

Client-observable targets for the collaboration transport defined in §1.16, filling the gap noted there (no matching entry yet exists in 09_performance_analysis.md):

Interaction	Target (p50)	Target (p95)	Measurement point
Presence (cursor move → remote render)	60ms	100ms	presence frame send → CollaborationCursorMoved state applied on a remote client
Keystroke fan-out (controller keypress →)	80ms	150ms	controller keystroke frame send → output frame rendered on farthest-connected viewer

Interaction	Target (p50)	Target (p95)	Measurement point
all viewers see output)			
Control-request round trip (request → owner sees toast)	100ms	200ms	CollaborationControlRequested send → toast rendered on owner's client
Control-grant round trip (grant tap → requester can type)	100ms	200ms	Grant tap → control_grant applied, CollaborationActive.controllingUserId updated
Join (share code entry → first frame of shared terminal visible)	600ms	1.2s	CollaborationJoinRequested → first output frame rendered
Heartbeat-timeout eviction (participant silent → roster update)	—	30s (fixed, §1.16)	last heartbeat received → roster frame removing the entry

These are client round-trip budgets (network + server fan-out + render), not pure client-side render costs — they compose with, but are not identical to, the terminal-render frame budget in §7.2 (which governs local-session paint cost only, before any network hop).

8. Platform-Specific Features

8.1 macOS

Menu Bar App Mode:

```
// platform/macos/menu_bar_service.dart
class MacOSMenuBarService {
  static const _channel =
    MethodChannel('io.helixterminator.client/menu_bar');

  // Show app in menu bar with connection status
  static Future<void> showMenuBarIcon() async {
    await _channel.invokeMethod('showMenuBarIcon');
  }

  // Update menu bar icon badge with active session count
  static Future<void> updateBadge(int sessionCount) async {
    await _channel.invokeMethod('updateBadge', {'count':
      sessionCount});
  }

  // Menu items: Quick Connect, Recent Hosts (last 5), Separator,
  // Open Main Window, Quit
  static Future<void> setMenuItems(List<MacOsMenuItem> items)
    async {
    await _channel.invokeMethod('setMenuItems', {
      'items': items.map((i) => i.toMap()).toList(),
    });
  }
}
```

macOS Application Menu Structure:

```
HelixTerminator
├─ About HelixTerminator
├─ Preferences... (⌘,)
└─ Quit (⌘Q)
```

```
File
├─ New Connection... (⌘N)
├─ New Window (⌘⇧N)
├─ Import Hosts...
├─ Export Hosts...
└─ Close (⌘W)
```

Edit

- |— Copy (⌘C)
- |— Paste (⌘V)
- |— Select All (⌘A)
- |— Find... (⌘F)
- |— Clear Scrollback (⌘K)

View

- |— Toggle Sidebar (⌘⇧S)
- |— Enter Full Screen (^⌘F)
- |— Zoom In (⌘+)
- |— Zoom Out (⌘-)

Session

- |— New Terminal Tab (⌘T)
- |— Split Horizontally (⌘D)
- |— Split Vertically (⌘⇧D)
- |— Disconnect (⌘⇧W)

Window

- |— Minimize (⌘M)
- |— Zoom
- |— [open windows list]

Handoff (Continuity):

```
// platform/macos/handoff_service.dart
class HandoffService {
  static const _channel =
    MethodChannel('io.helixterminator.client/handoff');

  // Register current session for Handoff (requires signed-in
  // iCloud)
  static Future<void> registerSession(SshSession session) async {
    await _channel.invokeMethod('registerUserActivity', {
      'activityType': 'io.helixterminator.client.ssh-session',
      'userInfo': {
        'hostId': session.hostId,
        'sessionId': session.id,
        'hostname': session.hostname,
        'username': session.username,
      },
    });
  }
}
```

```

// Handle incoming Handoff from another device
static void handleContinuation(Map<String, dynamic> userInfo) {
  final hostId = userInfo['hostId'] as String;
  getIt<SshSessionCubit>().reconnectToHost(hostId);
}
}

```

Multiple Windows (macOS):

```

// platform/macos/window_manager.dart
class MacOSWindowManager {
  static Future<void> openNewWindow({String? hostId}) async {
    await windowManager.ensureInitialized();
    // Create a new Flutter engine + window
    final window = await windowManager.createWindow();
    if (hostId != null) {
      window.setTitle('HelixTerminator - $hostId');
    }
  }
}

```

Touch Bar Support (MacBook Pro models):

```

// macOS/Runner/TouchBarProvider.swift
class TouchBarProvider: NSObject, NSTouchBarDelegate {
  func makeTouchBar() -> NSTouchBar? {
    let bar = NSTouchBar()
    bar.defaultItemIdentifiers = [
      .ctrlKey, .altKey, .tabKey,
      .flexibleSpace,
      .arrowKeys,
    ]
    bar.delegate = self
    return bar
  }
}

```

8.2 Windows

System Tray:

```

// platform/windows/system_tray_service.dart
import 'package:tray_manager/tray_manager.dart';

class WindowsSystemTrayService {

```

```

Future<void> initialize() async {
  await trayManager.setIcon('assets/icons/tray_icon.ico');
  await trayManager.setContextMenu(Menu(
    items: [
      MenuItem(label: 'Open HelixTerminator', onClick:
        _openMainWindow),
      MenuItem.separator(),
      MenuItem(label: 'New Connection...', onClick:
        _newConnection),
      MenuItem.submenu(
        label: 'Recent Hosts',
        submenu: Menu(items: await _getRecentHostMenuItems()),
      ),
      MenuItem.separator(),
      MenuItem(label: 'Quit', onClick: _quit),
    ],
  ));
}

void _openMainWindow(_) => windowManager.show();
void _quit(_) => windowManager.destroy();
}

```

Windows Hello:

```

// platform/windows/windows_hello_service.dart
class WindowsHelloService {
  static const _channel =
    MethodChannel('io.helixterminator.client/windows_hello');

  static Future<bool> isAvailable() async {
    final result = await
      _channel.invokeMethod<bool>('isAvailable');
    return result ?? false;
  }

  static Future<bool> authenticate(String message) async {
    final result = await
      _channel.invokeMethod<bool>('authenticate', {
        'message': message,
      });
    return result ?? false;
  }
}

```

Jump List:

```
// platform/windows/jump_list_service.dart
class WindowsJumpListService {
  static const _channel =
    MethodChannel('io.helixterminator.client/jump_list');

  static Future<void> updateJumpList(List<HostEntity>
    recentHosts) async {
    await _channel.invokeMethod('updateJumpList', {
      'tasks': recentHosts.take(5).map((h) => {
        'title': h.label,
        'description': '${h.username}@${h.hostname}:${h.port}',
        'arguments': '--host ${h.id}',
        'iconPath': 'assets/icons/host_icon.ico',
      }).toList(),
    });
  }
}
```

8.3 Linux

AppIndicator (System Tray):

```
// platform/linux/app_indicator_service.dart
class LinuxAppIndicatorService {
  static const _channel =
    MethodChannel('io.helixterminator.client/app_indicator');

  Future<void> initialize() async {
    await _channel.invokeMethod('initialize', {
      'id': 'io.helixterminator.client',
      'iconName': 'helix-terminator',
      'category': 'APPLICATION_STATUS',
    });
  }
}
```

D-Bus Notifications:

```
// platform/linux/dbus_notification_service.dart
import 'package:dbus/dbus.dart';

class DBusNotificationService {
  final DBusClient _client = DBusClient.session();

  Future<void> sendNotification({
    required String summary,
```

```

    String? body,
    String? appIcon,
    int timeout = 5000,
  }) async {
    await _client.callMethod(
      destination: 'org.freedesktop.Notifications',
      path: DBusObjectPath('/org/freedesktop/Notifications'),
      interface: 'org.freedesktop.Notifications',
      name: 'Notify',
      values: [
        DBusString('HelixTerminator'), // app_name
        DBusUInt32(0), // replaces_id
        DBusString(appIcon ?? 'terminal'), // app_icon
        DBusString(summary),
        DBusString(body ?? ''),
        DBusArray.string([]), // actions
        DBusDict.stringVariant({}), // hints
        DBusInt32(timeout),
      ],
    );
  }
}

```

Wayland + X11 Support:

```

// platform/linux/display_server_service.dart
class LinuxDisplayServerService {
  static bool get isWayland {
    final display = Platform.environment['WAYLAND_DISPLAY'];
    return display != null && display.isNotEmpty;
  }

  static bool get isX11 {
    final display = Platform.environment['DISPLAY'];
    return display != null && display.isNotEmpty;
  }

  // X11 Forwarding: requires DISPLAY env var to be forwarded
  // through SSH
  static String? get x11Display =>
    Platform.environment['DISPLAY'];
}

```

8.4 iOS / iPadOS

External Keyboard Shortcuts:

```
// platform/ios/keyboard_shortcuts.dart
class IosKeyboardShortcuts {
  static List<UIKeyCommand> get commands => [
    UIKeyCommand(
      input: 't',
      modifierFlags: [.command],
      action: 'newTerminalTab',
      discoverabilityTitle: 'New Tab',
    ),
    UIKeyCommand(
      input: 'w',
      modifierFlags: [.command],
      action: 'closeTab',
      discoverabilityTitle: 'Close Tab',
    ),
    UIKeyCommand(
      input: 'f',
      modifierFlags: [.command],
      action: 'findInTerminal',
      discoverabilityTitle: 'Find',
    ),
    UIKeyCommand(
      input: 'k',
      modifierFlags: [.command],
      action: 'clearScrollbar',
      discoverabilityTitle: 'Clear Scrollback',
    ),
  ];
}
```

Haptic Feedback:

```
// platform/ios/haptics_service.dart
class HapticsService {
  static Future<void> onConnect() async {
    await HapticFeedback.mediumImpact();
  }

  static Future<void> onDisconnect() async {
    await HapticFeedback.lightImpact();
  }
}
```

```

static Future<void> onError() async {
  await HapticFeedback.heavyImpact();
}

static Future<void> onCopy() async {
  await HapticFeedback.selectionClick();
}
}

```

iPad Stage Manager (Multiple Windows):

```

// platform/ios/multitasking_service.dart
class IPadMultitaskingService {
  static const _channel =
    MethodChannel('io.helixterminator.client/multitasking');

  static Future<void> openSessionInNewWindow(String sessionId)
    async {
    if (!await _isMultiWindowSupported()) return;
    await _channel.invokeMethod('openNewScene', {
      'sessionId': sessionId,
      'sceneTitle': 'HelixTerminator',
    });
  }

  static Future<bool> _isMultiWindowSupported() async {
    final result = await
      _channel.invokeMethod<bool>('isMultiWindowSupported');
    return result ?? false;
  }
}

```

8.5 Android

Material You Dynamic Theming:

```

// core/theme/dynamic_color_service.dart
import 'package:dynamic_color/dynamic_color.dart';

class DynamicColorService {
  static Future<ColorScheme?> getDynamicColorScheme(Brightness
    brightness) async {
    final corePalette = await DynamicColorPlugin.getCorePalette();
    if (corePalette == null) return null;
  }
}

```



```

    return brightness == Brightness.light
      ? corePalette.toColorScheme(brightness: Brightness.light)
      : corePalette.toColorScheme(brightness: Brightness.dark);
  }
}

```

// Usage in app.dart:

```

class HelixTerminatorApp extends StatelessWidget {
  const HelixTerminatorApp({super.key});

  @override
  Widget build(BuildContext context) {
    return DynamicColorBuilder(
      builder: (lightDynamic, darkDynamic) {
        return MaterialApp.router(
          theme: AppTheme.lightTheme(colorScheme: lightDynamic),
          darkTheme: AppTheme.darkTheme(colorScheme: darkDynamic),
          themeMode: ThemeMode.system,
          routerConfig: getIt<AppRouter>().router,
        );
      },
    );
  }
}

```

Notification Channels:

```

// platform/android/notification_channels.dart
class AndroidNotificationChannels {
  static const sessions = AndroidNotificationChannel(
    'ssh_sessions',
    'SSH Sessions',
    description: 'Notifications for SSH connection events',
    importance: Importance.high,
  );

  static const transfers = AndroidNotificationChannel(
    'sftp_transfers',
    'File Transfers',
    description: 'SFTP transfer progress and completion',
    importance: Importance.defaultImportance,
  );

  static const security = AndroidNotificationChannel(
    'security_alerts',

```

```

        'Security Alerts',
        description: 'Security warnings and alerts',
        importance: Importance.max,
        sound: RawResourceAndroidNotificationSound('alert'),
    );

    static Future<void> registerAll() async {
        final plugin = FlutterLocalNotificationsPlugin();
        for (final channel in [sessions, transfers, security]) {
            await plugin
                .resolvePlatformSpecificImplementation<
                    AndroidFlutterLocalNotificationsPlugin>()
                ?.createNotificationChannel(channel);
        }
    }
}

```

Quick Settings Tile:

```

// platform/android/quick_tile_service.dart
// Implemented in Kotlin via platform channel
// Allows toggling a port forwarding rule from the notification
    shade

```

8.6 Web (PWA)

Service Worker and Offline Cache:

```

// web/service_worker.js (generated via flutter_pwa_service)
const CACHE_NAME = 'helix-terminator-v1';
const STATIC_ASSETS = [
    '/',
    '/main.dart.js',
    '/flutter.js',
    '/manifest.json',
    // Font files
    '/fonts/JetBrainsMono-Regular.ttf',
    '/fonts/Inter-Regular.ttf',
];

self.addEventListener('install', (event) => {
    event.waitUntil(
        caches.open(CACHE_NAME).then((cache) =>
            cache.addAll(STATIC_ASSETS))
    );
});

```

```
);
});
```

Web Crypto API for Key Operations:

```
// platform/web/web_crypto_service.dart
@JS('crypto.subtle')
external dynamic get _subtle;

class WebCryptoService {
  Future<CryptoKeyPair> generateEd25519KeyPair() async {
    final result = await promiseToFuture<dynamic>(
      _subtle.generateKey(
        js_util.jsify({'name': 'Ed25519'}),
        true, // extractable
        js_util.jsify(['sign', 'verify']),
      ),
    );
    return CryptoKeyPair(
      privateKey: result.privateKey,
      publicKey: result.publicKey,
    );
  }
}
```

File System Access API for SFTP:

```
// platform/web/file_system_access.dart
@JS('window.showDirectoryPicker')
external dynamic _showDirectoryPicker([dynamic options]);

class WebFileSystemAccess {
  Future<WebDirectory?> pickDirectory() async {
    try {
      final handle = await
        promiseToFuture<dynamic>(_showDirectoryPicker(
          js_util.jsify({'mode': 'readwrite'}),
        ));
      return WebDirectory(handle);
    } catch (_) {
      return null;
    }
  }
}
```

8.7 Mobile Background Execution for SSH/SFTP Sessions

Neither §8.4 (iOS) nor §8.5 (Android) previously specified what happens to an active SSH/SFTP session when the app is backgrounded. Both mobile OSes suspend or kill a backgrounded socket within seconds unless the app declares the platform's specific background-execution mechanism — this section is that mechanism, for both platforms, plus the shared client-side keepalive/reconnect contract they hand back to.

iOS / iPadOS — BGTaskScheduler + VoIP Push

iOS gives a backgrounded app only ~30s of guaranteed execution before suspension unless the app holds a recognized background mode. HelixTerminator declares two:

```
<!-- ios/Runner/Info.plist -->
<key>UIBackgroundModes</key>
<array>
  <string>voip</string>
  <string>processing</string>
  <string>fetch</string>
</array>

// platform/ios/background_session_service.dart
class IosBackgroundSessionService {
  static const _channel =
    MethodChannel('io.helixterminator.client/background');

  /// Called when the app enters background with ≥1 active SSH/
  /// SFTP session.
  /// Registers a BGProcessingTask (long, deferred, best-effort –
  /// used to
  /// flush buffered session output to local storage before the OS
  /// may
  /// suspend) AND holds a PushKit VoIP registration alive, which
  /// is the
  /// only iOS mechanism that reliably wakes a suspended app on an
  /// external
  /// signal without the App Store review restrictions that apply
  /// to
  /// non-VoIP "keep alive" background execution.
  static Future<void> onEnterBackground(List<String>
    activeSessionIds) async {
    await _channel.invokeMethod('scheduleBackgroundProcessing', {
      'taskId': 'io.helixterminator.client.session-flush',
      'sessionIds': activeSessionIds,
    });
  }
};
```

```

    }

    /// The server-side SSH proxy (see doc 01) holds the session
    /// open and,
    /// on new output arriving while the client is backgrounded,
    /// sends a
    /// silent VoIP push. iOS wakes the app in the background (VoIP
    /// pushes are
    /// exempt from the standard background-refresh throttling) and
    /// the app
    /// re-attaches its WebSocket/SSH multiplex without user
    /// interaction.
    static Future<void> onVoipPushReceived(String sessionId) async {
        await getIt<SshConnectionPool>().reattach(sessionId);
        await getIt<SyncManager>().performFullSync();
    }
}

```

Honest boundary: this is a *bridging* mechanism, not indefinite background execution — iOS does not grant an app an unbounded background socket the way Android’s foreground service does. An SSH session backgrounded for an extended period (industry-observed practical ceiling: minutes, not hours, and entirely at the OS’s discretion) will still be reaped by iOS; the VoIP-push reattach path exists to make *resumption* fast and silent, not to guarantee the TCP socket itself survived. Sessions the server proxy considers stale past its own idle timeout are already covered by the reconnect logic in §3.4.

Android — Foreground Service

Android requires a foreground service (with a persistent, user-visible notification — Android cannot hide this) to keep a process alive indefinitely while backgrounded:

```

// android/app/src/main/kotlin/.../SshForegroundService.kt
class SshForegroundService : Service() {
    override fun onStartCommand(intent: Intent?, flags: Int,
        startId: Int): Int {
        val notification = NotificationCompat.Builder(this,
            "ssh_sessions") // §8.5 channel
            .setContentTitle("HelixTerminator")
            .setContentText("$activeSessionCount active SSH
                session(s)")
            .setSmallIcon(R.drawable.ic_terminal)
            .setOngoing(true)
            .build()
        startForeground(NOTIFICATION_ID, notification,
            ServiceInfo.FOREGROUND_SERVICE_TYPE_DATA_SYNC)
        return START_STICKY
    }
}

```

```

    }

    override fun onDestroy() {
        // Notify the Dart side so SshConnectionPool can mark all
        // sessions
        // as backgrounded-without-service and begin the §3.4
        // reconnect path
        // on next foreground resume rather than assuming they are
        // alive.
        methodChannel?.invokeMethod("onForegroundServiceDestroyed")
        super.onDestroy()
    }
}

// platform/android/background_session_service.dart
class AndroidBackgroundSessionService {
    static const _channel =
        MethodChannel('io.helixterminator.client/background');

    static Future<void> onEnterBackground(List<String>
        activeSessionIds) async {
        if (activeSessionIds.isEmpty) return; // no service needed
        // with zero active sessions
        await _channel.invokeMethod('startForegroundService', {
            'activeSessionCount': activeSessionIds.length,
        });
    }

    static Future<void> onEnterForeground() async {
        await _channel.invokeMethod('stopForegroundServiceIfIdle');
    }
}

```

The service starts only when ≥ 1 SSH/SFTP session is active at background time and stops itself the moment the app returns to foreground with no active sessions — it is never started speculatively, keeping the persistent-notification surface area minimal and honest about what is actually running.

Shared Session-Keepalive + Reconnect Contract

Both platforms hand control back to the same client-side logic once the app is foregrounded again (or, for Android, at any point while the foreground service is alive):

- SSH keep-alive interval (§7.4) continues uninterrupted on Android (process never suspended); on iOS it is best-effort while a BGProcessingTask window is granted and otherwise paused,

relying on the VoIP-push reattach path above to recover the session rather than a continuous keep-alive.

- On foreground resume, `ConnectivityService._onNetworkRestored()` (§4.4) unconditionally triggers `SshConnectionPool.reconnectAll()` and `SyncManager.performFullSync()` regardless of which platform mechanism kept (or failed to keep) the session alive — the reconnect path is the single source of truth for “is this session still good,” never an assumption based on which background mode was used.
 - SFTP transfers in progress at background time are checkpointed (byte offset persisted to the local SQLite `transfer_progress` table, §4.5) so a resumed transfer continues from its last acknowledged chunk rather than restarting.
-

9. Testing Strategy

9.1 Test Architecture Overview

All tests follow the standard Flutter test structure. Coverage targets: unit 95%, widget 85%, integration 70%.

```
test/
├─ unit/
│   ├─ features/
│   │   ├─ auth/
│   │   │   ├─ auth_bloc_test.dart
│   │   │   ├─ login_use_case_test.dart
│   │   │   └─ auth_repository_test.dart
│   │   └─ ssh_session/
│   │       ├─ ssh_connection_service_test.dart
│   │       ├─ ssh_state_machine_test.dart
│   │       └─ reconnect_manager_test.dart
│   │   └─ ... (all features)
│   └─ core/
│       ├─ sync_manager_test.dart
│       ├─ conflict_resolver_test.dart
│       └─ vault_crypto_service_test.dart
├─ widget/
│   └─ features/
│       └─ terminal/
│           ├─ terminal_page_test.dart
│           └─ terminal_view_test.dart
```

```

|   |   └─ ... (all screens)
|   └─ golden/
|       ├── host_list_light.png
|       ├── host_list_dark.png
|       └─ terminal_dracula.png
└─ integration_test/
    ├── flows/
    |   ├── login_to_connect_flow_test.dart
    |   ├── sftp_transfer_flow_test.dart
    |   └─ port_forward_flow_test.dart
    └─ performance/
        ├── terminal_frame_time_test.dart
        └─ host_list_scroll_test.dart
└─ mocks/
    ├── mock_ssh_connection.dart
    └─ mock_repositories.dart

```

9.2 Unit Tests: BLoC/Cubit

```

// test/unit/features/auth/auth_bloc_test.dart
import 'package:bloc_test/bloc_test.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:mocktail/mocktail.dart';

class MockLoginUseCase extends Mock implements LoginUseCase {}
class MockSubmitMfaUseCase extends Mock implements
  SubmitMfaUseCase {}
class MockBiometricAuthUseCase extends Mock implements
  BiometricAuthUseCase {}
class MockLogoutUseCase extends Mock implements LogoutUseCase {}
class MockRestoreSessionUseCase extends Mock implements
  RestoreSessionUseCase {}
class MockSsoLoginUseCase extends Mock implements SsoLoginUseCase
  {}

void main() {
  late AuthBloc authBloc;
  late MockLoginUseCase mockLogin;
  late MockSubmitMfaUseCase mockSubmitMfa;
  late MockBiometricAuthUseCase mockBiometricAuth;
  late MockLogoutUseCase mockLogout;
  late MockRestoreSessionUseCase mockRestoreSession;
  late MockSsoLoginUseCase mockSsoLogin;

  setUp(() {

```



```

mockLogin = MockLoginUseCase();
mockSubmitMfa = MockSubmitMfaUseCase();
mockBiometricAuth = MockBiometricAuthUseCase();
mockLogout = MockLogoutUseCase();
mockRestoreSession = MockRestoreSessionUseCase();
mockSsoLogin = MockSsoLoginUseCase();

authBloc = AuthBloc(
  login: mockLogin,
  submitMfa: mockSubmitMfa,
  ssoLogin: mockSsoLogin,
  biometricAuth: mockBiometricAuth,
  logout: mockLogout,
  restoreSession: mockRestoreSession,
);
});

tearDown(() => authBloc.close());

group('AuthLoginRequested', () {
  const testEmail = 'test@helixterminator.io';
  const testPassword = 'SecureP@ss1';
  final testUser = AuthUser(
    id: '123',
    email: testEmail,
    displayName: 'Test User',
    role: AuthRole.member,
    enabledMfaMethods: [],
    biometricStatus: BiometricStatus.notEnrolled,
    lastLoginAt: DateTime.now(),
  );

  blocTest<AuthBloc, AuthState>(
    'emits [AuthLoading, AuthAuthenticated] when login succeeds without MFA',
    build: () {
      when(() => mockLogin(any())).thenAnswer(
        (_) async => Right(LoginResult(
          requiresMfa: false,
          user: testUser,
        )),
      );
    },
    return authBloc;
  },

```

```

    act: (bloc) => bloc.add(AuthLoginRequested(
      email: testEmail,
      password: testPassword,
    )),
    expect: () => [
      isA<AuthLoading>(),
      isA<AuthAuthenticated>()
        .having((s) => s.user.email, 'email', testEmail),
    ],
    verify: (_) {
      verify(() => mockLogin(LoginParams(
        email: testEmail,
        password: testPassword,
      ))).called(1);
    },
  );

blocTest<AuthBloc, AuthState>(
  'emits [AuthLoading, AuthMfaRequired] when MFA is required',
  build: () {
    when(() => mockLogin(any())).thenAnswer(
      (_) async => Right(LoginResult(
        requiresMfa: true,
        mfaMethods: [MfaMethod.totp],
        challenge: const MfaChallenge(token:
          'temp_token_123'),
      )),
    );
    return authBloc;
  },
  act: (bloc) => bloc.add(AuthLoginRequested(
    email: testEmail,
    password: testPassword,
  )),
  expect: () => [
    isA<AuthLoading>(),
    isA<AuthMfaRequired>()
      .having((s) => s.availableMethods, 'methods',
        [MfaMethod.totp]),
  ],
);

blocTest<AuthBloc, AuthState>(
  'emits [AuthLoading, AuthError] on invalid credentials',
  build: () {

```

```

        when(() => mockLogin(any())).thenAnswer(
            (_) async => Left(InvalidCredentialsFailure()),
        );
        return authBloc;
    },
    act: (bloc) => bloc.add(AuthLoginRequested(
        email: testEmail,
        password: 'wrongpassword',
    )),
    expect: () => [
        isA<AuthLoading>(),
        isA<AuthError>()
            .having((s) => s.code, 'code',
                AuthErrorCode.invalidCredentials),
    ],
    );
});

group('AuthLogoutRequested', () {
    blocTest<AuthBloc, AuthState>(
        'emits [AuthUnauthenticated] on logout',
        build: () {
            when(() => mockLogout(any())).thenAnswer(
                (_) async => const Right(unit),
            );
            return authBloc;
        },
        act: (bloc) => bloc.add(AuthLogoutRequested()),
        expect: () => [isA<AuthUnauthenticated>()],
    );
});
}

```

9.3 Unit Tests: SSH Connection

```

// test/unit/features/ssh_session/ssh_state_machine_test.dart
void main() {
    late SshStateMachine stateMachine;

    setUp(() => stateMachine = SshStateMachine());

    test('initial state is idle', () {
        expect(stateMachine.state, SshConnectionState.idle);
    });
}

```

```

test('valid transitions from idle to resolving', () {
    stateMachine.transition(SshConnectionState.resolving);
    expect(stateMachine.state, SshConnectionState.resolving);
});

test('throws assertion on invalid transition', () {
    expect(
        () => stateMachine.transition(SshConnectionState.connected),
        throwsAssertionError,
    );
});

test('reconnect attempts increment on reconnecting transition',
    () {
        // Fast-forward to connected
        stateMachine
            ..transition(SshConnectionState.resolving)
            ..transition(SshConnectionState.tcpConnecting)
            ..transition(SshConnectionState.sshBannerExchange)
            ..transition(SshConnectionState.keyExchange)
            ..transition(SshConnectionState.hostKeyVerification)
            ..transition(SshConnectionState.authInProgress)
            ..transition(SshConnectionState.openingChannels)
            ..transition(SshConnectionState.connected);

        expect(stateMachine._reconnectAttempts, 0);

        stateMachine.transition(SshConnectionState.reconnecting);
        expect(stateMachine._reconnectAttempts, 1);
    });

test('reconnect attempts reset on successful reconnection', () {
    // Simulate multiple failed reconnects
    for (int i = 0; i < 3; i++) {
        stateMachine.transition(SshConnectionState.reconnecting);
    }
    expect(stateMachine._reconnectAttempts, 3);

    stateMachine.transition(SshConnectionState.resolving);
    stateMachine.transition(SshConnectionState.connected);
    expect(stateMachine._reconnectAttempts, 0);
});

```

```

test('backoff delays increase with reconnect attempts', () {
  expect(stateMachine.nextReconnectDelay, const
    Duration(seconds: 1));
  stateMachine.transition(SshConnectionState.reconnecting);
  expect(stateMachine.nextReconnectDelay, const
    Duration(seconds: 2));
  stateMachine.transition(SshConnectionState.reconnecting);
  expect(stateMachine.nextReconnectDelay, const
    Duration(seconds: 4));
});
}

```

9.4 Widget Tests

```

// test/widget/features/hosts/host_list_page_test.dart
void main() {
  group('HostListPage', () {
    late MockHostListBloc mockBloc;

    setUp(() {
      mockBloc = MockHostListBloc();
    });

    Widget buildSut() {
      return MaterialApp(
        home: BlocProvider<HostListBloc>.value(
          value: mockBloc,
          child: const HostListPage(),
        ),
      );
    }

    testWidgets('shows shimmer when loading', (tester) async {
      when(() => mockBloc.state).thenReturn(HostListLoading());
      when(() => mockBloc.stream).thenAnswer((_) =>
        Stream.value(HostListLoading()));

      await tester.pumpWidget(buildSut());

      expect(find.byType(HostListShimmer), findsOneWidget);
      expect(find.byType(HostListItem), findsNothing);
    });

    testWidgets('shows empty state when no hosts', (tester) async
      {
        when(() => mockBloc.state).thenReturn(HostListLoaded(

```

```

        groups: [],
        hosts: [],
        activeFilter: const HostFilter(),
        viewModel: HostListViewMode.list,
        sortField: HostSortField.label,
        sortDirection: SortDirection.ascending,
        searchQuery: '',
    ));
    when(() => mockBloc.stream).thenAnswer((_) => const
        Stream.empty());

    await tester.pumpWidget(buildSut());

    expect(find.text('Add your first host'), findsOneWidget);
    expect(find.byType(HostListItem), findsNothing);
});

testWidgets('shows host items when loaded', (tester) async {
    final hosts = List.generate(3, (i) => HostEntity(
        id: 'host-$i',
        label: 'Host $i',
        hostname: 'server-$i.example.com',
        port: 22,
        username: 'admin',
        protocol: HostProtocol.ssh,
        createdAt: DateTime.now(),
        updatedAt: DateTime.now(),
    ));

    when(() => mockBloc.state).thenReturn(HostListLoaded(
        groups: [],
        hosts: hosts,
        activeFilter: const HostFilter(),
        viewModel: HostListViewMode.list,
        sortField: HostSortField.label,
        sortDirection: SortDirection.ascending,
        searchQuery: '',
    ));

    when(() => mockBloc.stream).thenAnswer((_) =>
        Stream.value(HostListLoaded(
            groups: [],
            hosts: hosts,
            activeFilter: const HostFilter(),
            viewModel: HostListViewMode.list,
            sortField: HostSortField.label,

```

```

        sortDirection: SortDirection.ascending,
        searchQuery: '',
    ));

    await tester.pumpWidget(buildSut());

    expect(find.byType(HostListItem), findsNWidgets(3));
    expect(find.text('Host 0'), findsOneWidget);
  });
});
}

```

9.3 Unit Tests: Repositories

```

// test/unit/features/hosts/host_repository_test.dart
void main() {
  late HostRepositoryImpl repository;
  late MockHostLocalDataSource mockLocal;
  late MockHostRemoteDataSource mockRemote;
  late MockNetworkInfo mockNetworkInfo;

  setUp(() {
    mockLocal = MockHostLocalDataSource();
    mockRemote = MockHostRemoteDataSource();
    mockNetworkInfo = MockNetworkInfo();
    repository = HostRepositoryImpl(
      localDataSource: mockLocal,
      remoteDataSource: mockRemote,
      networkInfo: mockNetworkInfo,
    );
  });

  group('getHosts', () {
    final tHosts = [
      HostModel(id: '1', label: 'Server 1', hostname:
        'srv1.example.com', port: 22),
    ];

    test('returns remote hosts and caches when online', () async {
      when(() => mockNetworkInfo.isConnected).thenAnswer((_)
        async => true);
      when(() => mockRemote.getHosts()).thenAnswer((_) async =>
        tHosts);
      when(() => mockLocal.cacheHosts(tHosts)).thenAnswer((_)
        async => {});
    });
  });
}

```

```

    final result = await repository.getHosts();

    expect(result, Right(tHosts));
    verify(() => mockLocal.cacheHosts(tHosts)).called(1);
  });

  test('returns cached hosts when offline', () async {
    when(() => mockNetworkInfo.isConnected).thenAnswer((_)
      async => false);
    when(() => mockLocal.getHosts()).thenAnswer((_) async =>
      tHosts);

    final result = await repository.getHosts();

    expect(result, Right(tHosts));
    verifyNever(() => mockRemote.getHosts());
  });

  test('returns CacheFailure when offline and no cache', ()
    async {
    when(() => mockNetworkInfo.isConnected).thenAnswer((_)
      async => false);
    when(() =>
      mockLocal.getHosts()).thenThrow(CacheException('Empty
      cache'));

    final result = await repository.getHosts();

    expect(result, isA<Left<Failure, List<HostModel>>>());
  });
});
}

```

9.4 Widget Tests

```

// test/widget/features/hosts/host_list_page_test.dart
void main() {
  late MockHostListBloc mockBloc;

  setUp(() {
    mockBloc = MockHostListBloc();
  });

  // continued from partial file...

```



```

testWidgets('shows sort direction indicator correctly',
  (tester) async {
    when(() => mockBloc.state).thenReturn(HostListLoaded(
      groups: [],
      hosts: [],
      activeFilter: const HostFilter(),
      viewModel: HostListViewModel.list,
      sortField: HostSortField.label,
      sortDirection: SortDirection.descending, // descending order
      searchQuery: '',
    ));
    when(() => mockBloc.stream).thenAnswer((_) => const
      Stream.empty());

    await tester.pumpWidget(MaterialApp(
      home: BlocProvider<HostListBloc>.value(
        value: mockBloc,
        child: const HostListPage(),
      ),
    ));

    expect(find.byIcon(Icons.arrow_downward), findsOneWidget);
  });
}

```

9.5 Integration Tests (E2E)

Integration tests use `flutter_test` with `integration_test` package, running on real device or emulator.

```

// integration_test/flows/login_to_connect_flow_test.dart
import 'package:flutter_test/flutter_test.dart';
import 'package:integration_test/integration_test.dart';
import 'package:helix_client/main.dart' as app;

void main() {
  IntegrationTestWidgetsFlutterBinding.ensureInitialized();

  group('Login → Connect → Terminal E2E', () {
    testWidgets('completes full connection flow', (tester) async {
      app.main();
      await tester.pumpAndSettle(const Duration(seconds: 3));

      // Step 1: Login

```

```

    await tester.enterText(find.byKey(const
      Key('email_field')), 'test@example.com');
    await tester.enterText(find.byKey(const
      Key('password_field')), 'Test1234!');
    await tester.tap(find.byKey(const Key('login_button')));
    await tester.pumpAndSettle(const Duration(seconds: 5));
    expect(find.byKey(const Key('host_list_page')),
      findsOneWidget);

    // Step 2: Select host
    await tester.tap(find.text('Development Server'));
    await tester.pumpAndSettle();
    expect(find.byKey(const Key('host_detail_page')),
      findsOneWidget);

    // Step 3: Connect
    await tester.tap(find.byKey(const Key('connect_button')));
    await tester.pumpAndSettle(const Duration(seconds: 5));

    // Expect terminal is shown
    expect(find.byKey(const Key('terminal_page')),
      findsOneWidget);
    expect(find.byKey(const Key('terminal_view')),
      findsOneWidget);

    // Step 4: Type a command
    await tester.tap(find.byKey(const Key('terminal_view')));
    await tester.pumpAndSettle();
    await tester.sendKeyEvent(LogicalKeyboardKey.keyA);
    // ... verify command echo in terminal

    // Step 5: Disconnect
    await tester.tap(find.byKey(const
      Key('disconnect_button')));
    await tester.pumpAndSettle(const Duration(seconds: 2));
    expect(find.byKey(const Key('host_list_page')),
      findsOneWidget);
  });

testWidgets('SFTP browse and download flow', (tester) async {
  app.main();
  await tester.pumpAndSettle(const Duration(seconds: 3));

  // ... login ...
  await _loginWithTestCredentials(tester);

  // Open SFTP browser

```

```

    await tester.tap(find.byKey(const Key('sftp_button')));
    await tester.pumpAndSettle(const Duration(seconds: 4));
    expect(find.byKey(const Key('sftp_browser_page')),
      findsOneWidget);

    // Navigate to /home/user
    await tester.tap(find.text('/'));
    await tester.pumpAndSettle();
    await tester.tap(find.text('home'));
    await tester.pumpAndSettle();
    await tester.tap(find.text('user'));
    await tester.pumpAndSettle();

    // Download a file
    await tester.longPress(find.text('.bashrc'));
    await tester.pumpAndSettle();
    await tester.tap(find.text('Download'));
    await tester.pumpAndSettle(const Duration(seconds: 3));

    // Verify transfer appears in queue
    await tester.tap(find.byKey(const
      Key('transfer_queue_tab')));
    await tester.pumpAndSettle();
    expect(find.text('.bashrc'), findsOneWidget);
  });
});
}

Future<void> _loginWithTestCredentials(WidgetTester tester) async
{
  await tester.enterText(find.byKey(const Key('email_field')),
    'test@example.com');
  await tester.enterText(find.byKey(const Key('password_field')),
    'Test1234!');
  await tester.tap(find.byKey(const Key('login_button')));
  await tester.pumpAndSettle(const Duration(seconds: 5));
}

```

9.6 Golden Tests (Pixel-Perfect UI Regression)

```

// test/widget/golden/host_list_golden_test.dart
import 'package:golden_toolkit/golden_toolkit.dart';

void main() {
  setUpAll(() async {
    await loadAppFonts();

```

```

});

testGoldens('HostListPage light mode', (tester) async {
  await tester.pumpWidgetBuilder(
    BlocProvider<HostListBloc>(
      create: (_) => HostListBloc(
        getHostsUseCase: MockGetHostsUseCase(),
      )..add(HostListInitialized()),
      child: const HostListPage(),
    ),
    surfaceSize: const Size(375, 812),
    wrapper: materialAppWrapper(theme: AppThemes.light),
  );
  await screenMatchesGolden(tester, 'host_list_light');
});

testGoldens('HostListPage dark mode', (tester) async {
  await tester.pumpWidgetBuilder(
    BlocProvider<HostListBloc>(
      create: (_) => HostListBloc(
        getHostsUseCase: MockGetHostsUseCase(),
      )..add(HostListInitialized()),
      child: const HostListPage(),
    ),
    surfaceSize: const Size(375, 812),
    wrapper: materialAppWrapper(theme: AppThemes.dark),
  );
  await screenMatchesGolden(tester, 'host_list_dark');
});

testGoldens('Terminal Dracula theme', (tester) async {
  await tester.pumpWidgetBuilder(
    BlocProvider<TerminalBloc>(
      create: (_) => TerminalBloc(
        terminalService: MockTerminalService(),
      ),
      child: const TerminalPage(),
    ),
    surfaceSize: const Size(1280, 720),
    wrapper: materialAppWrapper(theme: AppThemes.dark),
  );
  await screenMatchesGolden(tester, 'terminal_dracula');
});
}

```

9.7 Performance Tests

```
// integration_test/performance/terminal_frame_time_test.dart
import 'package:flutter/scheduler.dart';
import 'package:flutter_test/flutter_test.dart';
import 'package:integration_test/integration_test.dart';

void main() {
  final binding =
    IntegrationTestWidgetsFlutterBinding.ensureInitialized();

  testWidgets('Terminal renders at 60fps during rapid output',
    (tester) async {
    final frameTimings = <FrameTiming>[];

    SchedulerBinding.instance.addTimingsCallback((timings) {
      frameTimings.addAll(timings);
    });

    // Pump terminal page with simulated rapid SSH output
    await tester.pumpWidget(buildTerminalWithMockData());
    await tester.pumpAndSettle();

    // Simulate 5000 lines of rapid output
    for (int i = 0; i < 5000; i++) {
      binding.reportData ??= <String, dynamic>{};
      // Simulate writing a line to terminal
      final terminalCubit = tester.state<TerminalPageState>(
        find.byType(TerminalPage),
      ).cubit;
      terminalCubit.processOutput('Line $i: ${_randomLine()}
        \r\n');
      if (i % 100 == 0) await tester.pump(const
        Duration(milliseconds: 16));
    }

    await tester.pumpAndSettle();

    // Analyze frame timings
    final slowFrames = frameTimings.where(
      (f) => f.totalSpan.inMilliseconds > 16,
    ).length;

    final jankFrames = frameTimings.where(
      (f) => f.totalSpan.inMilliseconds > 32,
    ).length;
```

```

binding.reportData!['frame_count'] = frameTimings.length;
binding.reportData!['slow_frame_count'] = slowFrames;
binding.reportData!['jank_frame_count'] = jankFrames;
binding.reportData!['slow_frame_percentage'] =
    frameTimings.isNotEmpty ? slowFrames /
        frameTimings.length * 100 : 0;

// Assert: less than 5% slow frames, zero jank
expect(slowFrames / frameTimings.length, lessThan(0.05));
expect(jankFrames, equals(0));
});

testWidgets('Host list scrolls smoothly with 10000 hosts',
    (tester) async {
    await tester.pumpWidget(buildHostListWith10000Hosts());
    await tester.pumpAndSettle();

    final stopwatch = Stopwatch()..start();
    await tester.fling(
        find.byKey(const Key('host_list_view')),
        const Offset(0, -5000),
        3000,
    );
    await tester.pumpAndSettle();
    stopwatch.stop();

    // Should scroll 10000 items within reasonable time
    expect(stopwatch.elapsedMilliseconds, lessThan(3000));
});
}

String _randomLine() {
    const chars = 'abcdefghijklmnopqrstuvwxyz0123456789 ';
    return List.generate(80, (_) =>
        chars[Random().nextInt(chars.length)]).join();
}

```

9.8 SSH Protocol Fuzz Tests

```

// test/unit/features/ssh_session/ssh_protocol_fuzz_test.dart
void main() {
    group('SSH packet parsing fuzz', () {
        late MockSshTransport mockTransport;
        late SshPacketParser parser;

```

```

setUp(() {
  mockTransport = MockSshTransport();
  parser = SshPacketParser(transport: mockTransport);
});

test('handles malformed SSH banner gracefully', () {
  // Fuzz: random bytes as SSH banner
  final random = Random();
  for (int i = 0; i < 1000; i++) {
    final malformedBanner = List.generate(
      random.nextInt(256),
      (_) => random.nextInt(256),
    );
    expect(
      () =>
        parser.parseBanner(Uint8List.fromList(malformedBanner)),
      returnsNormally, // should not throw, should return
                        error result
    );
  }
});

test('handles truncated KEX_INIT packet', () {
  // KEX_INIT with insufficient bytes
  final truncated = Uint8List.fromList([0x14, 0x00]); //
  SSH_MSG_KEXINIT = 20
  final result = parser.parseKexInit(truncated);
  expect(result.isLeft(), isTrue);
  expect(result.fold((l) => l, (r) => null),
    isA<SshProtocolError>());
});

test('handles sequence number wraparound', () {
  // Simulate 4294967295 packets to test sequence number
  // overflow
  final sequenceManager = SshSequenceNumberManager();
  for (int i = 0; i < 0xFFFFFFFF; i += 1000000) {
    sequenceManager.setSequence(i);
  }
  sequenceManager.setSequence(0xFFFFFFFF);
  sequenceManager.increment(); // Should wrap to 0
  expect(sequenceManager.current, equals(0));
});
});
}

```

9.9 Offline Mode Sync Conflict Tests

```
// test/unit/core/conflict_resolver_test.dart
void main() {
  late ConflictResolver resolver;

  setUp(() {
    resolver = ConflictResolver();
  });

  group('Host entity conflict resolution', () {
    test('server wins for concurrent label edits (timestamp-based)', () {
      final local = HostEntity(
        id: 'h1',
        label: 'Local Label',
        hostname: 'server.example.com',
        port: 22,
        username: 'admin',
        protocol: HostProtocol.ssh,
        updatedAt: DateTime(2026, 1, 1, 12, 0, 0),
        createdAt: DateTime(2026, 1, 1),
      );

      final remote = HostEntity(
        id: 'h1',
        label: 'Remote Label',
        hostname: 'server.example.com',
        port: 22,
        username: 'admin',
        protocol: HostProtocol.ssh,
        updatedAt: DateTime(2026, 1, 1, 13, 0, 0), // newer
        createdAt: DateTime(2026, 1, 1),
      );

      final resolved = resolver.resolveHostConflict(local: local,
        remote: remote);
      expect(resolved.label, equals('Remote Label'));
    });

    test('local wins when local timestamp is newer', () {
      final local = HostEntity(
        id: 'h1',
        label: 'Local Updated',
        hostname: 'server.example.com',

```



```

        port: 22,
        username: 'admin',
        protocol: HostProtocol.ssh,
        updatedAt: DateTime(2026, 1, 1, 14, 0, 0), // newer
        locally
        createdAt: DateTime(2026, 1, 1),
    );

    final remote = HostEntity(
        id: 'h1',
        label: 'Remote Stale',
        hostname: 'server.example.com',
        port: 22,
        username: 'admin',
        protocol: HostProtocol.ssh,
        updatedAt: DateTime(2026, 1, 1, 10, 0, 0),
        createdAt: DateTime(2026, 1, 1),
    );

    final resolved = resolver.resolveHostConflict(local: local,
        remote: remote);
    expect(resolved.label, equals('Local Updated'));
});

test('deletion remote takes priority over local edit', () {
    final localEdit = HostEntity(
        id: 'h1',
        label: 'Edited Host',
        hostname: 'server.example.com',
        port: 22,
        username: 'admin',
        protocol: HostProtocol.ssh,
        updatedAt: DateTime(2026, 1, 1, 13, 0, 0),
        createdAt: DateTime(2026, 1, 1),
    );

    final remoteDeleted = SyncTombstone(
        entityId: 'h1',
        entityType: 'host',
        deletedAt: DateTime(2026, 1, 1, 12, 0, 0),
        deletedBy: 'user_remote',
    );

    // Remote deletion wins regardless of local edit timestamp
    final resolved = resolver.resolveWithTombstone(

```

```

        localEntity: localEdit,
        tombstone: remoteDeleted,
        policy: ConflictPolicy.remoteDeletionWins,
    );

    expect(resolved, isNull); // entity should be deleted
                              locally
  });
});
}

```

9.10 Accessibility Tests

```

// test/widget/accessibility/host_list_accessibility_test.dart
import 'package:flutter_test/flutter_test.dart';

void main() {
  testWidgets('HostListPage meets accessibility guidelines',
    (tester) async {
    final SemanticsHandle handle = tester.ensureSemantics();

    await tester.pumpWidget(buildHostListWithData());
    await tester.pumpAndSettle();

    // Check for accessibility violations
    await expectLater(tester,
      meetsGuideline(androidTapTargetGuideline));
    await expectLater(tester,
      meetsGuideline(iOSTapTargetGuideline));
    await expectLater(tester,
      meetsGuideline(labeledTapTargetGuideline));
    await expectLater(tester,
      meetsGuideline(textContrastGuideline));

    handle.dispose();
  });

  testWidgets('Terminal screen has semantic labels', (tester)
    async {
    await tester.pumpWidget(buildTerminalPage());
    await tester.pumpAndSettle();

    expect(
      tester.getSemantics(find.byKey(const
        Key('terminal_input'))),
      matchesSemantics(
        label: 'Terminal input',

```

```

        isTextField: true,
        isFocusable: true,
    ),
);

expect(
  tester.getSemantics(find.byKey(const
    Key('disconnect_button'))),
  matchesSemantics(
    label: 'Disconnect from server',
    isButton: true,
    isFocusable: true,
  ),
);
});
}

```

10. Accessibility

10.1 WCAG 2.1 AA Compliance

HelixTerminator targets WCAG 2.1 Level AA compliance across all supported platforms. Compliance is verified at build time via automated accessibility checks integrated into the CI/CD pipeline.

10.1.1 Perceivable

1.1 Text Alternatives: Every non-text element carries a programmatic text alternative.

- All icons use `Semantics(label: '...')` wrappers or `Tooltip` widgets.
- Images and illustrations carry `Image(semanticLabel: '...')` descriptions.
- Terminal output is exposed to screen readers via `AccessibilityNode` with appropriate live region semantics (`liveRegion: LiveRegionMode.polite` for normal output, `assertive` for error output).
- Status indicators (connection dot, transfer progress) carry both visual and textual representation.

1.2 Time-based Media: Not applicable for primary use case. Session recordings exported as video files will carry captions when speech is present.

1.3 Adaptable:

- Layout is entirely widget-based, allowing screen readers to traverse the widget tree in logical reading order.
- MergeSemantics and ExcludeSemantics are used to group and simplify complex composite widgets.
- Semantics(sortKey: OrdinalSortKey(...)) enforces correct traversal order in drag-and-drop interfaces.
- Headings are marked with Semantics(header: true).

1.4 Distinguishable:

- All text/background combinations meet 4.5:1 contrast ratio for normal text and 3:1 for large text (≥ 18 pt normal or ≥ 14 pt bold).
- Color is never the sole means of conveying information. Status indicators pair color with icon shape and/or text label.
- Text can be resized up to 200% without loss of content or functionality (Flutter text scale factor support).
- Reflow: content reflows correctly at all zoom levels using LayoutBuilder and adaptive breakpoints.
- Non-text contrast: UI component borders and focus indicators meet 3:1 contrast ratio.

```
// Accessible color token example
class AccessibleColors {
  // All tokens verified against WCAG 2.1 AA
  static const textPrimary = Color(0xFF1A1A1A);           // on
    white: 16.1:1
  static const textSecondary = Color(0xFF595959);         // on
    white: 7.0:1
  static const textMuted = Color(0xFF767676);            // on
    white: 4.54:1 (AA)
  static const success = Color(0xFF2E7D32);              // on
    white: 7.5:1
  static const warning = Color(0xFF795548);               // on
    white: 5.1:1
  static const error = Color(0xFFC62828);                // on
    white: 7.1:1
  static const linkDefault = Color(0xFF0D47A1);          // on
    white: 8.6:1

  // Focus ring - always visible against both light and dark
  // backgrounds
```

```

    static const focusRing = Color(0xFF1565C0);           // 5.9:1
    on white, 3.1:1 on #333
}

```

10.1.2 Operable

2.1 Keyboard Accessible:

All functionality is operable via keyboard alone. No keyboard trap exists anywhere in the application. The terminal widget requires special handling because it captures all keystrokes by design; the escape key sequence (default: Ctrl+Shift+X) always transfers focus out of the terminal and back to the application chrome.

```

// KeyboardTrapHandler – prevents terminal from indefinitely
// capturing focus

```

```

class KeyboardTrapHandler extends StatefulWidget {
  final Widget child;
  final VoidCallback onEscapeFocusTrap;

  const KeyboardTrapHandler({
    required this.child,
    required this.onEscapeFocusTrap,
    super.key,
  });
}

```

@override

```

State<KeyboardTrapHandler> createState() =>
  _KeyboardTrapHandlerState();

```

```

}

```

```

class _KeyboardTrapHandlerState extends
  State<KeyboardTrapHandler> {

```

@override

```

Widget build(BuildContext context) {
  return Focus(
    onKey: (node, event) {
      if (event is RawKeyDownEvent &&
        event.isControlPressed &&
        event.isShiftPressed &&
        event.logicalKey == LogicalKeyboardKey.keyX) {
        widget.onEscapeFocusTrap();
        return KeyEventResult.handled;
      }
      return KeyEventResult.ignored;
    },
  ),
}

```

```

        child: widget.child,
      );
    }
  }
}

```

Full keyboard navigation map:

Key	Action
Tab	Move focus to next interactive element
Shift+Tab	Move focus to previous interactive element
Enter / Space	Activate focused button or link
Arrow keys	Navigate within list/grid
Escape	Close dialog / cancel action
Ctrl+Shift+X	Escape terminal keyboard trap
Ctrl+Tab	Switch between terminal tabs
F6	Rotate focus between split panes
Ctrl+Shift+H	Return to host list from anywhere
Ctrl+Shift+S	Open SFTP browser
Ctrl+Shift+P	Open AI command palette
? (in host list)	Show keyboard shortcuts cheat sheet

2.2 Enough Time: No time limits are imposed on user interactions. Session timeout warnings appear 5 minutes before expiry with clear options to extend. Auto-lock countdown displays a progress indicator and “Stay unlocked” button.

2.3 Seizures and Physical Reactions: No flashing content exceeds 3 flashes per second. All animations respect `MediaQuery.of(context).disableAnimations` and the OS-level “Reduce Motion” preference.

```

// Animation-aware wrapper respects system reduce-motion setting
class MotionSensitiveAnimation extends StatelessWidget {
  final Widget child;
  final Widget reducedMotionChild;

  const MotionSensitiveAnimation({
    required this.child,
    required this.reducedMotionChild,
    super.key,
  });
}

```

```

@override
Widget build(BuildContext context) {
  final reduceMotion = MediaQuery.of(context).disableAnimations;
  return reduceMotion ? reducedMotionChild : child;
}
}

```

2.4 Navigable:

- Every page has a descriptive title announced to screen readers: `Semantics(namesRoute: true, label: 'Host List – HelixTerminator')`.
- Skip navigation links are provided: a visually hidden “Skip to terminal” link receives keyboard focus first on the terminal page.
- Focus is managed programmatically on route transitions: `FocusScope.of(context).requestFocus(_pageFocusNode)` after navigation.
- Breadcrumb navigation is present in nested screens (e.g., Organization → Team → Vault).

2.5 Input Modalities:

- All multi-finger gesture shortcuts have keyboard equivalents.
- Drag-and-drop in SFTP browser has a keyboard-accessible alternative (context menu → Copy/Move).
- Touch targets are minimum 44×44dp on mobile (iOS HIG) and 48×48dp on Android (Material guidelines).
- Pointer precision is not required for any core function.

10.1.3 Understandable

3.1 Readable:

- App language is declared to the OS: `MaterialApp(locale: Locale('en', 'US'), ...)`.
- Technical abbreviations (e.g., “SFTP”, “MFA”) are expanded on first use in all user-facing content.
- Error messages are written in plain language: “Your session timed out. Please log in again.” instead of “403 Forbidden.”

3.2 Predictable:

- Navigation is consistent throughout the app. The sidebar always appears on the left (or bottom on mobile).
- Context-sensitive actions change dynamically, but their location within the UI remains fixed.

- Focus never moves unexpectedly except in response to user-initiated actions.

3.3 Input Assistance:

- All form fields carry labels (not just placeholders).
- Inline validation provides real-time, specific error messages.
- Password strength meters include both visual bar and textual description (“Weak”, “Fair”, “Strong”, “Very Strong”).
- Error messages identify the field and describe the fix: “Hostname is required.” not just “Invalid input.”

```
// Accessible form field with persistent label and error
class AccessibleTextField extends StatelessWidget {
  final String label;
  final String? error;
  final TextEditingController controller;
  final TextInputType keyboardType;
  final bool obscureText;

  const AccessibleTextField({
    required this.label,
    required this.controller,
    this.error,
    this.keyboardType = TextInputType.text,
    this.obscureText = false,
    super.key,
  });

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(
          label,
          style: Theme.of(context).textTheme.labelMedium,
        ),
        const SizedBox(height: 4),
        TextFormField(
          controller: controller,
          keyboardType: keyboardType,
          obscureText: obscureText,
          decoration: InputDecoration(
            border: const OutlineInputBorder(),
```



```

        errorText: error,
        // errorText renders as both red text AND is announced
        by screen readers
      ),
      // Semantic label combines field label + current error
      for screen reader
      semanticsLabel: error != null ? '$label: $error' :
      label,
    ),
  ],
);
}
}

```

10.1.4 Robust

4.1 Compatible:

- Semantic widget tree is compatible with TalkBack (Android), VoiceOver (iOS/macOS), NVDA (Windows), and Orca (Linux).
- Flutter’s accessibility layer maps widget semantics to platform-native accessibility APIs automatically.
- Custom interactive widgets always implement GestureDetector with onTap (not just onTapDown) to ensure screen reader activation works correctly.
- SemanticsService.announce is used for dynamic content changes (e.g., “Connected to server.example.com” after successful SSH connection).

10.2 Screen Reader Support

10.2.1 TalkBack (Android)

- Touch exploration enabled: all interactive areas are discoverable by touch.
- Swipe-to-navigate traverses elements in logical order.
- Grouping: related elements (e.g., host card: name + status + last connected) are merged into a single focusable unit using MergeSemantics.
- Terminal output: live region announces last output line when screen reader is active.
- Gesture shortcuts: TalkBack local context menus expose host actions (Connect, Edit, Delete) as menu items.

```

// Host card with merged semantics for screen readers
class AccessibleHostCard extends StatelessWidget {
  final HostEntity host;
  final VoidCallback onTap;

  const AccessibleHostCard({
    required this.host,
    required this.onTap,
    super.key,
  });

  @override
  Widget build(BuildContext context) {
    return MergeSemantics(
      child: Semantics(
        label: '${host.label}, ${host.hostname}, '
          'port ${host.port}, '
          '${host.isOnline ? "Online" : "Offline"}',
        'last connected $
          {_formatRelativeTime(host.lastConnectedAt)}',
        button: true,
        onTap: onTap,
        child: _buildCardVisual(context),
      ),
    );
  }

  String _formatRelativeTime(DateTime? dt) {
    if (dt == null) return 'never';
    final diff = DateTime.now().difference(dt);
    if (diff.inMinutes < 60) return '${diff.inMinutes} minutes
      ago';
    if (diff.inHours < 24) return '${diff.inHours} hours ago';
    return '${diff.inDays} days ago';
  }

  Widget _buildCardVisual(BuildContext context) {
    // ... visual layout ...
    return const SizedBox();
  }
}

```

10.2.2 VoiceOver (iOS/macOS)

- Rotor support: all lists expose custom rotor actions for quick navigation by headings, links, and form fields.
- iPad: keyboard shortcuts (VoiceOver + arrow keys) navigate the split-pane terminal layout.
- macOS: the menu bar app exposes all primary actions as accessible menu items.
- Focus order in modal dialogs is trapped correctly within the dialog boundary.

10.2.3 Windows NVDA / JAWS

- ARIA-equivalent Flutter semantics map to Windows UI Automation (UIA) tree automatically.
- Virtual cursor mode works for browsing the host list and settings.
- Application mode is automatically activated in the terminal, passing raw keystrokes to NVDA's applications layer.
- Tab stop order matches visual layout.

10.2.4 Linux Orca

- AT-SPI2 bridge exposes Flutter widget tree.
- Orca flat review mode works across all screens.
- All interactive elements have SpeechAccessibilityRole equivalents exposed through AT-SPI2.

10.3 High Contrast Mode

The app automatically detects high contrast preferences via `MediaQuery.of(context).highContrast` and applies a dedicated `HighContrastTheme`.

```
// High contrast theme overrides
class AppThemes {
  static ThemeData get highContrast => ThemeData(
    colorScheme: const ColorScheme(
      brightness: Brightness.light,
      primary: Color(0xFF000080),          // Navy blue – distinct,
      high contrast
      onPrimary: Color(0xFFFFFFFF),
      secondary: Color(0xFF006400),
      onSecondary: Color(0xFFFFFFFF),
      error: Color(0xFF8B0000),
      onError: Color(0xFFFFFFFF),
    ),
  );
}
```

```

        surface: Color(0xFFFFFFFF),
        onSurface: Color(0xFF000000),
    ),
    textTheme: Typography.blackCupertino.copyWith(
        bodyMedium: const TextStyle(
            fontSize: 16,
            fontWeight: FontWeight.w600,
            color: Color(0xFF000000),
        ),
    ),
    inputDecorationTheme: const InputDecorationTheme(
        border: OutlineInputBorder(
            borderSide: BorderSide(color: Color(0xFF000000), width:
                2),
        ),
        focusedBorder: OutlineInputBorder(
            borderSide: BorderSide(color: Color(0xFF000080), width:
                3),
        ),
    ),
);

static ThemeData get highContrastDark => ThemeData(
    colorScheme: const ColorScheme(
        brightness: Brightness.dark,
        primary: Color(0xFF00FFFF),           // Cyan on black
        onPrimary: Color(0xFF000000),
        secondary: Color(0xFFFFFFFF00),
        onSecondary: Color(0xFF000000),
        error: Color(0xFFFF4444),
        onError: Color(0xFF000000),
        surface: Color(0xFF000000),
        onSurface: Color(0xFFFFFFFF),
    ),
);
}

```

High contrast terminal color schemes override the selected theme:

```

class TerminalHighContrastTheme extends TerminalTheme {
    @override
    Color get background => const Color(0xFF000000);
    @override
    Color get foreground => const Color(0xFFFFFFFF);
    @override

```

```

Color get cursor => const Color(0xFFFFF00);
@override
Color get selectionBackground => const Color(0xFF0000FF);
@override
Color get selectionForeground => const Color(0xFFFFFFFF);

// ANSI colors – all ensure 7:1+ contrast against background
@override
Color get black => const Color(0xFF000000);
@override
Color get red => const Color(0xFFFF4444);
@override
Color get green => const Color(0xFF44FF44);
@override
Color get yellow => const Color(0xFFFFF00);
@override
Color get blue => const Color(0xFF4444FF);
@override
Color get magenta => const Color(0xFFFF44FF);
@override
Color get cyan => const Color(0xFF44FFFF);
@override
Color get white => const Color(0xFFFFFFFF);

// Bright variants are identical in high contrast mode
@override Color get brightBlack => const Color(0xFF666666);
@override Color get brightRed => const Color(0xFFFF6666);
@override Color get brightGreen => const Color(0xFF66FF66);
@override Color get brightYellow => const Color(0xFFFFFF66);
@override Color get brightBlue => const Color(0xFF6666FF);
@override Color get brightMagenta => const Color(0xFFFF66FF);
@override Color get brightCyan => const Color(0xFF66FFFF);
@override Color get brightWhite => const Color(0xFFFFFFFF);
}

```

10.4 Focus Management

Focus management is critical for keyboard and screen reader users. All route transitions, modal opens/closes, and dynamic content updates manage focus explicitly.

```

// FocusManager utility
class HelixFocusManager {
  /// Call after navigating to a new route.

```

```

/// Announces the route name and moves focus to the page's first
/// interactive element.
static void onRouteChange(BuildContext context, String
    routeTitle) {
  WidgetsBinding.instance.addPostFrameCallback((_) {
    SemanticsService.announce(routeTitle, TextDirection.ltr);
    // Find the first tabbable element in the new route
    final firstFocus =
      FocusScope.of(context).traversalDescendants
        .where((n) => n.canRequestFocus)
        .firstOrNull;
    firstFocus?.requestFocus();
  });
}

/// Call when opening a modal / bottom sheet.
/// Saves the trigger element's focus node to restore on close.
static FocusNode? _savedFocus;

static void onModalOpen(BuildContext context) {
  _savedFocus = FocusManager.instance.primaryFocus;
}

/// Call when closing a modal.
static void onModalClose() {
  WidgetsBinding.instance.addPostFrameCallback((_) {
    _savedFocus?.requestFocus();
    _savedFocus = null;
  });
}
}

```

10.5 Semantic Labels

Every interactive element in the application carries a semantic label. The following table defines the labeling convention:

Widget Type	Labeling Strategy	Example
Icon button	<code>Semantics(label: 'Action name')</code>	<code>label: 'Delete host'</code>
Image	<code>Image(semanticLabel: '...')</code>	<code>semanticLabel: 'SSH key fingerprint QR code'</code>

Widget Type	Labeling Strategy	Example
Progress indicator	Semantics(label: '...', value: '...%')	label: 'Upload progress', value: '47%'
Toggle	Semantics(toggled: bool)	Checked state for checkboxes
Status badge	Semantics(label: 'Status: Connected')	Combined visual + text
Text field	Persistent label + error state	label: 'Hostname: Must not be empty'
List item	MergeSemantics wrapping all sub-elements	Full card announced as one unit
Chart	Semantics(label: descriptive summary)	label: 'Transfer speed: 24.3 MB/s, 5-second history'
Terminal	liveRegion + custom announcements	Last line + connection status changes

10.6 Keyboard-Only Navigation Map

Complete keyboard navigation is possible through all screens using standard keyboard interactions.

Application-level:

- Ctrl+, (comma) → Open Settings
- Ctrl+H → Host List
- Ctrl+W → Close current tab
- Ctrl+T → New terminal tab
- Ctrl+N → New connection dialog
- Ctrl+Shift+F → Global search

Host List:

- / → Focus search field
- Arrow Up/Down → Navigate host list
- Enter → Open selected host
- Space → Toggle selection
- Ctrl+Enter → Connect to selected host

Delete	→ Delete selected host (with confirmation)
Ctrl+G	→ Create new group
Ctrl+E	→ Edit selected host

Terminal:

Ctrl+C	→ Send SIGINT (not copy – use Ctrl+Shift+C for copy)
Ctrl+Shift+C	→ Copy selection
Ctrl+Shift+V	→ Paste
Ctrl+Shift+F	→ Find in terminal
Ctrl+Shift+Up/Down	→ Scroll terminal one line
PgUp/PgDown	→ Scroll terminal one page
Ctrl+Shift+Home	→ Scroll to top
Ctrl+Shift+End	→ Scroll to bottom
Ctrl+Shift+X	→ Release terminal keyboard focus to app

chrome

Ctrl+D	→ Send EOF
Ctrl+Z	→ Send SIGTSTP

SFTP Browser:

Arrow keys	→ Navigate file list
Enter	→ Open directory / download file
Backspace	→ Navigate to parent directory
F2	→ Rename selected item
F5	→ Refresh directory listing
Delete	→ Delete selected item (with confirmation)
Ctrl+A	→ Select all
Ctrl+D	→ Download selected
Ctrl+U	→ Upload files
Ctrl+Shift+N	→ New folder

10.7 Dynamic Type and Text Scaling

HelixTerminator fully supports system font size preferences.

```
// Responsive text scaling
class AppTextScaler extends StatelessWidget {
  final Widget child;

  const AppTextScaler({required this.child, super.key});

  @override
  Widget build(BuildContext context) {
    // Clamp text scale between 0.8 and 2.0 to prevent layout
    // breakage
```



```

// while supporting large text needs
return MediaQuery.withClampedTextScaling(
  minScaleFactor: 0.8,
  maxScaleFactor: 2.0,
  child: child,
);
}
}

```

Terminal font size is independent of system text scale — it follows the user’s explicit terminal font size setting — but the application chrome (navigation, dialogs, labels) scales with system preferences.

10.8 Reduced Motion

All animations check for the system “Reduce Motion” preference.

```

// AnimationConfig respects system preferences
class AnimationConfig {
  static Duration pageFadeIn(BuildContext context) =>
    MediaQuery.of(context).disableAnimations
      ? Duration.zero
      : const Duration(milliseconds: 250);

  static Duration listItemSlideIn(BuildContext context) =>
    MediaQuery.of(context).disableAnimations
      ? Duration.zero
      : const Duration(milliseconds: 200);

  static Duration connectionPulse(BuildContext context) =>
    MediaQuery.of(context).disableAnimations
      ? Duration.zero
      : const Duration(milliseconds: 1500);

  // Terminal cursor blink respects reduce motion
  static Duration cursorBlink(BuildContext context) =>
    MediaQuery.of(context).disableAnimations
      ? const Duration(seconds: 999999) // effectively
        disabled
      : const Duration(milliseconds: 530);
}

```

11. Auto-Update Mechanism

No client-side update mechanism was previously specified for any of the six release platforms. This section defines release channels, staged rollout, per-platform code-signing/notarization, the forced-security-patch path, and rollback — the full lifecycle an installed client goes through from “new build published” to “running on every user’s device.”

11.1 Release Channels

Two channels, both fed from the same CI pipeline (doc 03) at different promotion gates:

Channel	Audience	Promotion trigger	Typical cadence
stable	All users (default)	Manual promotion from beta after $\geq 72\text{h}$ with no P0/P1 regression reported against it	Every 2–4 weeks
beta	Opt-in users (Settings → Advanced → “Join beta channel”)	Automatic on every merge to release/* that passes the full CI gate (doc 03)	Every 3–7 days

The channel is a client-persisted preference (HelixConfig field, Appendix C) checked by the update-check request (GET /v1/client/manifest?channel=stable&platform=android&version=1.4.2); switching from beta to stable mid-cycle never downgrades an already-installed beta build — the user is offered the next stable build that supersedes their current version once it ships.

11.2 Staged Rollout

Every stable-channel release ships to a percentage of the eligible install base at a time, not all at once, so a defect surfaces against a bounded blast radius before full exposure:

Stage	% of stable install base	Minimum soak time before next stage	Auto-halt condition
Canary	1%	4h	Crash-free-session rate < 99.5% (Crashlytics, §11.4.47)
Early	10%	12h	Crash-free-session rate < 99.7%, or any new P0 issue opened
Broad	50%	24h	Crash-free-session rate < 99.8%
Full	100%	—	—

Rollout percentage is server-controlled (the manifest endpoint returns “update available” to a deterministic, stable hash-bucketed subset of installIds, not a per-request random roll — so a given device’s inclusion/exclusion is consistent across repeated checks within a stage) and is independent of the app-store-level staged-rollout controls Google Play / App Store separately offer (§11.3) — for store-distributed platforms, the store’s own staged rollout and this server-side gate compose (an update is only offered at all once BOTH the store rollout and this percentage admit the requesting device).

Any auto-halt condition immediately freezes the rollout percentage (no further devices admitted to the new version) and pages the on-call rotation (doc 04); resuming requires an explicit human decision, never an automatic retry.

11.3 Per-Platform Distribution, Code-Signing, and Notarization

Platform	Distribution path	Signing / notarization	Update transport
Android	Google Play (primary) + direct-APK sideload	App signed with Play App Signing key (Google-held)	Play Store’s own update mechanism for

Platform	Distribution path	Signing / notarization	Update transport
iOS / iPadOS	channel (signed, checksummed)	for Play builds; a separate release keystore (org-held, HSM-backed, rotated per \$5 key-management policy) signs the sideload APK	Play installs; in-app manifest-check + download + PackageInstaller session API for sideload installs
	Apple App Store only (no sideloading in scope)	Apple-issued distribution certificate + provisioning profile; App Store notarization/review is Apple's own gate, not ours	App Store's own update mechanism exclusively — the in-app manifest-check still runs (to drive the “forced security update” UI in §11.4) but iOS builds can never self-install; the check can only deep-link to the App Store update page
	Direct download (primary) + Mac App Store (secondary, store-gated like iOS)	Direct-download builds: Apple Developer ID signing + notarytool notarization + stapled ticket (required for Gatekeeper to allow launch without a manual override); Mac App Store builds: Apple's own signing/sandboxing	In-app updater downloads the notarized .dmg/.zip, verifies the stapled notarization ticket before mounting, then hands off to the OS installer flow
Windows	Direct download (primary) +	Direct-download builds: EV code-signing certificate	In-app updater downloads the signed installer/

Platform	Distribution path	Signing / notarization	Update transport
Linux	Microsoft Store (secondary)	(Authenticode), timestamped so signatures remain valid after cert expiry; MSIX package for Store builds	MSIX delta, verifies the Authenticode signature chain before executing
	Direct .deb/.rpm/ AppImage download + optional distro package-manager channel (best-effort, community-maintained)	Packages signed with a GPG release key published at a well-known URL (https://helixterminator.io/release.gpg); package-manager repos (apt/dnf) verify against that key at install/upgrade time	AppImage/direct builds: in-app updater verifies the GPG signature over the downloaded artifact before replacing the running binary; package-manager installs: the OS's own package manager handles the update, in-app updater only surfaces "update available via your package manager"
			No update mechanism needed — every page load fetches the current deployed build (service-worker cache, §8.6, is invalidated by content-hashed asset filenames on every deploy,
Web (WASM)	Served directly from helixterminator.io	TLS-only (§7.1); no binary signing applies to a served-not-installed artifact	

Platform	Distribution path	Signing / notarization	Update transport
			so a stale cached build is never served past one reload)

11.4 Forced Security-Patch Path

A release manifest entry MAY be marked `severity: security-critical`. This bypasses the normal staged-rollout percentage gate (§11.2) — a security-critical patch is offered to 100% of eligible devices on the FIRST manifest check after publish, on every platform, subject only to each platform’s own distribution mechanics (§11.3):

- **Android (sideload) / macOS / Windows / Linux (direct-download):** the in-app updater shows a non-dismissible “Critical security update required” screen once the current session’s active work is safely at a pausable point (never interrupts an in-flight SSH keystroke or SFTP transfer mid-byte — it waits for the next idle point, checked every 30s) and blocks continued use of network-facing features (SSH connect, SFTP, collaboration) until the update installs; local-only features (viewing cached host list, editing snippets offline) remain available so the block is not a full app lockout.
- **Android (Play) / iOS / macOS (App Store):** the same non-dismissible screen deep-links to the store’s update page (the client cannot self-install on these paths); the network-facing feature block is still enforced client-side while the store update has not yet been applied.
- **Web:** the next page load already serves the patched build (§11.3); no separate forced-update UI is needed, but an already-open tab with a stale in-memory build receives a manifest WebSocket push (piggybacking the collaboration channel’s transport, §1.16, on a reserved system topic) prompting an immediate reload for any session with network-facing features active.

A security-critical manifest entry MUST cite a tracked vulnerability/incident record (per this project’s governance) so a forced update is never issued without an auditable justification.

11.5 Rollback

If a rollout stage's auto-halt condition fires (§11.2), or a `stable` release is manually identified as regressed after full rollout, the manifest endpoint is updated to advertise the **previous** `stable` version as the current target for any device already on the regressed version — this is a manifest-side rollback (steering devices back), not an uninstall/reinstall:

- Devices that have not yet updated past the last-known-good version simply never receive the regressed manifest entry (the rollout was halted before reaching them, or the endpoint change landed before their next check).
 - Devices already on the regressed version receive a normal (non-forced, unless the regression is itself security-critical) update offer back to the last-known-good version on their next manifest check.
 - Local data (SQLite schema, secure-storage vault) is designed forward-compatible-then-back-compatible across one version step (a drift migration **MUST** provide a `down()` path for the immediately-preceding schema version, not only `up()`) specifically so a rollback never corrupts or strands locally-cached data.
 - Store-distributed platforms (iOS App Store, Android Play, macOS App Store) cannot be server-side rolled back the way direct-download platforms can — Apple/Google review a new build submission even for a revert; on these paths the client-side network-feature block (§11.4's mechanism, reused here) is the interim mitigation while a reverting build clears store review, and the incident is tracked exactly as any other release-blocking regression (per this project's governance).
-

Appendix A: Dart Package Dependencies

`pubspec.yaml` – complete dependency list

```
name: helix_client
description: HelixTerminator SSH client
version: 1.0.0+1
publish_to: none

environment:
  sdk: '>=3.3.0 <4.0.0'
  flutter: '>=3.24.0'
```

```
dependencies:
  flutter:
    sdk: flutter

  # SSH & Terminal
  dartssh2: ^2.9.0
  xterm: ^3.6.0

  # State management
  flutter_bloc: ^8.1.6
  bloc: ^8.1.4
  equatable: ^2.0.5

  # Dependency injection
  get_it: ^7.7.0
  injectable: ^2.4.2

  # Navigation
  go_router: ^14.2.7

  # Networking
  dio: ^5.5.0
  web_socket_channel: ^3.0.1
  connectivity_plus: ^6.0.3

  # Local storage
  drift: ^2.18.0
  sq_lite3_flutter_libs: ^0.5.21
  flutter_secure_storage: ^9.2.2
  path_provider: ^2.1.3

  # Cryptography
  pointycastle: ^3.9.1
  cryptography: ^2.7.0
  argon2_ffl: ^1.0.0

  # Authentication
  local_auth: ^2.3.0
  flutter_web_auth_2: ^4.0.0

  # Platform & System
  path: ^1.9.0
  uuid: ^4.4.0
  intl: ^0.19.0
```



```
  rxdart: ^0.28.0
  collection: ^1.18.0
  async: ^2.11.0

# UI components
flutter_svg: ^2.0.10+1
cached_network_image: ^3.4.1
shimmer: ^3.0.0
lottie: ^3.1.2

# File system
file_picker: ^8.1.2
open_file: ^3.3.2
share_plus: ^10.0.2
desktop_drop: ^0.4.4

# Notifications
flutter_local_notifications: ^17.2.2
firebase_messaging: ^15.1.3

# Analytics & Crash reporting
firebase_crashlytics: ^4.1.3
firebase_analytics: ^11.3.3

# Utils
dartz: ^0.10.1
logger: ^2.4.0
json_annotation: ^4.9.0
freezed_annotation: ^2.4.1

dev_dependencies:
  flutter_test:
    sdk: flutter

# Code generation
build_runner: ^2.4.11
injectable_generator: ^2.6.2
freezed: ^2.5.2
json_serializable: ^6.8.0
drift_dev: ^2.18.0

# Testing
bloc_test: ^9.1.7
mocktail: ^1.0.4
```

```
golden_toolkit: ^0.15.0
integration_test:
  sdk: flutter

# Linting
flutter_lints: ^4.0.0
very_good_analysis: ^6.0.0
```

Appendix B: Directory Structure

```
lib/
├─ main.dart
├─ main_development.dart
├─ main_staging.dart
├─ main_production.dart
├─
├─ app/
│   └─ app.dart # Root MaterialApp +
GoRouter setup
│   └─ app_bloc_observer.dart # BlocObserver for logging
│   └─ app_router.dart # All GoRouter routes
│   └─ app_theme.dart # Theme data
├─
├─ core/
│   └─ di/
│       └─ injection_container.dart # get_it setup
│       └─ injection_container.config.dart # generated
│   └─ error/
│       └─ failures.dart
│       └─ exceptions.dart
│   └─ network/
│       └─ dio_client.dart
│       └─ dio_interceptors.dart
│       └─ network_info.dart
│   └─ storage/
│       └─ app_database.dart # drift database
│       └─ secure_storage.dart
│       └─ daos/
│   └─ sync/
│       └─ sync_manager.dart
│       └─ conflict_resolver.dart
│       └─ sync_models.dart
```

```
|   |─ platform/
|   |   |─ platform_channel.dart
|   |   └─ channels/
|   └─ utils/
|       |─ extensions/
|       └─ validators/
|
|─ features/
|   |─ auth/
|   |   |─ data/
|   |   |   |─ datasources/
|   |   |   |─ models/
|   |   |   └─ repositories/
|   |   └─ domain/
|   |       |─ entities/
|   |       |─ repositories/
|   |       └─ usecases/
|   |   └─ presentation/
|   |       |─ bloc/
|   |       |─ pages/
|   |       └─ widgets/
|   |
|   |─ vault/
|   |   └─ ... (same structure)
|   |─ hosts/
|   |─ terminal/
|   |─ ssh_session/
|   |─ sftp/
|   |─ port_forwarding/
|   |─ workspace/
|   |─ snippets/
|   |─ keychain/
|   |─ collaboration/
|   |─ ai_autocomplete/
|   |─ session_logs/
|   |─ settings/
|   |─ organizations/
|   |─ audit/
|   |─ known_hosts/
|   └─ notifications/
|
|─ l10n/
|   |─ app_en.arb
|   └─ app_localizations.dart
```

Appendix C: Environment Configuration

```
// lib/core/config/environment.dart
enum HelixEnvironment { development, staging, production }

class HelixConfig {
  final HelixEnvironment environment;
  final String apiBaseUrl;
  final String wsBaseUrl;
  final String sentryDsn;
  final bool enableAnalytics;
  final bool enableCrashReporting;
  final Duration sessionTimeout;
  final Duration autoLockTimeout;
  final int maxScrollbackLines;
  final bool enableAiFeatures;

  const HelixConfig({
    required this.environment,
    required this.apiBaseUrl,
    required this.wsBaseUrl,
    required this.sentryDsn,
    required this.enableAnalytics,
    required this.enableCrashReporting,
    required this.sessionTimeout,
    required this.autoLockTimeout,
    required this.maxScrollbackLines,
    required this.enableAiFeatures,
  });

  static const development = HelixConfig(
    environment: HelixEnvironment.development,
    apiBaseUrl: 'https://api.dev.helixterminator.io',
    wsBaseUrl: 'wss://ws.dev.helixterminator.io',
    sentryDsn: '',
    enableAnalytics: false,
    enableCrashReporting: false,
    sessionTimeout: Duration(hours: 24),
    autoLockTimeout: Duration(minutes: 30),
    maxScrollbackLines: 50000,
    enableAiFeatures: true,
  );
```

```
static const staging = HelixConfig(  
  environment: HelixEnvironment.staging,  
  apiBaseUrl: 'https://api.staging.helixterminator.io',  
  wsBaseUrl: 'wss://ws.staging.helixterminator.io',  
  sentryDsn: 'https://sentry-staging-dsn@sentry.io/12345',  
  enableAnalytics: true,  
  enableCrashReporting: true,  
  sessionTimeout: Duration(hours: 8),  
  autoLockTimeout: Duration(minutes: 15),  
  maxScrollbackLines: 100000,  
  enableAiFeatures: true,  
);  
  
static const production = HelixConfig(  
  environment: HelixEnvironment.production,  
  apiBaseUrl: 'https://api.helixterminator.io',  
  wsBaseUrl: 'wss://ws.helixterminator.io',  
  sentryDsn: 'https://sentry-prod-dsn@sentry.io/12346',  
  enableAnalytics: true,  
  enableCrashReporting: true,  
  sessionTimeout: Duration(hours: 8),  
  autoLockTimeout: Duration(minutes: 10),  
  maxScrollbackLines: 100000,  
  enableAiFeatures: true,  
);  
}
```

End of HelixTerminator Client-Side Technical Specification v1.0
Document generated: 2026-06-28 Total estimated implementation effort:
24 person-months (team of 4 Flutter engineers)